

Ingegneria del software

Modulo 1

▼ 1.0 - Introduzione all'ingegneria del software

Introduzione all'ingegneria del software

L'ingegneria del software è una disciplina che permette di affrontare in modo sistematico e quantificabile lo sviluppo e la manutenzione dei prodotti software. Il processo di sviluppo di un software stabilisce un ordine di esecuzione delle attività, specifica quali elaborati devono essere forniti e quando, assegna le attività agli sviluppatori, fornisce criteri per monitorare il progresso dello sviluppo, misurarne i risultati, pianificare i progetti futuri e copre l'intero ciclo di vita del software.

Scrivere un buon software, per un ingegnere del software, significa:

- Trovare il miglior equilibrio tra diversi fattori: la soddisfazione dell'utente, l'organizzazione dell'applicazione, la facilità di estensione dell'applicazione, la comprensibilità delle soluzioni adottate, il sapere valutare e scegliere le giuste tecnologie e linguaggi, saper tenere conto anche del fattore tempo, ecc.
- Adottare tecniche adeguate a gestire la crescente complessità delle applicazioni.
- Utilizzare al meglio l'investimento, spesso ingente, necessario per produrre un'applicazione, garantendo in particolare il maggior tempo di vita possibile e il riutilizzo in altri progetti di parte del codice prodotto.

Linguaggio di modellazione

Durante il processo di sviluppo del software è indispensabile poter utilizzare un linguaggio per costruire i modelli da discutere con il cliente e gli altri sviluppatori: UML (Unified Modeling Language).

Design pattern

Un design pattern è una soluzione progettuale generale a un problema ricorrente, una descrizione o modello logico da applicare per la risoluzione di

un problema che può presentarsi in diverse situazioni durante le fasi di analisi e progettazione del software. Un design pattern consente di riutilizzare l'esperienza acquisita nell'affrontare e risolvere uno specifico problema progettuale in altri casi simili.

Fasi del processo di sviluppo di un software

1. Studio di fattibilità

Valutazione preliminare dei costi e dei benefici per stabilire se si debba avviare lo sviluppo dell'applicazione, in base ad alternative possibili, risorse finanziarie e umane necessarie per l'attuazione del progetto. Il risultato della fase di studio di fattibilità è un documento che dovrebbe contenere le seguenti informazioni:

- Definizione preliminare del problema.
- Possibili scenari che illustrino eventuali diverse strategie di soluzione.
- Costi, tempi e modalità di sviluppo per le diverse alternative.

2. Analisi dei requisiti

Un requisito può essere di due tipologie:

- Requisito funzionale: un comportamento del sistema.
- Requisito non funzionale: un vincolo.

Tale fase si suddivide in:

1. Formalizzazione dei requisiti che il sistema dovrà possedere per soddisfare le necessità del cliente.
2. Redazione di un documento di Specifica dei Requisiti Software (SRS – Software Requirement Specification), il quale elenca tutti i requisiti che il sistema deve soddisfare con completezza, precisione e comprensibilità sia da parte del cliente che dello sviluppatore. L'SRS è un contratto tra sviluppatori e cliente per la fornitura del sistema, il quale deve essere approvato e sottoscritto dal committente.
3. Convalida delle specifiche con il cliente, il quale deve convalidarle.

3. Analisi del problema.

L'analisi del problema parte sempre da un'attenta lettura del testo che descrive i requisiti. Da questa si individuano un insieme di primi problemi, si procede quindi ad effettuare l'analisi dei problemi avendo come obiettivo la definizione di un'architettura logica del sistema, basata sul modello del dominio. In sostanza l'architettura logica è un modello che descrive la struttura (insieme delle parti), il comportamento atteso (dal tutto e da ciascuna parte) e le interazioni così come possono essere dedotte dai requisiti, dalle caratteristiche del problema e del dominio applicativo, senza alcun riferimento alle tecnologie realizzative.

4. Progettazione

Si definisce l'architettura generale (hardware e software) del sistema. Si decompone l'architettura software in uno o più programmi eseguibili. Per ogni programma, si descrivono le funzioni che svolge e le relazioni con gli altri programmi. Si decompone ogni programma eseguibile in più moduli, per ogni modulo si descrivono le funzioni che svolge e le relazioni con gli altri moduli.

Si progettano il tipo di dato, l'ereditarietà, la gestione degli eventi, quella delle eccezioni, i design pattern, la concorrenza, il progetto di interfacce utente e di basi di dati.

Il risultato di questa fase è un documento di specifiche di progetto nel quale la definizione dell'architettura software può anche essere data in maniera rigorosa, o addirittura formale, usando opportuni linguaggi di specifica di progetto.

5. Realizzazione e collaudo dei moduli

Comprende la verifica del software realizzato tramite analisi statica, dinamica, debugging, collaudo dei moduli (unit testing, verifica che un modulo soddisfi le specifiche di progetto).

6. Integrazione e collaudo del sistema

Si integrano i singoli moduli e/o programmi tra loro e si esegue il test del sistema completo per assicurarsi che le specifiche siano soddisfatte. Il test del sistema può essere effettuato tramite alfa test, ovvero il sistema è rilasciato solo all'interno dell'organizzazione del produttore, e beta test, ovvero con un rilascio controllato a pochi e selezionati utenti del prodotto.

7. Installazione e training

Il sistema software viene consegnato al cliente, installato e messo in funzione.

8. Utilizzo e manutenzione

Il sistema viene utilizzato e mantenuto. La fase di manutenzione è la fase più lunga del ciclo di vita di un prodotto software 50-80% dei costi complessivi. La manutenzione può essere di 3 tipologie:

- Manutenzione correttiva: correzione degli errori che non sono stati scoperti nelle fasi precedenti.
- Manutenzione adattativa: aumento dei servizi forniti dal sistema in seguito alla definizione di nuovi requisiti.
- Manutenzione perfettiva: miglioramento delle caratteristiche delle unità del sistema.

Fattori di qualità del software

Il processo di revisione può avvenire a prodotto costruito o in itinere e secondo approcci a black box o a white box:

- Black box: qualità esterne percepibili da un osservatore esterno che esamina il prodotto software come se fosse una scatola nera. Queste possono essere l'affidabilità, la facilità d'uso, la velocità, ecc.
- White box: qualità interne osservabili esaminando la struttura interna del prodotto software, come modularità e leggibilità. Le qualità interne influenzano le qualità esterne.

I fattori di qualità del software sono i seguenti:

- Correttezza: data una definizione dei requisiti che il software deve soddisfare, il software si dice corretto se rispetta tali requisiti.
- Robustezza: il software si dice robusto se si comporta in maniera accettabile anche in corrispondenza di situazioni anomale e comunque non specificate nei requisiti. Nel caso di situazioni anomale, il software non deve causare disastri; o termina l'esecuzione in modo pulito, oppure entra

in una modalità particolare, in cui non sono più attive alcune funzionalità (graceful degradation mode).

- Affidabilità: correttezza + robustezza. Il software si dice affidabile se le funzionalità offerte corrispondono ai requisiti e, in caso di guasto, non produce danni fisici o economici.
- Facilità d'uso: facilità con cui l'utilizzatore del software è in grado di imparare ad usare il sistema, utilizzare il sistema, fornire i dati da elaborare, interpretare i risultati e gestire condizioni di errore.
- Efficienza: buon utilizzo delle risorse hardware (processori, memoria principale, memorie secondarie e canali di comunicazione).
- Estensibilità: facilità con cui il software può essere modificato per soddisfare nuovi requisiti. Due principi essenziali:
 - Semplicità architetturale: più l'architettura del sistema è semplice più è facile da modificare.
 - Modularità e decentralizzazione: più il sistema è suddiviso in moduli autonomi, più è facile che una modifica coinvolga un numero limitato di moduli.
- Riutilizzabilità: il software è riutilizzabile se può essere riutilizzato completamente o in parte in nuove applicazioni.
- Verificabilità: facilità con cui il prodotto software può essere sottoposto a test.
- Portabilità: facilità con cui il prodotto software può essere trasferito su altre architetture hardware e/o software.

Alcune caratteristiche sono in contrapposizione, ad esempio efficienza vs portabilità, e l'importanza dell'una o dell'altra caratteristica varia a seconda del settore applicativo.

▼ 2.0 - Principi e concetti Object-Oriented

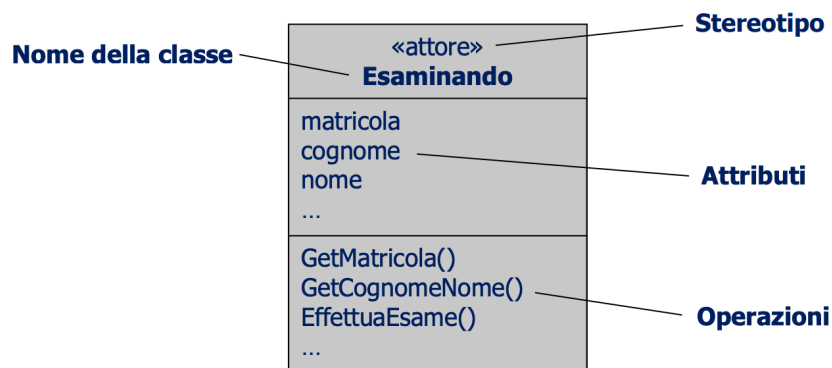
ADT

Inizialmente i linguaggi di programmazione presentavano caos sia nel flusso di controllo (istruzioni goto) che nell'accesso ai dati. Per questo motivo sono

state realizzate tipologie di programmazione più ad alto livello, come quella basata sugli oggetti. Per implementarla occorre utilizzare un ADT (Abstract Data Type), composto da un interfaccia (visibile) e un implementazione (nascosta). Questo consente di utilizzare una programmazione basata sugli oggetti che sono accessibili solo mediante l'interfaccia del suo ADT, il quale espone operazioni pubbliche applicabili ai singoli oggetti di quel tipo. Un ADT nasconde ai suoi utilizzatori tutti i dettagli della sua struttura interna e del suo funzionamento interno. Un esempio di ADT è una lista di interi, che espone metodi come add, insert o remove, e che può essere implementata in diversi modi (array, linked list, ecc.).

Notazione UML

In notazione UML una classe si rappresenta come un rettangolo diviso in 1 o 3 sezioni.



- Prima sezione

Contiene:

- Nome della classe (in grassetto + corsivo se astratta)

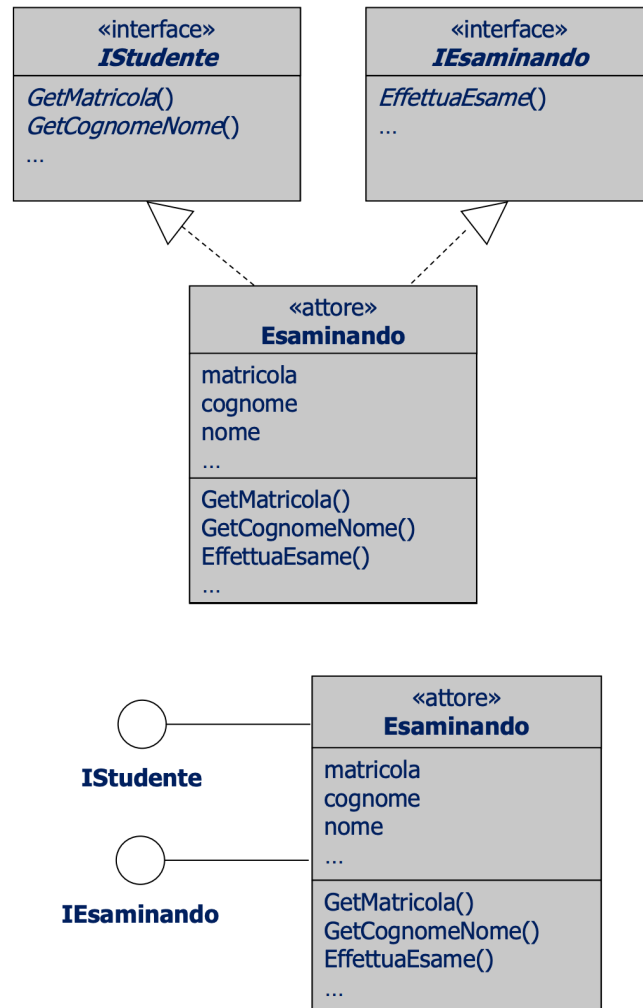
Può contenere:

- Lo stereotipo della classe
- Il nome del pacchetto

- Seconda sezione

- Attributi

- Terza sezione
 - Operazioni (in corsivo se astratte)





Ereditarietà

Le classi possono essere organizzate in una gerarchia di generalizzazione o di ereditarietà che mostra la relazione tra classi di oggetti generiche e classi di oggetti più specifiche. Gli oggetti della sottoclasse devono essere in grado di esibire tutti i comportamenti e le proprietà esibiti dagli oggetti appartenenti alla superclasse, in modo tale da poter essere "sostituiti" liberamente a questi ultimi. La sottoclasse può esibire caratteristiche aggiuntive rispetto alla superclasse ed eseguire in maniera differente alcune delle funzionalità della superclasse, a patto che questa differenza non sia osservabile dall'esterno.

Esistono diverse tipologie di ereditarietà:

- Ereditarietà di estensione o subtyping.

Una sottoclasse è un sottotipo compatibile con tutti i tipi definiti lungo la sua catena ereditaria. Questo segue la relazione "Is-A", ossia una classe derivata è un tipo del suo genitore. Esempio: se abbiamo una classe Animale e una sottoclasse Cane, possiamo dire che un Cane è un Animale (Cane is-a Animale).

- Ereditarietà di realizzazione o subclassing.

L'obiettivo è quello di riutilizzare il codice della superclasse, senza necessariamente rispettare la relazione "Is-A". Ammessa in C++, non ammessa in Java e .NET.

L'ereditarietà va usata se e solo se vale la relazione IsA, altrimenti usare la composizione, ovvero l'inserimento all'interno della struttura dati un oggetto derivante dalla classe che ci interessa utilizzare (Esempio: la classe Finestra mantiene al suo interno un oggetto Rettangolo per mantenere dati sulla sua forma e fare operazioni su di questi).

Tipologie di ereditarietà

- Ereditarietà semplice: ogni classe della gerarchia deriva da una e una sola superclasse, come in Java, oppure al più da una superclasse, come in C. In entrambe i casi la struttura che si ottiene è ad albero.
- Ereditarietà multipla: almeno una classe della gerarchia deriva da 2+ superclassi (possibile in C++). La struttura che si ottiene è un reticolo e la gestione è più complessa.

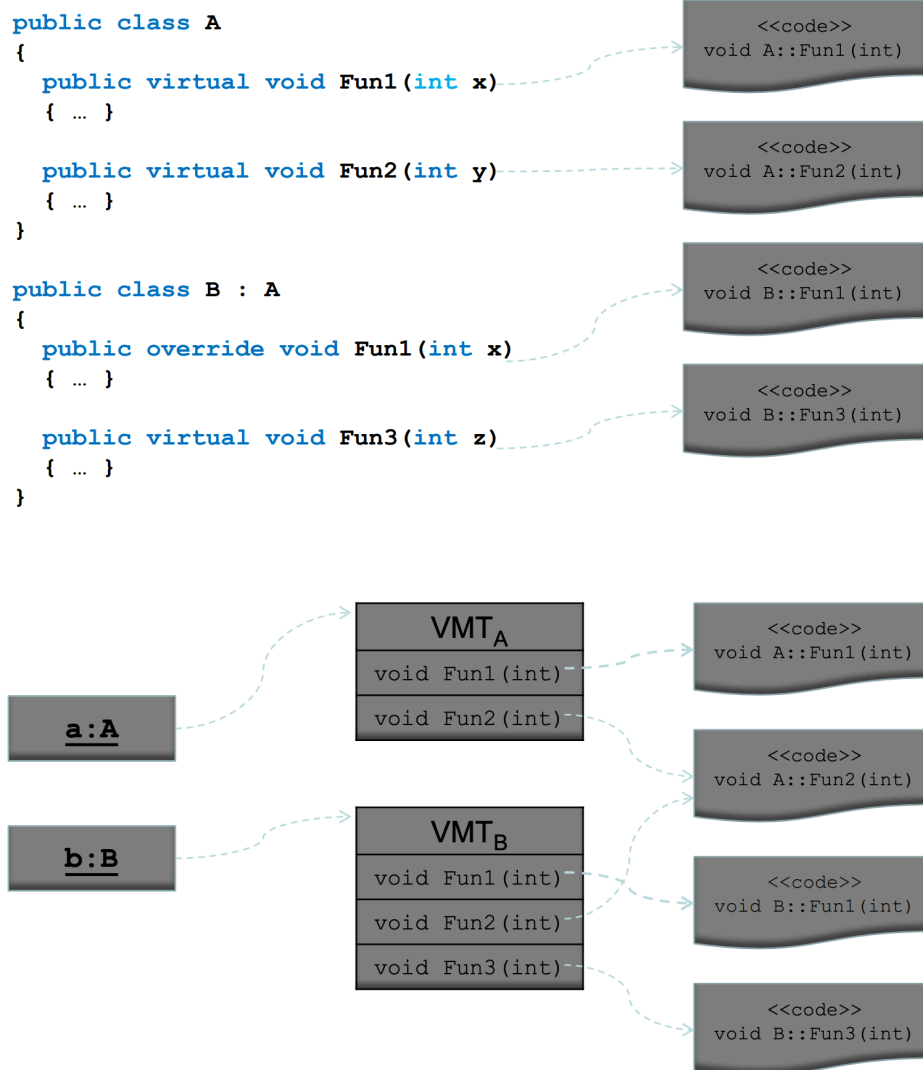
Polimorfismo

Per polimorfismo si intende la capacità della stessa cosa di apparire in forme diverse in contesti diversi, e di cose diverse di apparire sotto la stessa forma in un determinato contesto.

Ci sono diverse tipologie di polimorfismo secondo Cardelli-Wegner:

- Universale
 - Per inclusione: trattare oggetti di sottoclassi come oggetti della superclasse.

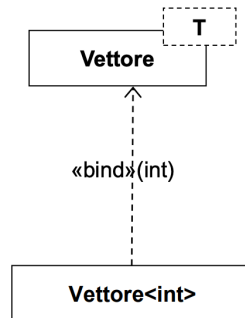
Viene utilizzata la VMT (Virtual Method Table), ovvero una tabella propria ad ogni classe che contiene metodi virtuali (ovvero metodi che possono essere sovrascritti nelle sottoclassi). Quando un oggetto di una sottoclasse invoca un metodo virtuale, la macchina virtuale o il compilatore utilizza la VMT per determinare quale metodo eseguire a runtime, in base al tipo effettivo dell'oggetto.



- Parametrico: scrivere codice generico che funziona con più tipi di dati (es. <T> in Java).
- Ad hoc
 - Overloading: definizione di più metodi con lo stesso nome, ma con parametri diversi (es. 5 + 10 e 5.2 + 11.7).
 - Coercion: un valore di un tipo può essere convertito automaticamente in un altro tipo compatibile (es. 5 + 11.7).

Classe generica rispetto ai tipi

Classe generica: classe in cui uno o più tipi sono parametrici (<T> in Java).
Ogni classe generata da una classe generica costituisce una classe indipendente (non esiste un legame di ereditarietà).



Relazioni

La maggior parte delle classi interagisce con altre classi in vari modi.
L'interazione tra entità diverse è possibile solo se tra loro esiste un qualche tipo di relazione.

Nella modellazione object-oriented le relazioni sono:

- Generalizzazione / Ereditarietà (IsA)
- Realizzazione (implements)
- Associazione
 - Associazione generica
 - Aggregazione (has)
 - Composizione (has subpart)
- Dipendenza
 - Collaborazione (usa)
 - Relazione Istanza – Classe
 - Relazione Classe – Metaclassa

In ogni tipo di relazione, esiste un cliente C che dipende da un fornitore di servizi F. C ha bisogno di F per lo svolgimento di alcune funzionalità che C non è in grado di effettuare autonomamente.

Tipo di relazione	Cliente	Fornitore
Ereditarietà	sottoclasse	superclasse
Associazione	contenitore	contenuto
Dipendenza	classe dipendente (che usa)	classe da cui si dipende (che viene usata)
	istanza	classe
	classe	metaclassa

Processo di sviluppo orientato agli oggetti

Il mondo reale è fatto di oggetti (i docenti, gli studenti, le materie d'insegnamento, ecc.). Un sistema software OO rappresenta una porzione del mondo reale o virtuale ed è composto di oggetti che interagiscono tra di loro.

Il processo di sviluppo orientato agli oggetti è composto da:

- Analisi orientata agli oggetti

L'obiettivo è quello di modellare la porzione del mondo reale o virtuale d'interesse. Gli oggetti di analisi modellano entità reali o concettuali della porzione del mondo reale d'interesse e le operazioni a esse associate. In questo contesto, ogni classe descrive una categoria di oggetti.

- Progettazione orientata agli oggetti

L'obiettivo è quello di modellare la soluzione. Gli oggetti di analisi possono subire trasformazioni e sono introdotti gli oggetti di progettazione (liste, dizionari, finestre, bottoni, tabelle, tuple, ecc.). In questo contesto, ogni classe descrive un tipo di dato.

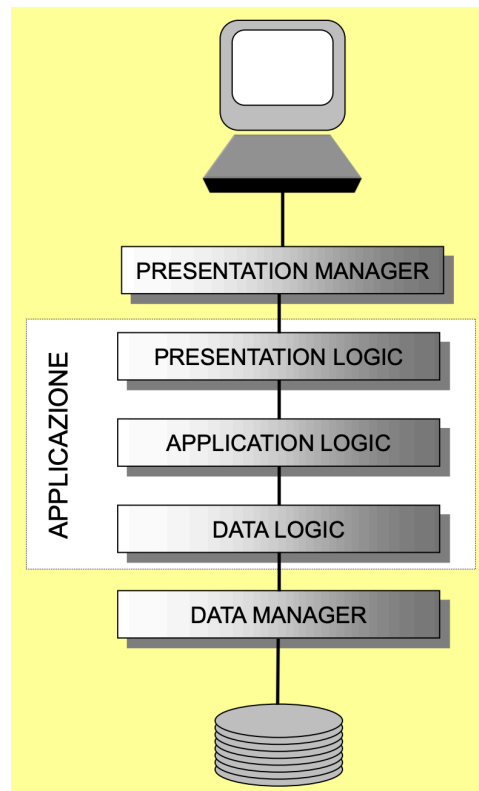
- Programmazione orientata agli oggetti

L'obiettivo è quello di realizzare la soluzione. Vengono utilizzati linguaggi di programmazione OO (C++, Java, C#) che permettono di definire le classi di oggetti e sistemi run-time che permettono di creare, utilizzare e distruggere istanze di tali classi durante l'esecuzione.

In questo contesto, ogni classe descrive l'implementazione di un tipo di dato.

▼ 3.0 - Struttura a livelli di un'applicazione

Struttura di un'applicazione



Presentation manager

Gestione dell'interazione con l'utente (Gestione dello schermo, Gestione della tastiera, Gestione del mouse ecc.).

- Interfacce grafiche, GUI (API di sistema, browser HTML)
- Interfacce a caratteri (emulatori)

Data manager

Gestione della persistenza. Può essere svolta tramite File system (API di sistema), RDBMS (DB2, Oracle, MS SQL Server, Informix, Sybase, ecc.), ecc.

Presentation logic

Gestione dell'interazione con l'utente a livello logico.

- Formattazione e visualizzazione delle informazioni
- Accettazione dei dati
- Prima validazione dei dati
- Gestione dei messaggi di errore

Application logic

Logica dell'applicazione e controllo componenti.

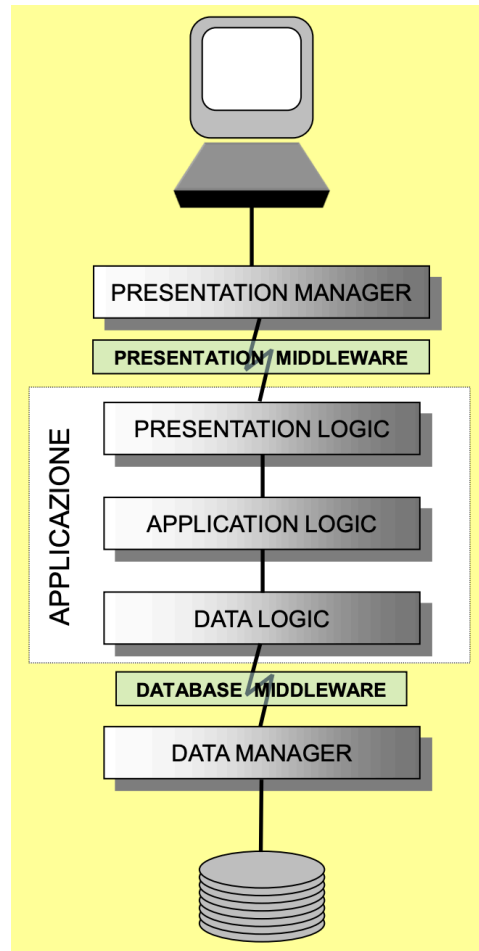
Data logic

Gestione della persistenza a livello logico.

- Consistenza dei dati
- Apertura/chiusura file, read, write e lock/unlock
- Istruzioni SQL, transazioni (commit e rollback)
- Gestione degli errori

Middleware

Il middleware è un software che permette la comunicazione tra processi. Si utilizzano API che isolano il codice dell'applicazione dai sottostanti formati e protocolli di comunicazione.



Presentation middleware

Può essere:

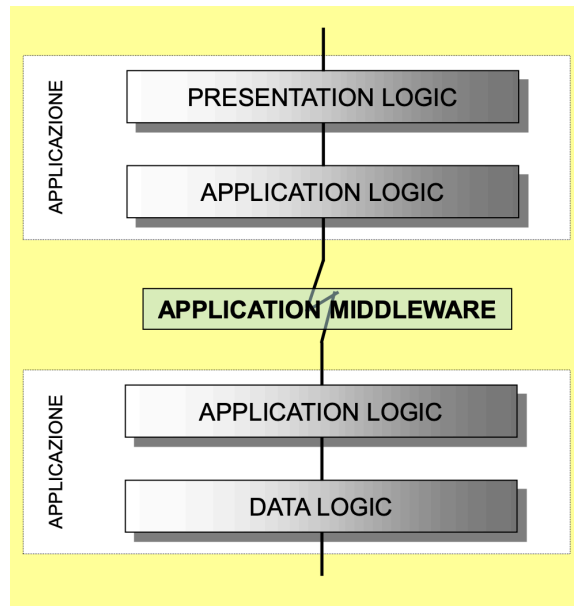
- Software di emulazione di terminali alfanumerici.
- Combinazione di Web browser (lato client), Web server e Protocollo HTTP.
- Altro.

Database middleware

Può essere:

- Software proprietario che trasferisce sulla rete le richieste SQL dall'applicazione al DBMS e i dati dal DBMS all'applicazione.
- ODBC, JDBC.
- Altro.

Application middleware



Gestisce la comunicazione tra due componenti della stessa applicazione o due tipi diversi di middleware, consentendo la presenza di meno vincoli progettuali. Può essere realizzata tramite Point-to-point (P2P), Remote procedure call (RPC), Message queuing middleware (MQM), Message oriented middleware (MOM), P2P + MQM, Object request broker (ORB), Distributed transaction monitor (DTM), ecc.

▼ 4.0 - Introduzione al framework .NET

Tecnologia COM (Component Object Model)

COM è uno standard di sistema platform-independent, distribuito, object-oriented per la creazione di componenti software binari. Specifica un modello a oggetti e requisiti di programmazione che permettono agli oggetti COM di interagire con altri oggetti, indipendente dal linguaggio di programmazione utilizzato.

```
interface IUnknown
{
    virtual HRESULT QueryInterface(IID iid, void **ppvObject) = 0;
    virtual ULONG AddRef(void) = 0;
```

```
virtual ULONG Release(void) = 0;  
};
```

- QueryInterface è utilizzata per ottenere un puntatore a un'altra interfaccia, dato un GUID che identifica univocamente tale interfaccia.
- AddRef è utilizzato dai client per indicare che un oggetto COM viene referenziato (usato).
- Release è utilizzato dai client per indicare che hanno finito di utilizzare l'oggetto COM.

Le specifiche COM richiedono l'utilizzo di una tecnica chiamata reference counting per assicurare che i singoli oggetti rimangano "vivi" fintantoché esistono client che hanno ottenuto l'accesso a una o più delle loro interfacce e, di converso, che gli stessi oggetti vengano appropriatamente cancellati quando i clienti che li utilizzavano hanno finito di usarli e non ne hanno più bisogno.

Framework .NET

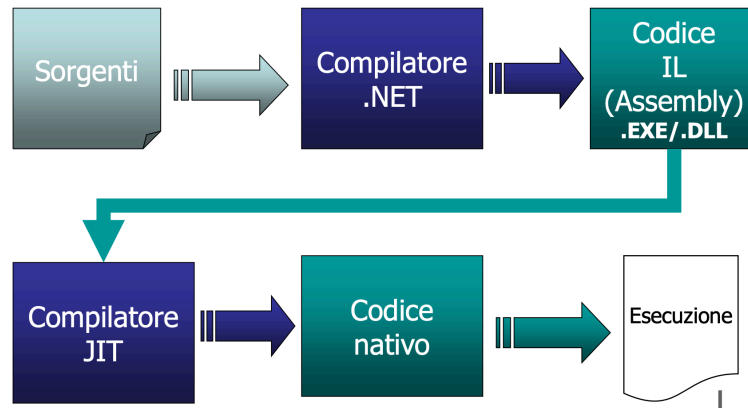
Il framework .NET fornisce un ambiente di esecuzione (runtime environment) + una libreria di classi (standard + estensioni MS), semplifica lo sviluppo e il deployment, aumenta l'affidabilità del codice e unifica il modello di programmazione. È completamente indipendente da COM, ma è comunque fortemente integrato con esso.

.NET è un ambiente object oriented, quindi qualsiasi entità è un oggetto, inoltre consente di ridurre errori di programmazione molto comuni, tramite garbage collector e linguaggi fortemente tipizzati.

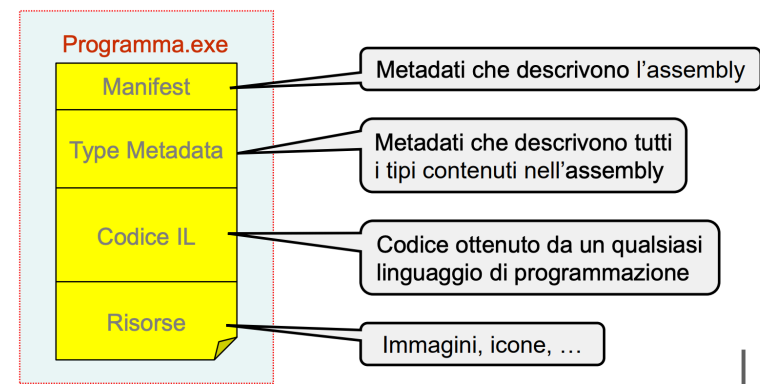
.NET fornisce libertà nella scelta del linguaggio, disponendo le funzionalità del framework a tutti i linguaggi .NET, consentendo anche di scrivere componenti della stessa applicazione in linguaggi diversi. .NET è infatti un'implementazione di CLI (Common Language Interface).

Esecuzione codice .NET

Il passaggio da codice sorgente all'esecuzione di un programma scritto nel framework .NET passa per uno step intermedio che prevede il codice IL (Assembly), il quale contiene codice + metadati.



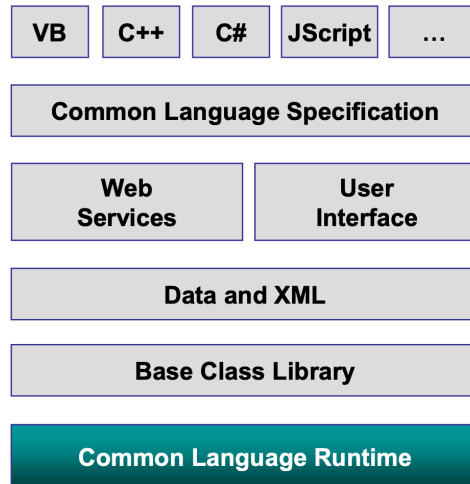
Il programma assembly è solitamente composto da un solo file (può essere composto anche da più file) e prevede la seguente struttura:



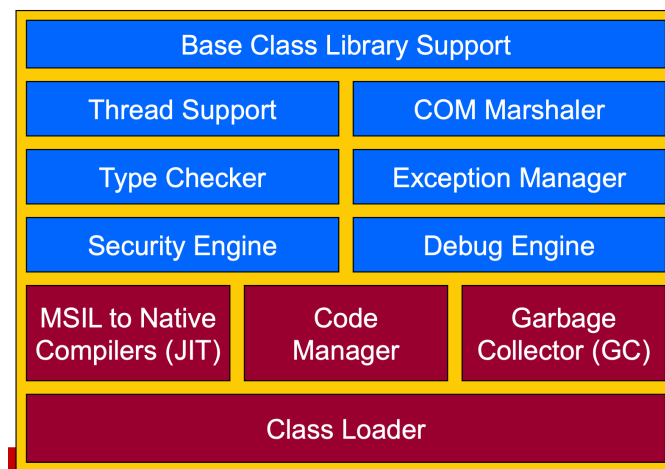
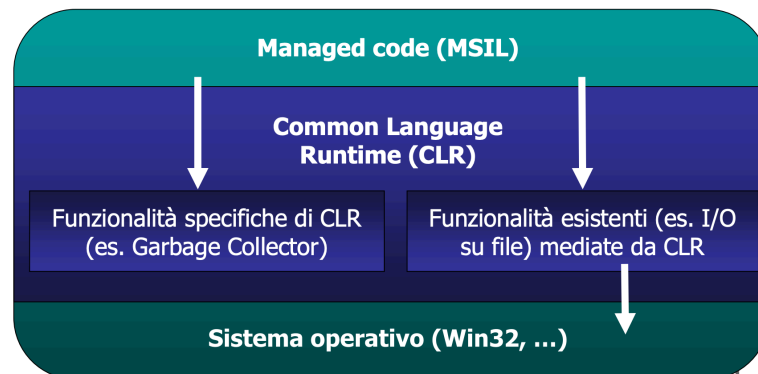
I metadati dell'assembly contengono informazioni sul programma, come nome, versione, lista dei file, dipendenze, permessi, tipi utilizzati, ecc., e queste informazioni vengono utilizzate dal compilatore e da altri tool.

Common language runtime

Il Common Language Runtime (CLR) è il motore di esecuzione del framework .NET.



Il CLR fornisce diversi servizi alle applicazioni:



Alcuni dei servizi più importanti sono:

- Garbage Collector

Gestisce il ciclo di vita di tutti gli oggetti .NET, i quali vengono distrutti automaticamente quando non sono più referenziati. A differenza di COM, non si basa sul Reference Counting, consentendo i riferimenti circolari e una maggiore velocità di allocazione.

- Gestore delle eccezioni

Concetti universali sono:

- Lanciare un'eccezione (throw)
- Catturare un'eccezione (catch)
- Eseguire codice di uscita da un blocco controllato (finally)

Questi sono disponibili in tutti i linguaggi .NET con sintassi diverse.

- Common Type System

.NET è fortemente tipizzato a tempo di compilazione. Alla base di tutto ci sono i tipi: classi, strutture, interfacce, enumerativi, delegati.

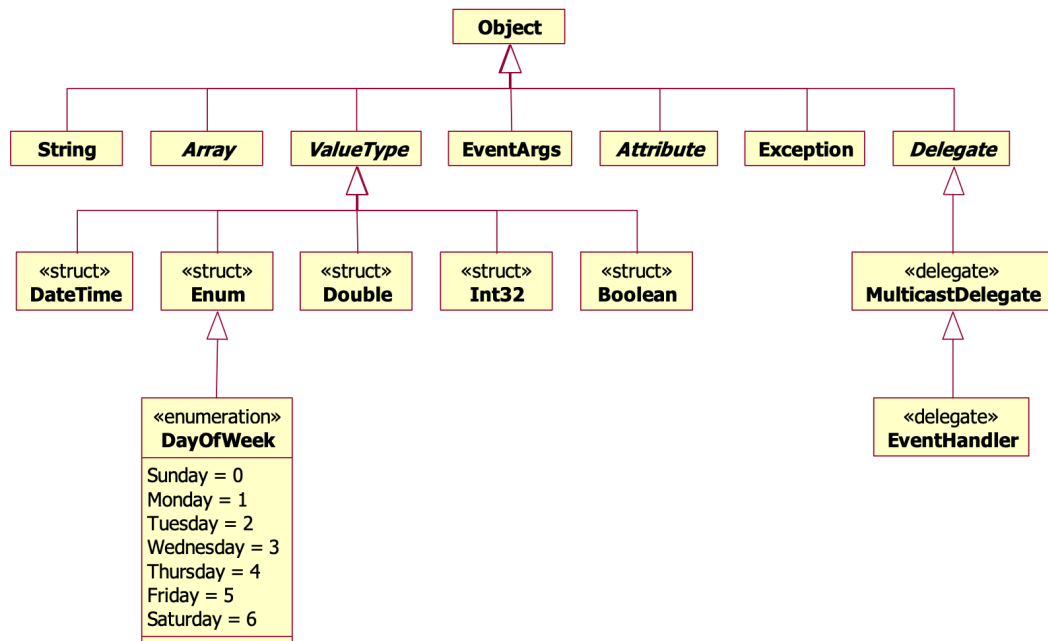
Tutto è un oggetto: System.Object è la classe radice. Ci sono poi due categorie di tipi:

- Tipi riferimento: riferimenti a oggetti allocati sull'heap gestito (indirizzi di memoria).
- Tipi valore: allocati sullo stack o parte di altri oggetti (sequenza di byte).

I tipi valore comprendono:

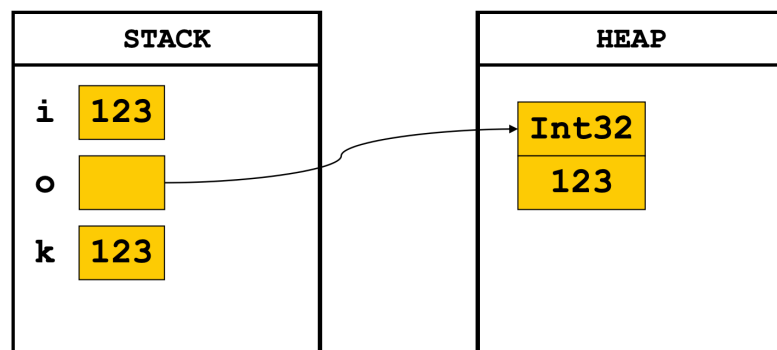
- Tipi primitivi (built-in)
 - Int32, ...
 - Single, Double
 - Decimal
 - Boolean
 - Char
- Tipi definiti dall'utente

- Strutture (struct)
- Enumerativi (enum)



Boxing/Unboxing:

```
int i = 123;
object o = i;
int k = (int) o;
```



- Common Language Specification

Definisce le regole di compatibilità tra linguaggi (regole per gli identificatori, per le proprietà, per i costruttori, ecc.).

▼ 5.0 - .NET Garbage Collection

Ciclo di vita di un oggetto

In un ambiente object-oriented, ogni oggetto di tipo riferimento ha il seguente ciclo di vita:

1. Allocare memoria per l'oggetto

- C/C++: malloc + new
- Java: new

2. Inizializzare la memoria per rendere utilizzabile l'oggetto (assignment)

A ogni variabile deve essere sempre assegnato un valore prima che essa venga utilizzata. Questo valore è quello di default per tipi valore, invece è dato dal costruttore per i tipi classe.

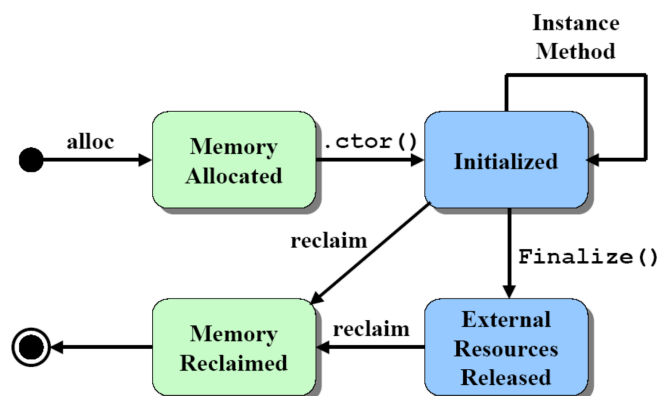
3. Usare l'oggetto

4. Eseguire un clean up dello stato dell'oggetto, se necessario

- C/C++: distruttore/finalizzatore.
- Java: finalize(), metodo di Object, invocato automaticamente dalla JVM alla distruzione dell'oggetto.

5. Liberare la memoria

- C/C++: free()/delete
- Java: garbage collector



Garbage Collection

La Garbage Collection è una modalità automatica di rilascio delle risorse utilizzate da un oggetto, la quale consente di evitare errori connessi alla necessità, da parte del programmatore, di manipolare direttamente i puntatori alle aree di memoria.

Strategie disponibili:

- Tracing

Determina quali oggetti sono potenzialmente raggiungibili ed elimina quelli non raggiungibili.

Un oggetto è raggiungibile in due soli casi:

- è un oggetto radice, ovvero è stato creato all'avvio del programma (oggetto globale), oppure creato da una subroutine (riferito sullo stack)
- è referenziato da un oggetto raggiungibile (diremo che è raggiungibile in maniera ricorsiva)

Svantaggi:

- Può accadere che un programma utilizzi per l'ultima volta un certo oggetto molto prima che questo diventi irraggiungibile.

- Reference counting

Ogni oggetto contiene un contatore che indica il numero di riferimenti a esso. La memoria può essere liberata quando il contatore raggiunge lo 0.

Svantaggi:

- Possibilità di cicli di riferimenti, ovvero se due oggetti si referenziano a vicenda, il loro contatore non raggiungerà mai 0.
- Aumento dell'occupazione di memoria.
- Riduzione della velocità delle operazioni sui riferimenti.
- Ogni modifica a un contatore deve essere resa operazione atomica in ambienti multi-threaded.

- Escape analysis

Si spostano oggetti dall'heap allo stack nel caso in cui questi, allocati all'interno di una subroutine, non siano accessibili al di fuori di essa.

Allocazione della memoria

In fase di inizializzazione di un processo, il CLR riserva una regione contigua di spazio di indirizzamento (managed heap) e memorizza in un puntatore (NextObjPtr) l'indirizzo di partenza della regione.



Quando deve eseguire una newobj, il CLR

- Calcola la dimensione in byte dell'oggetto e aggiunge all'oggetto due campi di 32 (o 64) bit:
 - Un puntatore alla tabella dei metodi
 - Un campo SyncBlockIndex
- Controlla che ci sia spazio sufficiente a partire da NextObjPtr e, in caso di spazio insufficiente:
 - garbage collection
 - OutOfMemoryException
- `thisObjPtr = NextObjPtr;`
- `NextObjPtr += sizeof(oggetto);`
- Invoca il costruttore dell'oggetto (`this = thisObjPtr`)
- Restituisce il riferimento all'oggetto

Garbage Collector

Il Garbage Collector ha due compiti:

- rilascia la memoria usata dagli oggetti non raggiungibili
- compatta la memoria ancora in uso, modificando nello stesso tempo tutti i riferimenti agli oggetti spostati e unifica la memoria disponibile, aggiornando il valore di NextObjPtr

Lavora dunque in queste due fasi:

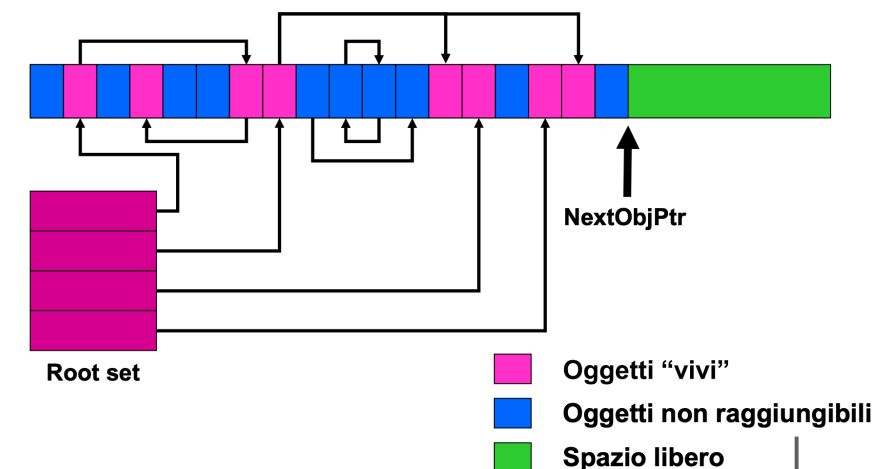
1. Fase di Mark

Ogni applicazione ha un insieme di radici (root), ovvero puntatori che contengono l'indirizzo di un oggetto di tipo riferimento. Le radici possono essere:

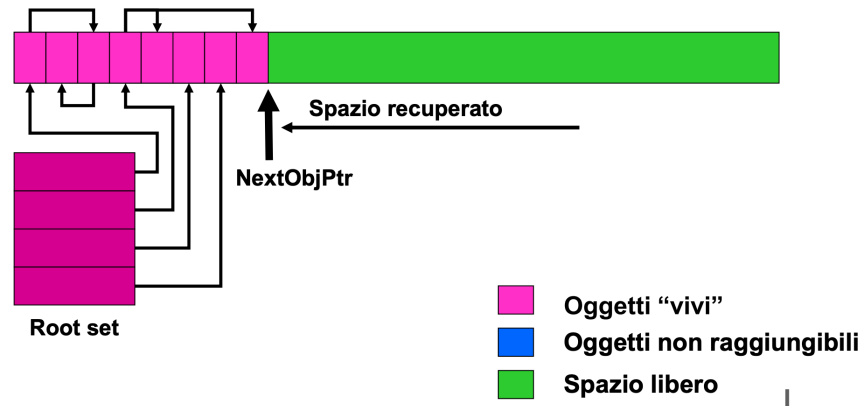
- Variabili globali e field statici di tipo riferimento
- Variabili locali o argomenti attuali di tipo riferimento sugli stack dei vari thread
- Registri della CPU che contengono l'indirizzo di un oggetto di tipo riferimento

Gli oggetti "vivi" sono quelli raggiungibili direttamente o indirettamente dalle radici. Gli oggetti garbage sono quelli NON raggiungibili direttamente o indirettamente dalle radici.

Quando parte, il GC ipotizza che tutti gli oggetti siano garbage. Quindi, scandisce le radici e per ogni radice marca l'eventuale oggetto referenziato e tutti gli oggetti a loro volta raggiungibili a partire da tale oggetto. Se durante la scansione incontra un oggetto già marcato in precedenza, lo salta sia per motivi di prestazioni, sia per gestire correttamente riferimenti ciclici. Una volta terminata la scansione delle radici, tutti gli oggetti NON marcati sono non raggiungibili e quindi garbage.



2. Fase di Compact



Finalization

La finalization non è responsabilità del GC, ma del programmatore.

- Se un oggetto contiene esclusivamente tipi valore e/o riferimenti a oggetti managed, non è necessario eseguire alcun codice particolare.
- Se un oggetto contiene almeno un riferimento a un oggetto unmanaged (in genera una risorsa del sistema operativo, file, connessione a database, socket, mutex, bitmap, ...), è necessario eseguire del codice per rilasciare la risorsa, prima della deallocazione dell'oggetto.

L'invocazione del metodo Finalize non avviene in modo deterministico. Inoltre, non essendo un metodo pubblico, il metodo Finalize non può essere invocato direttamente. Nel caso di utilizzo di risorse che devono essere rilasciate appena termina il loro uso, è di fondamentale importanza rilasciare (Dispose) o chiudere (Close) la risorsa in modo deterministico.

Se un tipo T vuole offrire ai suoi utilizzatori un servizio di clean up esplicito, deve implementare l'interfaccia IDisposable. In questo modo i clienti di T possono utilizzare l'istruzione using (all'uscita del blocco using, viene sempre invocato automaticamente il metodo Dispose).

```
public interface IDisposable
{
    void Dispose();
}

using (T tx = ...)
{
```

```
    utilizzo di tx...  
}  
  
using (FileStream fs = new FileStream("Temp.dat", FileMode.Create))  
{  
    fs.Write(bytesToWrite, 0, bytesToWrite.Length);  
}
```

▼ 6.0 - Tipi in .NET

Tipi in .NET

Dal punto di vista sintattico i tipi possono essere distinti in:

- Classi
- Interfacce
- Strutture
- Enumerativi
- Delegati
- Array

Tutti questi, in .NET, si concretizzano in una classe.

Modificatori di visibilità

Field

Modificatore	Tipi a livello base	Tipi annidati
private	Non applicabile	Visibile nel tipo contenitore (default)
protected	Non applicabile	Visibile nel tipo contenitore e nei suoi sottotipi
internal	Visibile nell' <i>assembly</i> contenitore (default)	Visibile nell' <i>assembly</i> contenitore
protected internal	Non applicabile	Visibile sia nel tipo contenitore e nei suoi sottotipi, sia nell' <i>assembly</i> contenitore
public	Visibilità completa	Visibilità completa

`private` è il modificatore da usare di default, mentre gli altri devono essere eventualmente applicati solo a campi read-only. La regola da seguire è quella di massimizzare l'incapsulamento minimizzando la visibilità.

I campi read-only possono essere di due tipologie:

- Costante a tempo di compilazione

```
public const int MaxEntries = 100;
```

Il valore viene sostituito direttamente nel codice durante la compilazione. Se il valore viene modificato e se il cliente e il fornitore sono in *assembly* diversi, è necessario ricompilare anche il codice del cliente.

- Costante a runtime

```
lic static readonly int MaxEntries = 100;
```

Il valore viene risolto quando la classe viene caricata in memoria. Se il valore viene modificato e se il cliente e il fornitore sono in *assembly* diversi, NON è necessario ricompilare anche il codice del cliente.

La regola da utilizzare è quella di definire `const` solo le costanti "vere" (nelle versioni del programma), cioè i valori veramente immutabili nel tempo, negli altri casi utilizzare field statici read-only

Metodi

I modificatori di metodi sono:

- virtual

Consente l'override polimorfico del metodo da parte di una sottoclasse. Per default, i metodi non sono virtuali.

- abstract

Indica che il metodo non contiene alcuna implementazione. Un metodo astratto è implicitamente virtuale. La dichiarazione di metodi astratti è permessa solo in classi astratte.

- override

Un metodo override fornisce una (nuova) implementazione di un metodo ereditato da una classe base, il quale deve essere virtual, abstract o override.

- override sealed / sealed override

L'uso del modificatore sealed impedisce a una qualsiasi classe derivata l'ulteriore sovrascrittura del metodo.

Passaggio argomenti

Ci sono tre tipi di argomenti:

- In: default in C# per i tipi valore.
- In/Out: passaggio per riferimento, occorre utilizzare la keyword ref.
- Out: l'argomento non deve essere inizializzato al momento del passaggio, occorre utilizzare la keyword out.

Tramite la keyword params è possibile anche passare un numero variabile di argomenti dello stesso tipo.

```
int Add(params int[] operands)
{
    int total = 0;
    foreach (int operand in operands)
        total += operand;
```

```
    return total;  
}
```

Costruttori di tipo

In C# un costruttore di tipo, o costruttore statico, è un metodo speciale che inizializza i membri statici di una classe. È sempre senza argomenti, deve essere dichiarato static e può accedere esclusivamente ai membri statici della classe. È da definire solo se strettamente necessario, cioè se i campi statici della classe non possono essere inizializzati in linea.

Se un costruttore di tipo lancia un'eccezione che non viene gestita al suo interno, la classe che presenta tale costruttore non potrà più essere utilizzata.

Interfacce

In C#, un'interfaccia può contenere esclusivamente metodi, proprietà, indexer ed eventi, ciascuno con l'obbligo di essere pubblico e astratto. Un'interfaccia è considerata reference type e può essere implementata sia dai reference type che dai value type (se si effettua il cast di un value type a un'interfaccia, avviene un boxing del value type).

```
public interface IBehavior  
{  
    void Method();  
    int Property { get; set; }  
    int this[int index] { get; }  
}  
  
public class A : IBehavior  
{  
    public void Method() // virtual sealed  
    { ... }  
    public int Property // virtual sealed  
    {  
        get { ... }  
        set { ... }  
    }  
}
```

```

    }
    public int this[int index] // virtual sealed
    {
        get { ... }
    }
}

```

```

public class B : A
{
    public override void Method() ...
    public override int Property ...
    public override int this[int index] ...
}

```

```

// implementazione esplicita di un interfaccia
public interface IMyInterface1
{ void Close(); }
public interface IMyInterface2
{ void Close(); }
public class MyClass : IMyInterface1, IMyInterface2
{
    void IMyInterface1.Close()
    { ... }
    void IMyInterface2.Close()
    { ... }
    public void Close()
    { ... }
}

```

```

MyClass a = new MyClass(...);
((IMyInterface1) a).Close();
((IMyInterface2) a).Close();
a.Close();

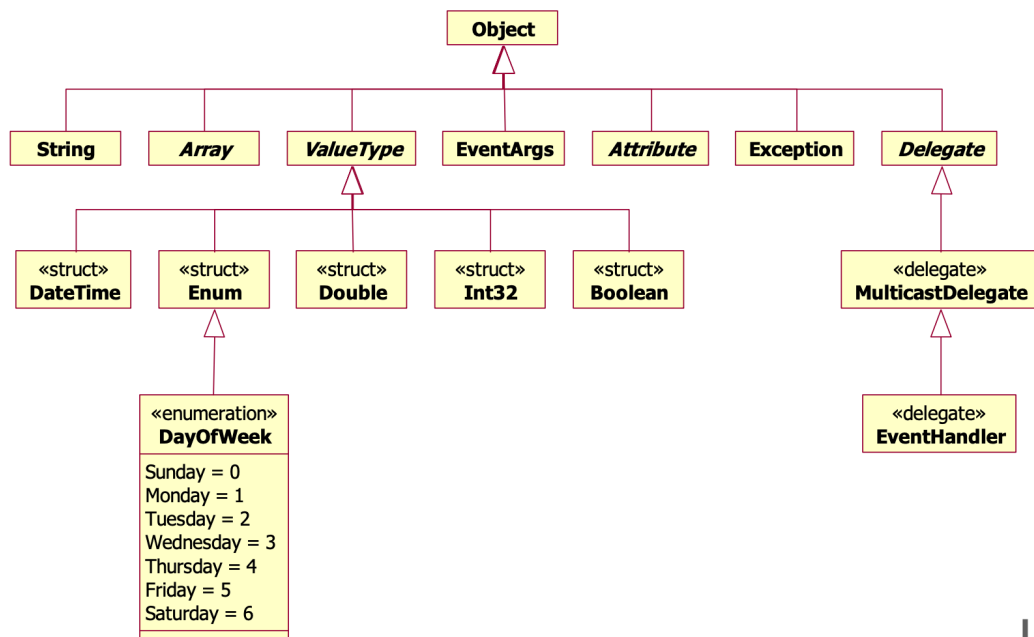
```

Interfaccia vs classe astratta

Interfaccia	Classe astratta
Descrive una funzionalità semplice, implementabile da oggetti eterogenei (es. ICloneable, IList ecc.)	Descrive una funzionalità anche complessa, comune a un insieme di oggetti omogenei
Può ereditare da: - 0+ interfacce	Può ereditare da: - 0+ interfacce - 0+ classi
Non può contenere uno stato	Può contenere uno stato comune a tutte le sottoclassi
Non contiene alcuna implementazione	Può essere implementata completamente, in parte o per niente
Non può gestire la creazione delle istanze delle classi che la implementano	Può gestire la creazione delle istanze delle sue sottoclassi tramite un metodo statico (metodo factory)

▼ 7.0 - Classi e interfacce base

.NET overview



System.Object

Ogni metodo definito nella classe Object è disponibile in tutti gli oggetti del sistema, i quali lo estendono.

Object
+ Object () # Finalize () + GetHashCode () : System.Int32 + Equals ([in] obj : System.Object) : System.Boolean + ToString () : System.String + Equals ([in] objA : System.Object , [in] objB : System.Object) : System.Boolean + ReferenceEquals ([in] objA : System.Object , [in] objB : System.Object) : System.Boolean + GetType () : System.Type # MemberwiseClone () : System.Object

Alcuni di questi possono essere sovrascritti, tra cui Equals, ToString, GetHashCode e Finalize (effettua operazioni di pulizia prima che l'oggetto venga automaticamente distrutto). Ad esempio Equals in Object è definita come identità reference equality, in String è ridefinita e controlla se le due stringhe contengono gli stessi caratteri, in oggetti ValueType è ridefinita come uguaglianza bit-a-bit value equality e così via.

System.IComparable

Confronta l'istanza corrente con un altro oggetto dello stesso tipo.

Valore di ritorno: un 32-bit signed integer che indica l'ordine relativo degli oggetti confrontati. La semantica del valore restituito è la seguente:

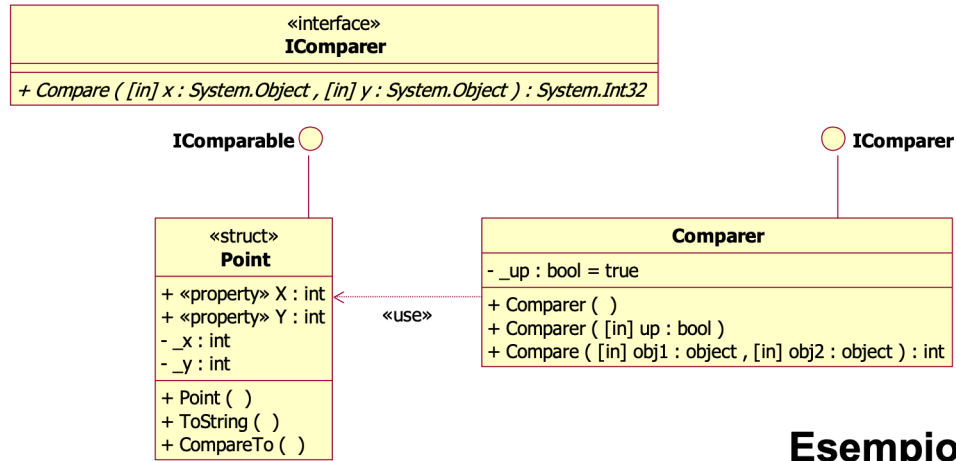
- Minore di zero – l'istanza this precede obj
- Zero – l'istanza this è uguale a obj
- Maggiore di zero – l'istanza this segue obj

«interface» IComparable
+ CompareTo ([in] obj : System.Object) : System.Int32

System.Collections.IComparer

Fornisce un modo per personalizzare il criterio di ordinamento di una collezione.

Questa interfaccia è usata, ad esempio, dai metodi Array.Sort e Array.BinarySearch. Fornisce un modo per personalizzare il criterio di ordinamento di una collezione.

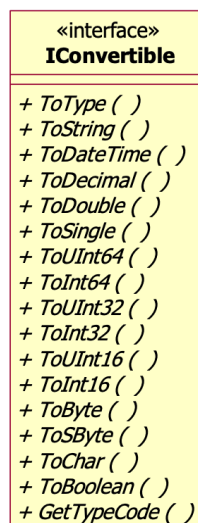


Esempio

System.IConvertible

Questa interfaccia fornisce metodi per convertire il valore di un'istanza di un tipo che implementa l'interfaccia in un valore equivalente di un tipo CLR. I tipi CLR sono Boolean, SByte, Byte, Int16, UInt16, Int32, UInt32, Int64, UInt64, Single, Double, Decimal, DateTime, Char, e String.

Se non esiste una conversione sensata verso un tipo CLR, l'implementazione particolare del corrispondente metodo dell'interfaccia deve lanciare una `InvalidCastException`.



System.ICloneable

Fornisce metodi per clonare un'istanza di una classe.

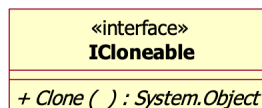
Il metodo Clone crea un nuovo oggetto che è una copia dell'istanza corrente.

Clone può essere implementato come:

- una copia superficiale (shallow), vengono duplicati solo gli oggetti di primo livello, non vengono create nuove istanze di alcun campo.

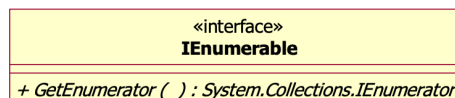
```
public object Clone()
{
    return MemberwiseClone();
}
```

- una copia profonda (deep), tutti gli oggetti vengono duplicati.



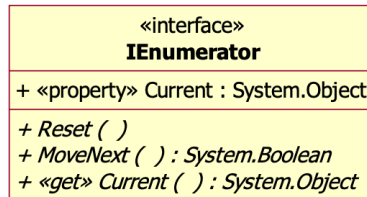
System.Collections.IEnumerable

GetEnumerator restituisce un enumeratore, ovvero un oggetto che supporta l'iterazione su una collezione. Gli enumerator permettono solamente di leggere i dati della collezione, ma non di modificarli.



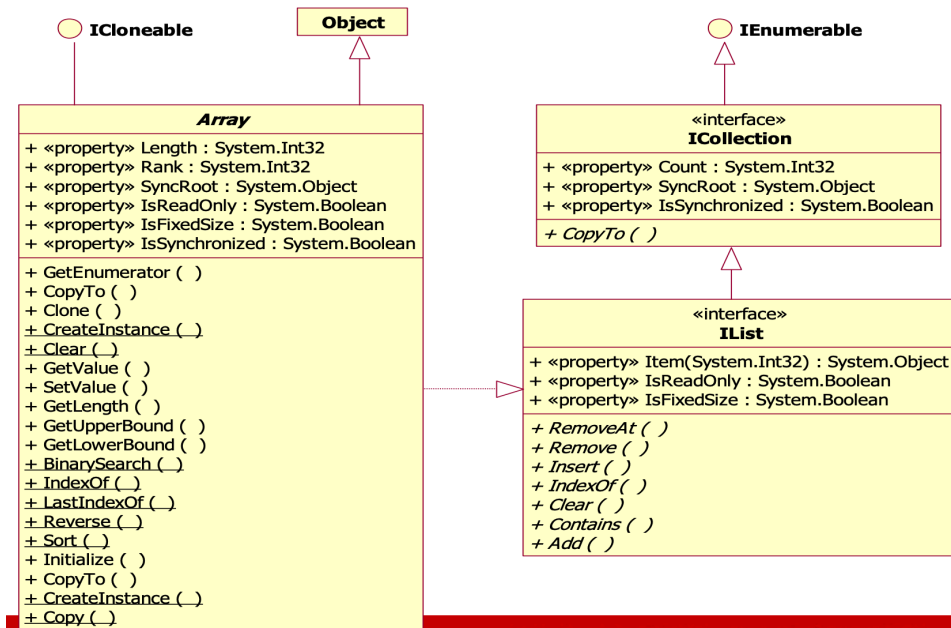
Su un oggetto di tipo IEnumerator:

- Reset riporta l'enumeratore allo stato iniziale
- MoveNext si sposta all'elemento successivo, restituendo
 - true se l'operazione ha successo
 - false se l'enumeratore ha oltrepassato l'ultimo elemento
- Current restituisce l'oggetto attualmente referenziato



Array

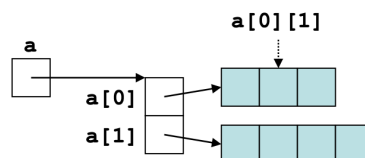
La classe Array ad esempio permette l'utilizzo di un enumeratore:



In C# si possono avere due tipi di Array:

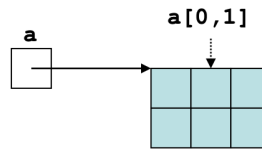
- Frastagliati (come in Java)

```
int[][] a = new int[2][];
a[0] = new int[3];
a[1] = new int[4];
```



- Matrici (come in C, più compatti ed efficienti)

```
int[,] a = new int[2, 3];
```



Conversioni di tipo

Una conversione di ampliamento avviene quando un valore di un tipo viene convertito verso un altro tipo che è di dimensione uguale o superiore. Una conversione di restrizione avviene quando un valore di un tipo viene convertito verso un altro tipo che è di dimensione inferiore.

Possiamo suddividere le conversioni in:

- Conversioni implicite: cast automatici, non generano eccezioni.

Per poter effettuare conversioni implicite occorre rispettare i seguenti vincoli:

- Il tipo di destinazione dovrebbe essere in grado di contenere, senza perdita di informazione, tutti i valori ammessi dal tipo di partenza.
- Principio di sostituibilità: deve sempre essere possibile utilizzare una classe derivata al posto della classe base.

```
B b = new B(...); // class B : A  
A a = b;
```

- Conversioni esplicite: cast manuali, possono generare eccezioni.

In questi casi:

- Il tipo di destinazione non sempre è in grado di contenere il valore del tipo di partenza.
- Può essere non rispettato il principio di sostituibilità.

È possibile proteggersi da cast non sicuri in due modi:

- is

```
if(a is B)
{
    b = (B) a; // non genera eccezioni
}
```

- as

```
b = a as B; // b = (a is B) ? (B) a : null;
if(b != null)
{
    ... // conversione senza errori, posso usare b
}
```

Alcune tipologie particolari di conversioni implicite ed esplicite sono:

- Boxing - up cast (conversione implicita)

```
int k1 = 100;
object o = k1; // copia
```

- Unboxing - down cast (conversione esplicita)

```
k1 = 200;
int k2 = (int) o; // k1 = 200, k2 = 100
```

È possibile definire conversioni personalizzate tra tipi inserendo un metodo statico di questo tipo all'interno di una delle due classi coinvolte:

```
public static implicit operator typeOut(typeIn obj)
public static explicit operator typeOut(typeIn obj)
```

È possibile convertire a String tramite i metodi ToString o String.Format, e da String tramite Parse.

▼ 8.0 - Delegati ed eventi

Delegati

Sono oggetti che possono contenere il riferimento a un metodo, tramite il quale il metodo può essere invocato. L'utilizzo standard dei delegati è quello di callback.

Un delegato definisce solo la firma dei metodi a cui può puntare, non i metodi veri e propri a cui punta.

```
// dichiarazione di un delegato
delegate int Azione(int param);

// definizione di un delegato
Azione azione;

// inizializzazione di un delegato
azione = new Azione(nomeMetodoStatico);
azione = new Azione(obj.nomeMetodo);
azione = nomeMetodoStatico;
azione = obj.nomeMetodo;

// invocazione del metodo referenziato dal delegato
int k1 = azione(10);
```

È anche possibile assegnare al delegato una lista di metodi. All'atto della chiamata del delegato, i metodi vengono invocati in sequenza e in modo sincrono.

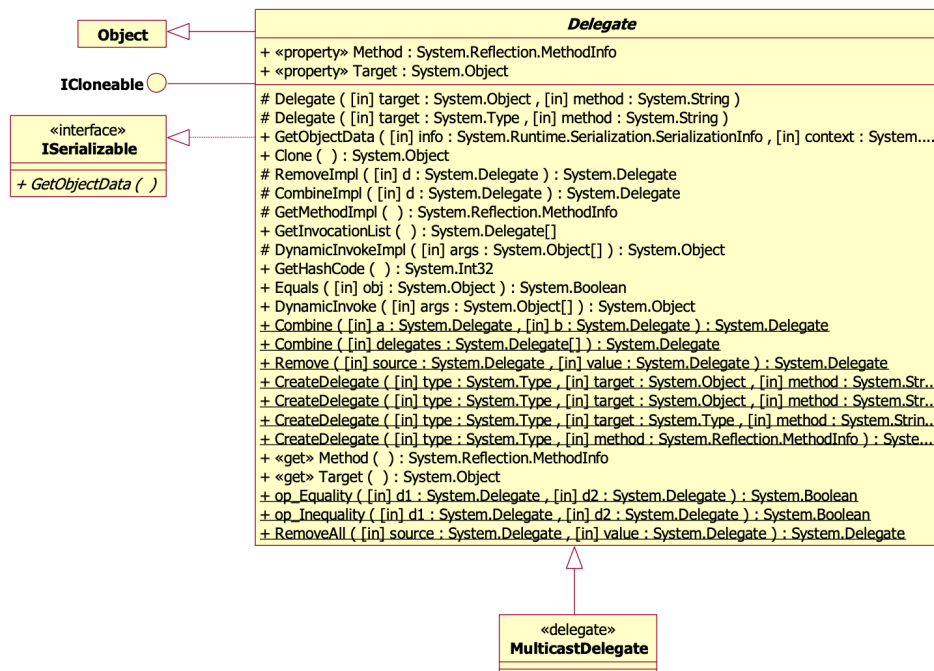
```
Azione azione = Fun1
azione(10) // Fun1(10)

azione += Fun2
azione(10) // Fun1(10), Fun2(10)
```

```
azione -= Fun1
azione(10) // Fun2(10)
```

Se la chiamata del delegato include parametri di output o un valore di ritorno, il loro valore finale verrà dall'invocazione dell'ultimo delegato nell'elenco.

In C#, la dichiarazione di un nuovo tipo di delegato definisce automaticamente una nuova classe derivata dalla classe `System.MulticastDelegate`.



Un esempio di situazione in cui dei delegati sono utili è la seguente: è necessario modellare un'interazione tra due componenti, un Worker che effettua un'attività (o lavoro) e un Boss che controlla l'attività dei suoi Worker. Ogni Worker deve notificare al proprio Boss:

- quando il lavoro inizia
- quando il lavoro è in esecuzione
- quando il lavoro finisce

Tramite delegati questa situazione può essere gestita in questo modo

```

public delegate void WorkStarted(Worker worker);
public delegate void WorkProgressing(Worker worker);
public delegate int WorkCompleted(Worker worker);

public interface IWorkerEvents
{
    void WorkStarted(Worker worker);
    void WorkProgressing(Worker worker);
    int WorkCompleted(Worker worker);
}

public class Worker
{
    public WorkStarted OnWorkStarted;
    public WorkProgressing OnWorkProgressing;
    public WorkCompleted OnWorkCompleted;

    public void DoWork()
    {
        OnWorkStarted(this);
        OnWorkProgressing(this);
        OnWorkCompleted(this);
    }
}

public class Boss : IWorkerEvents
{
    public void WorkStarted(Worker worker)
    {
        Console.WriteLine($"[Boss] {worker.Name} ha iniziato a lavorare.");
    }

    public void WorkProgressing(Worker worker)
    {
        Console.WriteLine($"[Boss] {worker.Name} sta ancora lavorando...");
    }
}

```

```

    }

    public void WorkCompleted(Worker worker)
    {
        Console.WriteLine($"[Boss] {worker.Name} ha finito il lavoro.");
    }
}

var worker = new Worker();
var boss = new Boss();

worker.OnWorkStarted = boss.WorkStarted;
worker.OnWorkProgressing = boss.WorkProgressing;
worker.OnWorkCompleted = boss.WorkCompleted;

worker.DoWork();

```

Eventi

L'esposizione di delegati come campi pubblici, come appena visto, fornisce un accesso eccessivo e porta dunque a grossi problemi di sicurezza. Chi usa la classe può infatti sovrascrivere i delegati o chiamarli direttamente.

È possibile ottenere dei delegati di tipo private che possono essere modificabili solo tramite aggiunta (+=) o rimozione di metodi (-=) utilizzando il modificatore event.

```

public class Worker {
    public event WorkStarted Started;
    public event WorkProgressing Progressing;
    public event WorkCompleted Completed;

    public void DoWork() {
        Started(); // solo la classe può chiamarlo
    }
}

```

```
peter.Started += WorkStarted;  
peter.Started = WorkStarted; // ERRORE: non si può assegnare direttamen  
te  
peter.Started();           // ERRORE: non si può invocare da fuori
```

È anche possibile personalizzare la registrazione a eventi con la seguente sintassi

```
class Worker  
{  
    public event WorkProgressing Progressing  
    {  
        add  
        {  
            if(DateTime.Now.Hour < 12)  
            { _progressing += value; }  
            else  
            { throw new InvalidOperationException  
              ("Must register before noon."); }  
        }  
        remove  
        { _progressing -= value; }  
    }  
  
    private WorkProgressing _progressing;  
}
```

In un evento ci sono tre entità:

- event sender: l'oggetto che scatena l'evento.
- event receiver: l'oggetto per il quale l'evento è determinante e che quindi desidera essere notificato quando l'evento si verifica.
- event handler: il metodo che viene eseguito all'atto della notifica.

Quando si verifica l'evento, il sender invia un messaggio di notifica a tutti i receiver. In genere, il sender NON conosce né i receiver, né gli handler. Il meccanismo che viene utilizzato per collegare sender e receiver/handler è il delegato.

Per convenzione, i delegati degli eventi in .NET hanno 2 parametri

- la sorgente che ha scatenato l'evento e
- i dati relativi all'evento

Molti eventi, inclusi eventi della GUI come i click del mouse, non generano dati. In tali situazioni, è sufficiente usare il delegato dell'evento fornito dalla libreria di classi per gli eventi senza dati, System.EventHandler.

```
public delegate void EventHandler(object sender, EventArgs e)
```

Per scatenare un evento è opportuno definire un metodo protetto virtuale OnNomeEvento e invocare sempre quello

```
public event EventHandler Changed;

protected virtual void OnChanged()
{
    if(Changed != null) Changed(this, EventArgs.Empty); // EventArgs.Empty
    // se non occorre passare informazioni aggiuntive al gestore
}

OnChanged();
```

▼ 9.0 - Metadati e introspezione

Metadati

I metadati sono dati che descrivono altri dati. Per esempio, la definizione di classe è un metadata.

Metadati nei diversi linguaggi

Metadati C/C++

Un file header .h C/C++ è un metadata. I client includono il file header in fase di compilazione per utilizzare i tipi che questo dichiara. Sono però specifici del linguaggio, e non permettono l'interoperabilità tra linguaggi diversi.

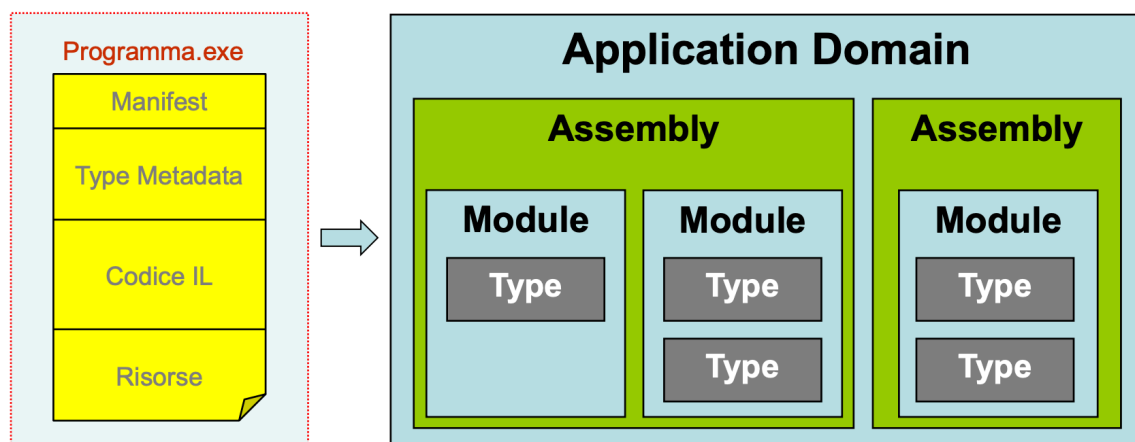
Metadata IDL

In sistemi come COM o CORBA, l'IDL permette di descrivere le interfacce, risolvendo in parte il problema dell'interoperabilità tra linguaggi diversi.

Il problema è che ciò richiede la scrittura manuale di un file separato e la conoscenza di un metalinguaggio.

Metadata .NET

In .NET i metadata sono generati automaticamente dalla definizione del tipo. Non si ha più un file separato, ma le informazioni descrittive sono integrate direttamente nell'assembly compilato. Si trovano infatti collegati al codice IL all'interno dei moduli, che a loro volta sono parte degli assembly.



Reflection

La Reflection è la capacità di un programma di esaminare i propri metadata o i metadata di altri programmi in tempo reale.

La Reflection consente di:

- Esaminare un Assembly: scoprire i tipi che contiene, quali metodi, campi, proprietà ha.
- Istanziare oggetti e chiamare metodi dinamicamente.

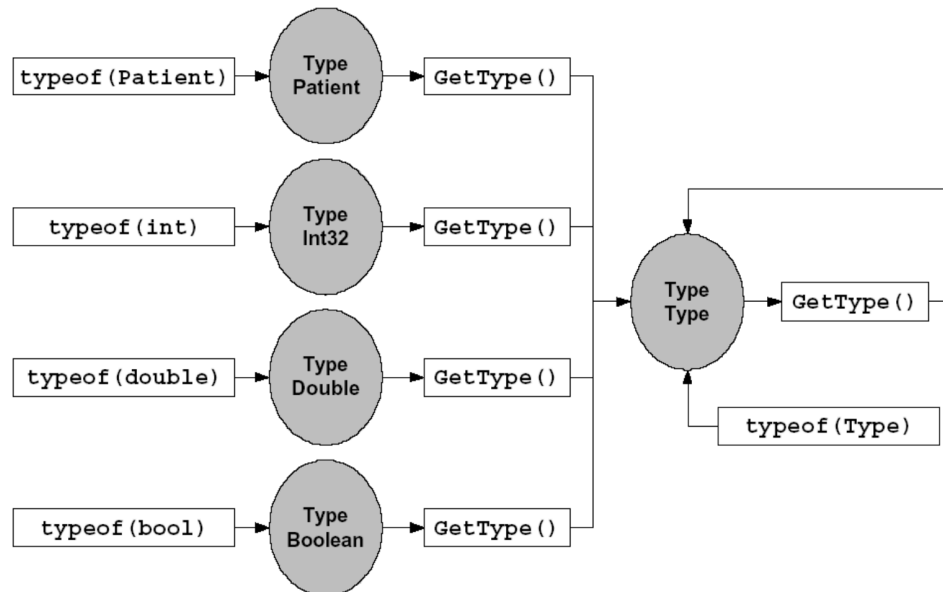
- Creare, compilare ed eseguire assembly "al volo".

System.Type

System.Type è il punto nodale per la reflection.

È possibile scoprire il tipo di un oggetto o di un value richiamando su di esso il metodo GetType(). I tipi restituiti sono a loro volta istanze del tipo System.Type.

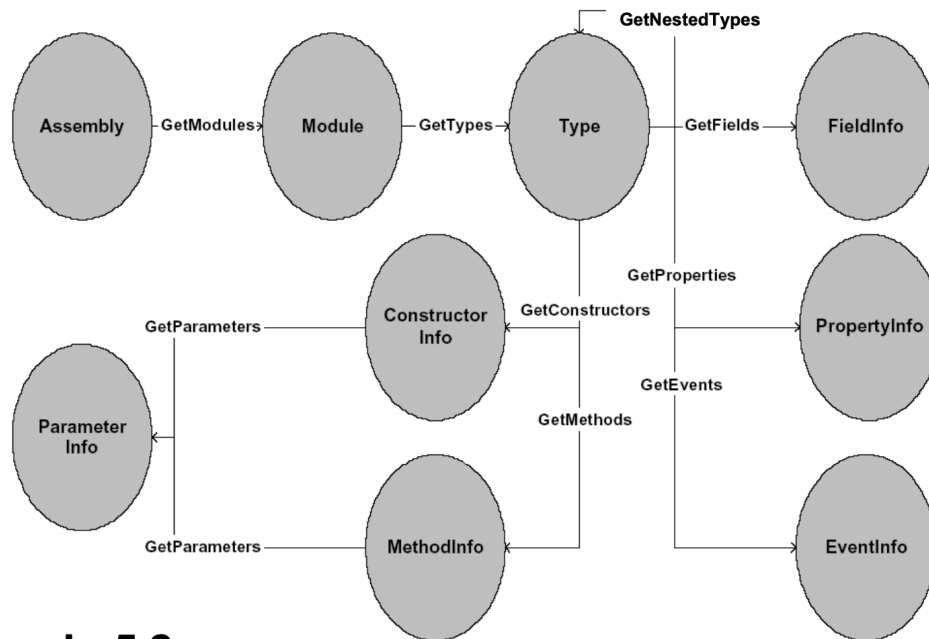
C'è un singolo oggetto Type per ciascun tipo definito nel sistema (se hai 100 istanze della classe Persona, c'è comunque solo un oggetto Type che descrive la classe Persona, e da cui questa è stata quindi creata).



Il modello a oggetti della Reflection

È possibile enumerare i tipi di un assembly tramite i seguenti comandi:

- `Assembly.Load(...)` restituisce un oggetto `Assembly`
- `Assembly.GetModules(...)` restituisce un array di oggetti `Module`
- `Module.GetTypes(...)` restituisce un array di `Type`



mpio 5 2

Very late binding

La Reflection consente di lavorare con i tipi a tempo di esecuzione, creandone istanze e/o accedendone ai membri.

- Si può istanziare il tipo in memoria, scegliendo il costruttore da chiamare: `Activator.CreateInstance(type, ...)`

È l'equivalente late-bound dell'operatore `new`.

```

T1 t = (T1) Activator.CreateInstance(typeof(T1));
T1 t = (T1) Activator.CreateInstance(typeof(T1), object[] args);
  
```

- È possibile invocare metodi: `MethodInfo.Invoke(...)`
- È possibile invocare accessori e modificatori di proprietà: `propertyInfo.GetValue(...)` e `propertyInfo.SetValue(...)`

Attributi personalizzabili

Gli attributi personalizzabili sono informazioni che possono essere aggiunte ai metadati generati automaticamente.

Sono in realtà solo classi comuni che derivano da `System.Attribute`, e possono contenere metodi e proprietà. Una volta dichiarata la classe è possibile:

- Dichiarare i costruttori
- Dichiarare le proprietà
- Applicare AttributeUsageAttribute (opzionale), il quale consente di controllare il comportamento della classe (dove questo marcatore può essere posizionato, se le classi figlie lo ereditano, e se possono esistere più istanze dell'attributo per lo stesso elemento).

È possibile applicare un attributo ad una classe utilizzando una sintassi IDL-like con []. Si passano informazioni usando i campi posizionali (specificati nel costruttore) o i campi con nome (le proprietà).

```
[ Author("Patella", Contact="marco.patella@unibo.it") ]
```

```
// definizione della classe target
```

Tramite Reflection è possibile leggere gli attributi personalizzati di un oggetto richiamando il metodo GetCustomAttributes su di esso.

Meta-Programmazione

La programmazione che all'interno di un sistema utilizza e manipola le informazioni sul sistema stesso è detta meta-programmazione.

Diverse classi in .NET funzionano insieme per effettuare meta-programmazione, permettendo di creare assembly "al volo". La classe System.Reflection.Emit consente di scrivere il codice IL necessario per creare e compilare un assembly che potrà essere chiamato direttamente dal programma che lo ha creato, e che potrà essere archiviato su disco in modo che altri programmi possano utilizzarlo.

▼ 10.0 - Principi di design

Qualità della progettazione

La qualità della progettazione è un concetto vago, questa dipende infatti da specifiche priorità dell'organizzazione (affidabilità, efficienza, manutenibilità ecc.). Gli argomenti qui discussi riguardano principalmente la manutenibilità del progetto.

Cosa rende un design cattivo

Un design cattivo è dato da uno o più dei seguenti fattori:

- Misdirection: non soddisfa i requisiti
- Rigidità del software

Sintomo: ogni modifica provoca una cascata di modifiche successive nei moduli dipendenti.

Effetto: i manager hanno timore ad accettare che gli sviluppatori risolvano problemi non critici, non sanno se/quando gli sviluppatori termineranno le modifiche.

- Fragilità del software

Sintomo: ogni correzione peggiora le cose, introducendo più problemi di quelli risolti su parti inaspettate del sistema.

Effetto: ogni volta che i manager autorizzano una correzione, temono che il software si "rompa" in modo inaspettato.

- Immobilità del software

Sintomo: i moduli scritti sono difficili da riutilizzare per via delle troppe dipendenze.

Effetto: il software viene semplicemente riscritto anziché riutilizzato.

- Viscosità

Sintomo: è difficile fare la cosa giusta, ma facile fare la cosa sbagliata.

Effetto: la manutenibilità del software degenera a causa di hack (modifiche che non preservano l'intento di progettazione originale).

Cause di progettazioni scandenti

Possono essere delle cause ovvie, come la mancanza di capacità/pratica di progettazione, le tecnologie in evoluzione, i vincoli di tempo/risorse, la complessità del dominio ecc.

Possono essere anche cause meno ovvie, come:

- "Putrefazione" del software avviene lentamente, rovinando nei mesi/anni un design originariamente pulito.

- I requisiti cambiano in maniera non prevista dal design originale.
- Dipendenze mal gestite tra i moduli.

Alcuni di questi problemi possono essere prevenuti tramite l'applicazione di principi e tecniche.

Premesse

Il principio zero (rasoglio di Occam)

Non bisogna introdurre concetti che non siano strettamente necessari.

Tra due soluzioni bisogna preferire quella che introduce il minor numero di ipotesi e che fa uso del minor numero di concetti.

Semplicità e semplicismo

La semplicità è un fattore importantissimo, il problema è che richiede uno sforzo non indifferente (è molto più facile essere complicati che semplici). Bisogna essere semplici ma non semplicisti.

Divide et impera

La decomposizione è una tecnica fondamentale per il controllo e la gestione della complessità. Non esiste un solo modo per decomporre un sistema, dunque per farlo nel migliore dei modi occorre seguire il seguente principio

Minimizzare il grado di accoppiamento tra i moduli del sistema.

Ovvero minimizzare la quantità di interazioni tra i moduli.

Rendere privati tutti i dati degli oggetti

Le modifiche ai dati pubblici rischiano sempre di "aprire" il modulo, rischiando di avere un effetto domino che porta a richiedere modifiche in molte posizioni impreviste.

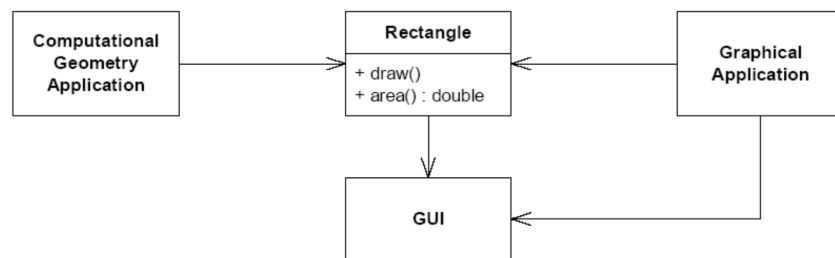
Principi di design

The Single Responsibility Principle / Principio di singola responsabilità

Una classe ha una sola responsabilità: la fa tutta, la fa bene e fa solo questa.

Se una classe ha più di una responsabilità, le modifiche a una di queste possono compromettere o inibire la capacità della classe di realizzare le altre
→ fragilità.

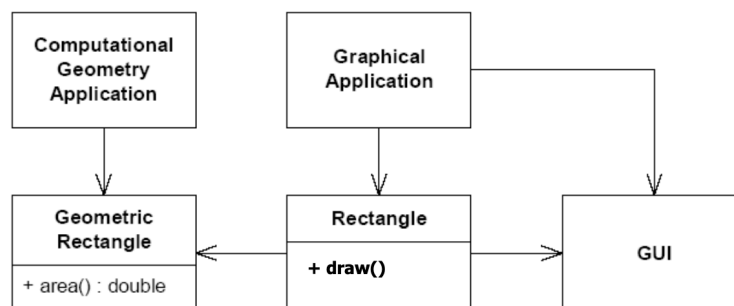
- Esempio



La classe Rectangle ha due responsabilità

- la prima responsabilità è fornire un modello matematico della geometria di un rettangolo
- la seconda responsabilità è disegnare il rettangolo su un'interfaccia grafica

Un progetto migliore consiste nel separare le due responsabilità in due classi completamente distinte.

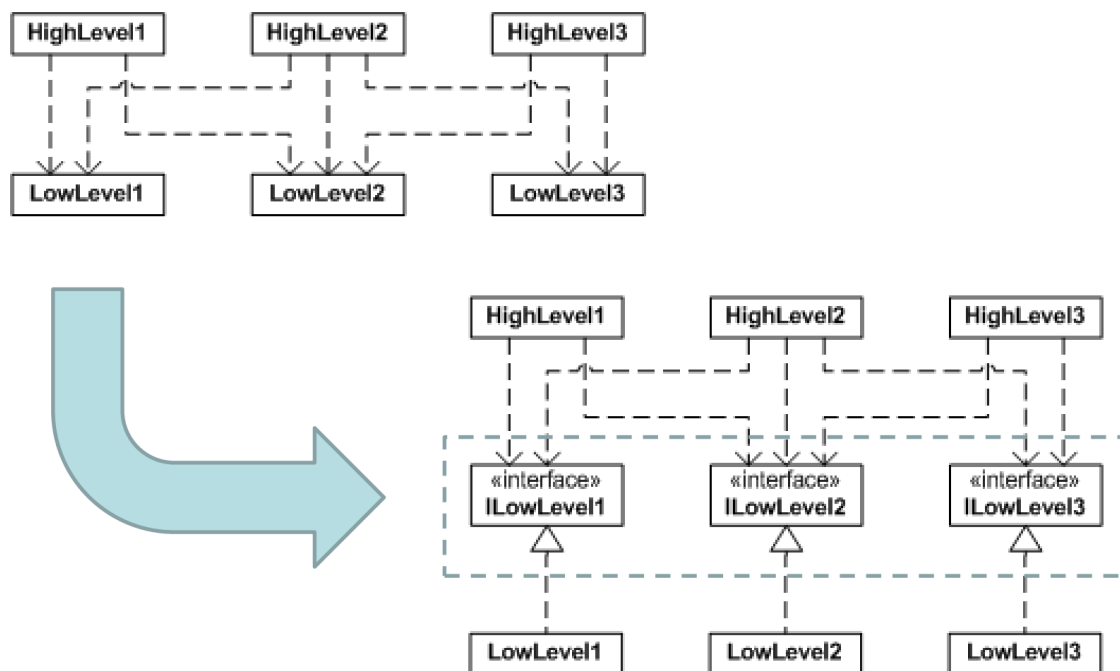


The Dependency Inversion Principle / Principio di inversione delle dipendenze

Dipendi dalle astrazioni, non dalle implementazioni concrete.

Ogni dipendenza dovrebbe puntare a un'interfaccia o a una classe astratta. Nessuna dipendenza dovrebbe puntare a una classe concreta.

I moduli di alto livello (i clienti) non dovrebbero dipendere dai moduli di basso livello (i fornitori di servizi). Entrambi dovrebbero dipendere da astrazioni. I moduli di basso livello contengono infatti la maggior parte del codice e della logica implementativa e quindi sono i più soggetti a cambiamenti. Se i moduli di alto livello dipendono dai dettagli dei moduli di basso livello i cambiamenti si propagano con conseguente rigidità, fragilità e immobilità.



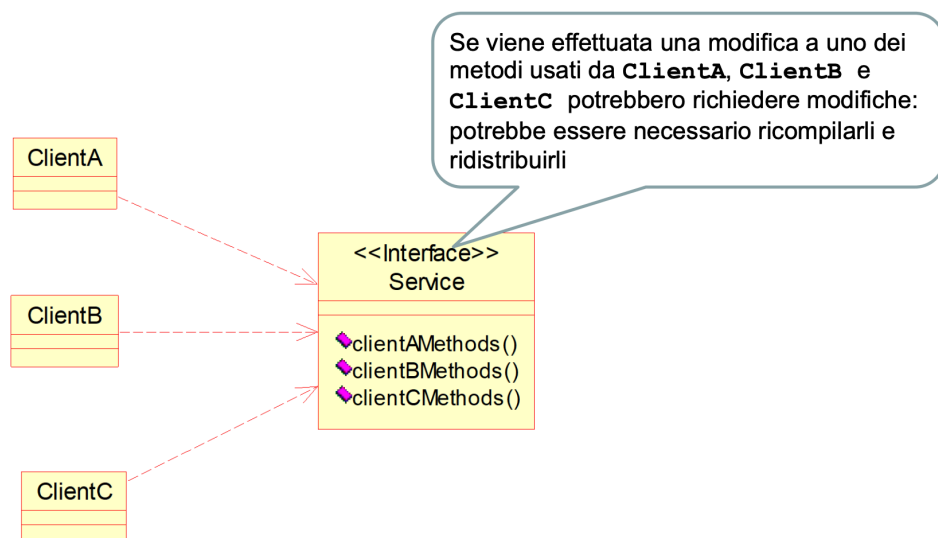
Questo funziona perché le astrazioni contengono pochissimo codice (in teoria nulla), quindi se i moduli non astratti sono soggetti a cambiamenti questi non si propagano (design for change). Nel contempo i singoli moduli sono maggiormente riusabili perché disaccoppiati tra loro (design for reuse).

Per questo i sistemi software dovrebbero essere stratificati, cioè organizzati a livelli, e le dipendenze transitive (A dipende da B e B dipende da C → A dipende da C) e le dipendenze cicliche (A dipende da B e B dipende da A) devono essere eliminate.

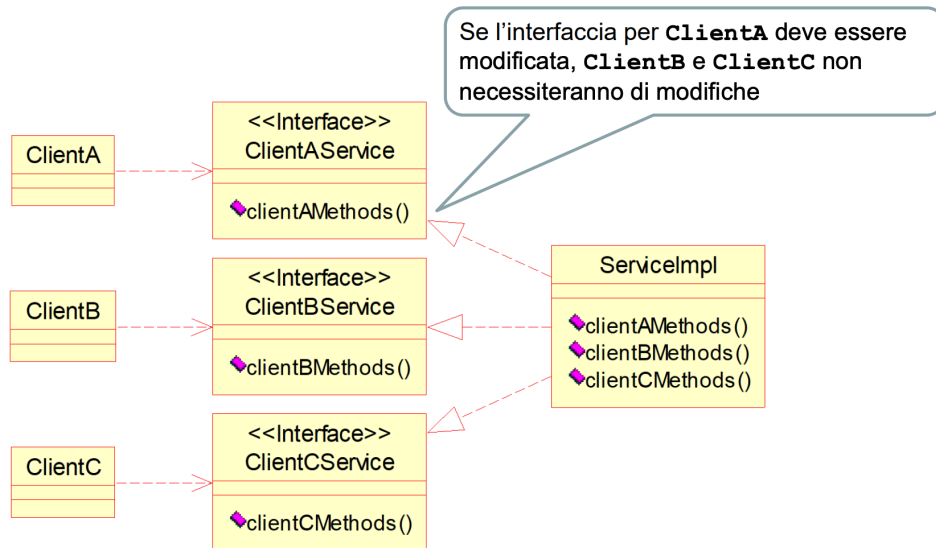
The Interface Segregation Principle / Principio di segregazione delle interfacce

I clienti non dovrebbero dipendere da servizi che non utilizzano.

Le interfacce che gestiscono più clienti (fat interface) creano una forma indiretta di accoppiamento inutile fra i clienti: se un cliente richiede l'aggiunta di una nuova funzionalità all'interfaccia, ogni altro cliente è costretto a cambiare anche se non è interessato alla nuova funzionalità.



È consigliato quindi creare interfacce specifiche per ogni tipo di cliente e implementare tutte le interfacce nella classe.



The Open/Closed Principle / Principio aperto/chiuso

Le entità software dovrebbero essere aperte per l'estensione, ma chiuse per la modifica.

- Open: possono essere estese aggiungendo nuovo stato o proprietà comportamentali.
- Closed: hanno un'interfaccia ben-definita, pubblica e stabile che non può essere cambiata.

Per fare ciò è necessario l'utilizzo di interfacce (o classi astratte) → a un'interfaccia immutabile possono corrispondere innumerevoli classi concrete che realizzano comportamenti diversi. In questo modo il modulo che utilizza l'astrazione non avrà mai bisogno di essere modificato, al più l'astrazione potrà esibire nuovi comportamenti se necessario.

The Liskov Substitution Principle / Principio di sostituibilità di Liskov

Le sottoclassi dovrebbero essere sostituibili con le loro classi base.

Un cliente che usa istanze di una classe A deve poter usare istanze di una qualsiasi sottoclasse di A senza accorgersi della differenza.

La principale causa di violazioni al principio di Liskov è dato dalla ridefinizione sbagliata di metodi virtuali nelle classi derivate. La chiave per evitare le violazioni al principio di Liskov risiede nel Design by Contract. Nel Design by Contract ogni metodo ha un contratto tra chi invoca il metodo e il metodo stesso, costituito da:

- un insieme di pre-condizioni: requisiti minimi che devono essere soddisfatti dal chiamante perché il metodo possa essere eseguito correttamente
- un insieme di post-condizioni: requisiti che devono essere soddisfatti dal metodo nel caso di esecuzione corretta

Quando un metodo viene ridefinito in una sottoclasse:

- le pre-condizioni devono essere identiche o meno stringenti
- le post-condizioni devono essere identiche o più stringenti

Affinché LSP possa valere, e con esso OCP, tutti i sottotipi devono essere conformi al comportamento che i clienti si aspettano dalle classi base che utilizzano.

Principi di architettura dei package

Release/Reuse Equivalency Principle / Principio di equivalenza di riuso/rilascio

L'unità di riutilizzo dovrebbe coincidere con l'unità di rilascio.

Un elemento riutilizzabile, sia esso un componente, una classe o un insieme di classi, non può essere riutilizzato a meno che non sia gestito da un sistema di rilascio di qualche tipo.

Common Closure Principle / Principio di chiusura comune

Le classi che cambiano insieme, devono stare insieme.

Più pacchetti cambiano in un dato rilascio, maggiore è il lavoro per ricostruire, testare e distribuire il rilascio. Per questo si punta a ridurre al minimo il numero

di pacchetti che vengono modificati in un dato ciclo di rilascio del prodotto. Per raggiungere questo obiettivo, raggruppiamo assieme classi che pensiamo cambieranno insieme.

Common Reuse Principle / Principio di riutilizzo comune

| Classi che vengono riutilizzate insieme non dovrebbero essere raggruppate insieme.

Quando un package cambia e il suo numero di rilascio viene aggiornato, tutti i client di quel package devono verificare di funzionare con il nuovo package, anche se nulla di ciò che hanno usato all'interno del package è effettivamente cambiato. Pertanto, le classi che non vengono riutilizzate insieme non dovrebbero essere raggruppate insieme.

Discussione

Questi tre principi non possono essere soddisfatti contemporaneamente: REP e CRP semplificano la vita ai riutilizzatori, mentre CCP semplifica la vita ai manutentori. CCP cerca infatti di rendere i package più grandi possibile; CRP, tuttavia, cerca di creare package molto piccoli.

All'inizio di un progetto, gli architetti possono impostare la struttura dei package in modo tale che CCP domini per facilità di sviluppo e manutenzione. Successivamente, quando l'architettura si stabilizza, gli architetti possono rifattorizzare la struttura dei package per massimizzare REP e CRP per i riutilizzatori esterni.

Relazioni tra i package

Acyclic Dependencies Principle

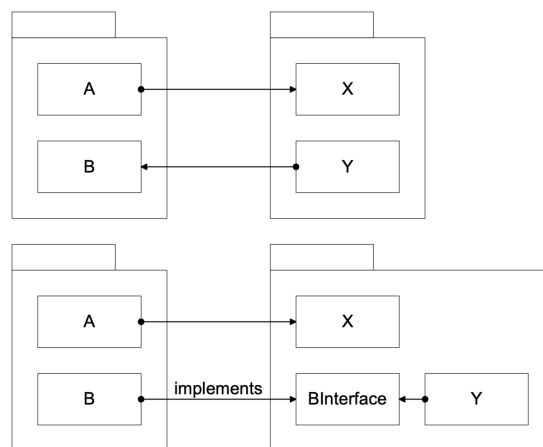
| Le dipendenze tra package non devono formare cicli.

Una volta apportate le modifiche a un package, gli sviluppatori possono rilasciare i package al resto del progetto. Prima di poter eseguire questo rilascio, tuttavia, devono verificare che il package funzioni. Per farlo, devono compilarlo e collegarlo a tutti i package da cui dipende. Una singola dipendenza ciclica che sfugge al controllo può rendere l'elenco delle

dipendenze molto lungo. Quindi, qualcuno deve osservare la struttura delle dipendenze dei package con regolarità e interrompere i cicli ovunque compaiano.

Come rompere un ciclo:

- Inframezzare un nuovo package
- Aggiungere una nuova interfaccia



Stable Dependencies Principle / Principio delle dipendenze stabili

Le dipendenze tra package in un design dovrebbero puntare nella direzione della stabilità dei package. Un package dovrebbe dipendere solo da package che sono più stabili.

Un pacchetto con molte dipendenze in entrata è molto stabile perché richiede molto lavoro per riconciliare qualsiasi modifica con tutti i pacchetti dipendenti.

Stable Abstractions Principle / Principio delle astrazioni stabili

Package stabili dovrebbero essere package astratti.

I package altamente stabili nella parte inferiore del grafo delle dipendenze possono essere molto difficili da modificare, ma secondo l'OCP non devono essere difficili da estendere. Se anche i pacchetti stabili in fondo sono molto astratti, possono essere facilmente estesi.

È possibile creare la nostra applicazione a partire da package instabili facili da modificare e package stabili facili da estendere.

▼ 11.0 - Design pattern

Design pattern

L'obiettivo di un design pattern è quello di risolvere problemi progettuali specifici e di rendere i progetti object-oriented più flessibili e riutilizzabili.

Ogni design pattern cattura e formalizza l'esperienza acquisita nell'affrontare e risolvere uno specifico problema progettuale e permette di riutilizzare tale esperienza in altri casi simil.

Ogni design pattern ha quattro elementi essenziali:

- un nome (significativo) – identifica il pattern
- il problema – descrive quando applicare il pattern
- la soluzione – descrive il pattern, cioè gli elementi che lo compongono (classi e istanze) e le loro relazioni, responsabilità e collaborazioni
- le conseguenze – descrivono vantaggi e svantaggi dell'applicazione del pattern e permettono di valutare le alternative progettuali

Classificazione dei design pattern

- Pattern di creazione (creational pattern): risolvono problemi inerenti il processo di creazione di oggetti
- Pattern strutturali (structural pattern): risolvono problemi inerenti la composizione di classi o di oggetti
- Pattern comportamentali (behavioural pattern): risolvono problemi inerenti le modalità di interazione e di distribuzione delle responsabilità tra classi o tra oggetti

<i>Pattern di creazione</i>	<i>Pattern strutturali</i>	<i>Pattern comportamentali</i>
Abstract Factory Builder Factory Method Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Interpreter Iterator Mediator Memento Observer State Strategy Template Method Visitor

Pattern Singleton

Assicura che una classe abbia una sola istanza e fornisce un punto di accesso globale a tale istanza.

La classe deve:

- tenere traccia della sua sola istanza
- intercettare tutte le richieste di creazione, al fine di garantire che nessuna altra istanza venga creata
- fornire un modo per accedere all'istanza unica

Un alternativa sarebbe quella di utilizzare una classe non istanziabile (static class), ma si preferiscono singleton poichè questi possono implementare interfacce e possono ritornare istanze specializzate a seconda del contesto.

```
public class Singleton
{
    ... attributi membro di istanza ...
    private static Singleton _instance = null;

    protected Singleton()
    { inizializzazione istanza }

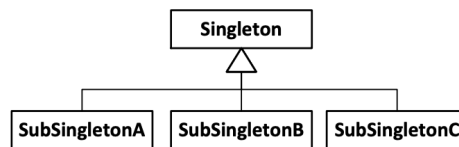
    public static Singleton GetInstance()
```

```

{
    if(_instance == null) _instance = CreateInstance();
    return _instance;
}

private static Singleton CreateInstance()
{
    if(...) return new SubSingletonA();
    else if(...) return new SubSingletonB();
    else return new SubSingletonC();
}
}

```



Pattern Observer

A volte, quando un oggetto cambia (es. stato, dati...), altri oggetti devono reagire a quel cambiamento. Un modo semplice per farlo è codificare direttamente nel soggetto cosa devono fare gli altri oggetti quando cambia. Ma così facendo, il soggetto deve conoscere i dettagli degli osservatori, creando un forte accoppiamento (closely coupled). Questo rende il codice meno flessibile e difficile da riutilizzare o estendere.

Il pattern Observer propone di creare una relazione uno-a-molti "lasca" (loose coupling) tra un oggetto soggetto e i suoi osservatori. Il soggetto non sa nulla di come gli osservatori reagiranno, sa solo che deve notificare tutti quando cambia. Gli osservatori ricevono la notifica e decidono da soli come aggiornarsi, mantenendo il codice modulare e riutilizzabile.

Esempio weather station (soggetto) che, ogni volta che cambia temperatura, deve notificare il display per cambiare a sua volta.


```

public interface IObservable
{
    void Update(float temperature);
}

public interface ISubject
{
    void RegisterObserver(IObservable o);
    void RemoveObserver(IObservable o);
    void NotifyObservers();
}

// soggetto
public class WeatherStation : ISubject
{
    private List<IObservable> observers = new List<IObservable>();
    private float temperature;

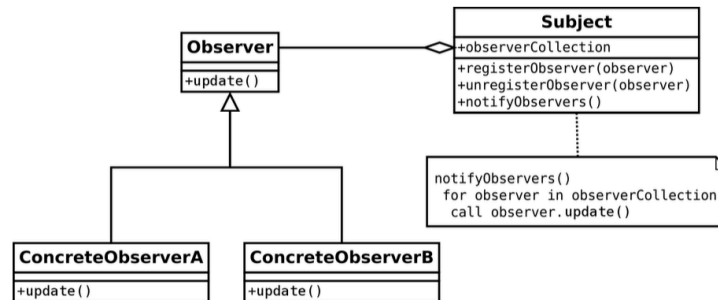
    public void RegisterObserver(IObservable o) ⇒ observers.Add(o);
    public void RemoveObserver(IObservable o) ⇒ observers.Remove(o);

    public void SetTemperature(float temp)
    {
        temperature = temp;
        NotifyObservers(); // ogni volta che cambia, avvisa tutti
    }

    public void NotifyObservers()
    {
        foreach (var o in observers)
        {
            o.Update(temperature);
        }
    }
}

```

```
// osservatore
public class PhoneDisplay : IObservable
{
    public void Update(float temperature)
    {
        Console.WriteLine($"Telefono: nuova temperatura = {temperature}");
    }
}
```



Pattern Model View Controller (MVC)

Permette di suddividere un'applicazione in tre parti:

- **Modello:** elaborazione/stato

Gestisce un insieme di dati logicamente correlati, rispondendo alle interrogazioni e alle istruzioni di modifica sullo stato di questi.

Genera un evento quando lo stato cambia registrando (in forma anonima) gli oggetti interessati alla notifica dell'evento.

- **View:** output

Gestisce un'area di visualizzazione, nella quale presenta all'utente una vista dei dati gestiti dal modello.

Si registra presso il modello per ricevere l'evento di cambiamento di stato.

- **Controller:** input

Gestisce gli input dell'utente (mouse, tastiera, ...) e invia i comandi corrispondenti al modello e/o alla view che effettuano le operazioni

appropriate.

Pattern Flyweight

Un flyweight è un oggetto condiviso che può essere utilizzato simultaneamente ed efficientemente da più clienti.

Per assicurare una corretta condivisione, i clienti non devono istanziare direttamente i flyweight, ma devono ottenerli esclusivamente tramite una FlyweightFactory.

```
private Dictionary<KeyType, FlyweightType> flyweights;

public FlyweightType GetFlyweight(KeyType key)
{
    if (!flyweights.ContainsKey(key))
    {
        flyweights.Add(key, CreateFlyweight(key));
    }
    return flyweights[key];
}
```

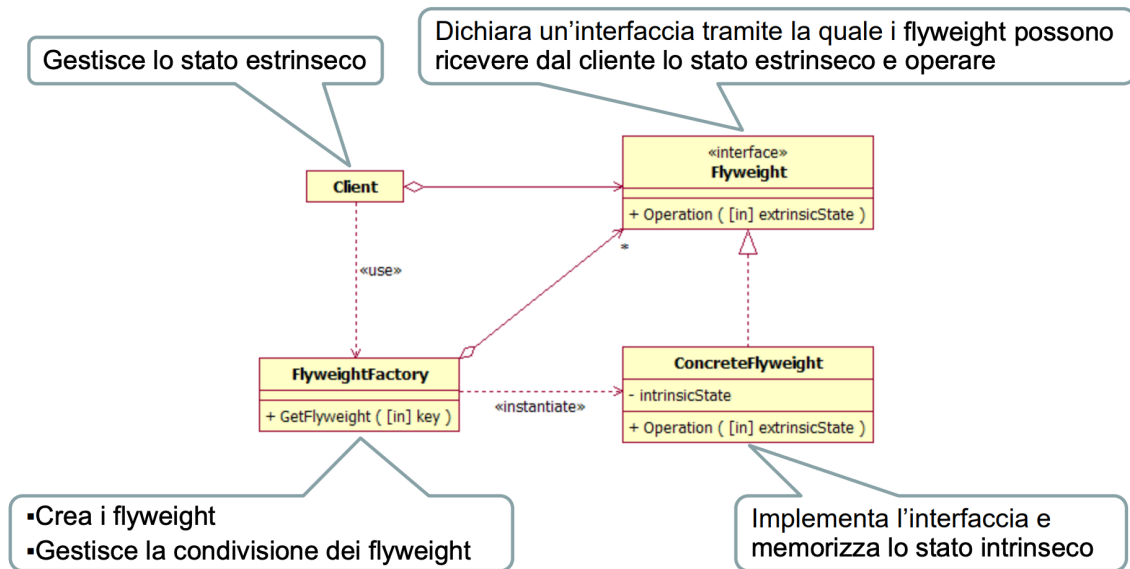
Nel pattern flyweight dividiamo tra:

- Stato intrinseco

Non dipende dal contesto di utilizzo e quindi può essere condiviso da tutti i clienti. Viene per questo memorizzato all'interno del flyweight.

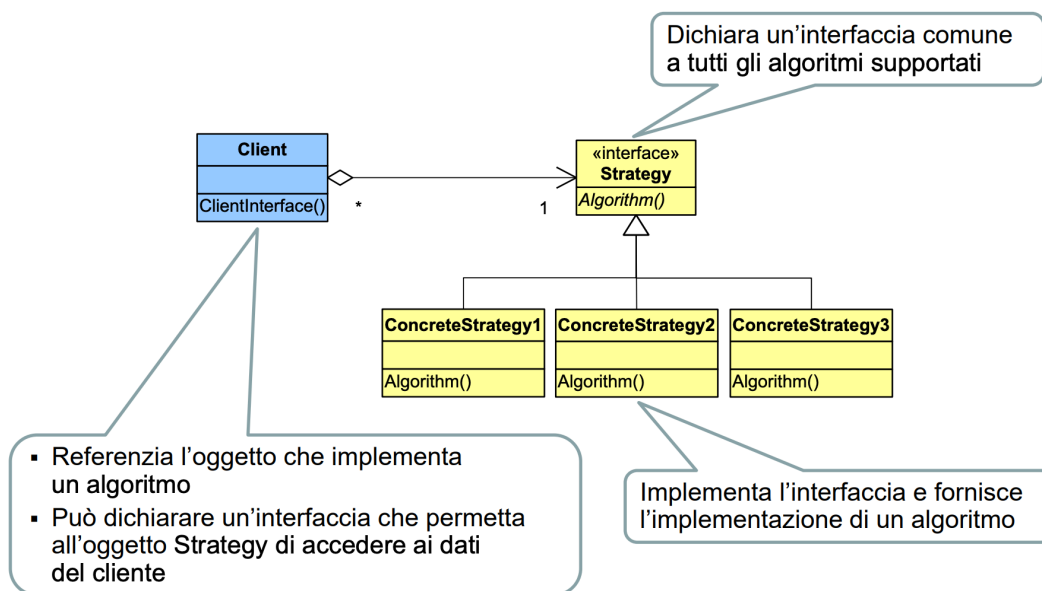
- Stato estrinseco

Dipende dal contesto di utilizzo e quindi non può essere condiviso dai clienti. Viene dunque calcolato e memorizzato dal cliente, e passato al flyweight quando viene invocata una sua operazione.

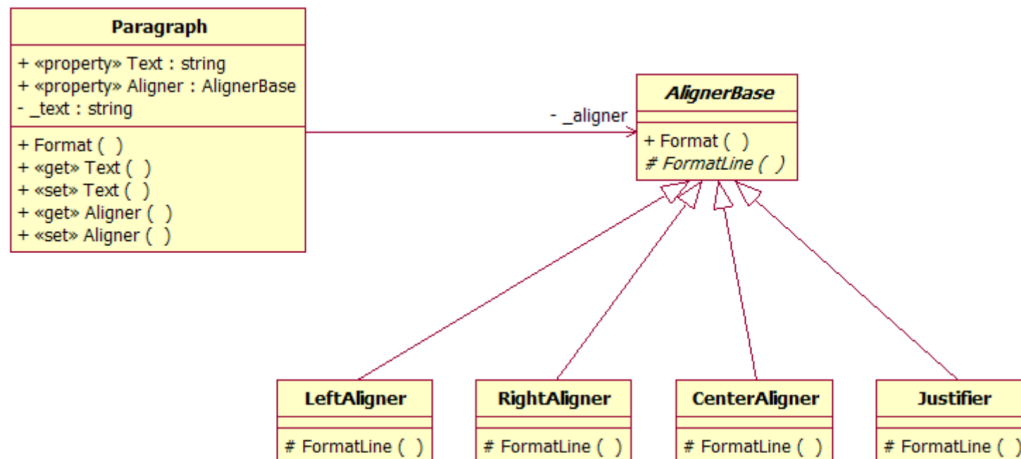


Pattern Strategy

Permette di definire un insieme di algoritmi tra loro correlati, incapsulare tali algoritmi in una gerarchia di classi e rendere gli algoritmi intercambiabili.



Un esempio è quello dell'allineamento del testo di un paragrafo. Esistono politiche diverse di allineamento, dunque è possibile incapsularle tramite pattern strategy.

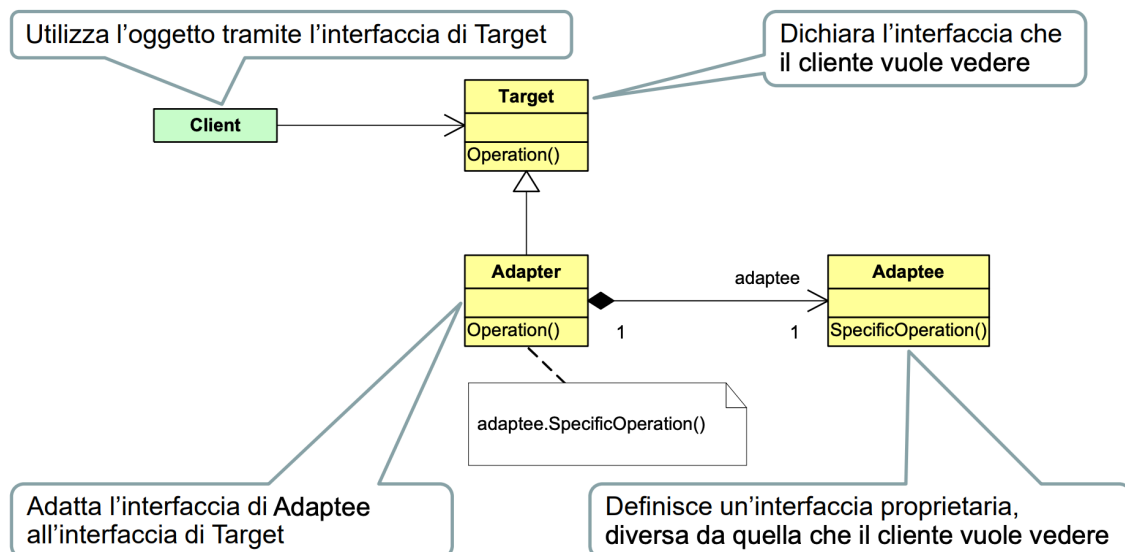


Pattern Adapter

Permettere a due interfacce incompatibili di lavorare insieme, adattandole tra loro. È utile quando occorre utilizzare, seguendo una certa interfaccia, una classe esistente che non rispetta tale interfaccia e non può essere modificata.

Troviamo diversi ruoli:

- target: interfaccia attesa dal cliente
- adaptee: classe esistente con un'interfaccia incompatibile
- adapter: traduce l'interfaccia del target in chiamate all'adaptee



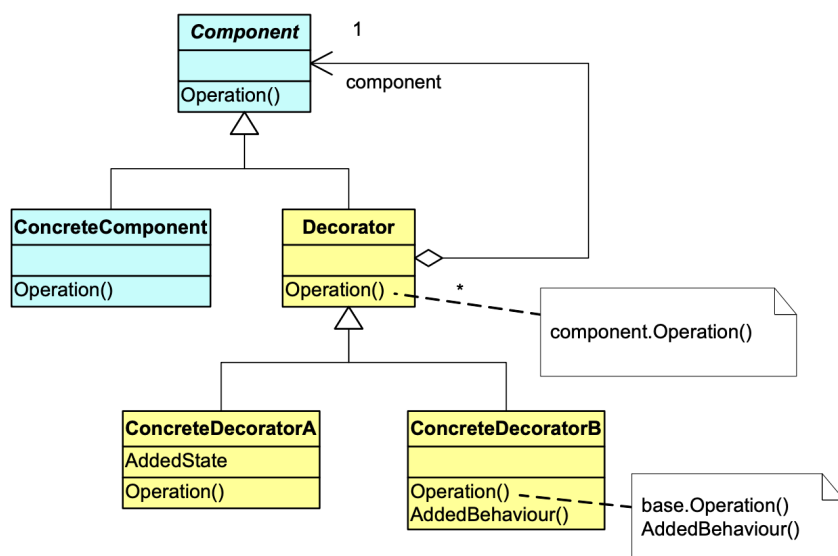
Pattern Decorator

Aggiungere dinamicamente nuove funzionalità a un oggetto senza modificarne il codice sorgente.

Si ha un'interfaccia base (Component) che definisce un comportamento, e una classe concreta che implementa quel comportamento. Si può utilizzare il pattern decorator nel caso in cui si voglia aggiungere responsabilità (nuove funzionalità) senza ereditare o toccare il codice esistente. Si creano dunque dei decorator che implementano la stessa interfaccia, avvolgono l'oggetto originale, e aggiungono funzionalità extra.

Si ha dunque:

- Component (interfaccia o classe astratta): dichiara l'interfaccia di tutti gli oggetti ai quali deve essere possibile aggiungere dinamicamente responsabilità.
- ConcreteComponent: definisce un tipo di oggetto al quale deve essere possibile aggiungere dinamicamente responsabilità.
- Decorator (classe astratta): mantiene un riferimento a un oggetto di tipo Component e definisce un'interfaccia conforme all'interfaccia di Component.
- ConcreteDecorator: aggiunge responsabilità al componente referenziato.



Pattern State

Permettere a un oggetto di cambiare comportamento quando cambia il suo stato interno.

Ogni stato diventa una classe con i suoi comportamenti. Il contesto mantiene un riferimento allo stato attuale. Quando lo stato cambia, il contesto delega il comportamento allo stato attuale.

```
// State
interface IState
{
    void InsertCoin();
    void EjectCoin();
    void Dispense();
}

// ConcreteState 1
class NoCoinState : IState
{
    private VendingMachine _machine;
    public NoCoinState(VendingMachine machine) ⇒ _machine = machine;

    public void InsertCoin()
    {
        Console.WriteLine("Moneta inserita.");
        _machine.SetState(_machine.HasCoinState);
    }

    public void EjectCoin() ⇒ Console.WriteLine("Nessuna moneta da restituire.");
    public void Dispense() ⇒ Console.WriteLine("Inserisci prima una moneta.");
}

// ConcreteState 2
class HasCoinState : IState
```

```

{
    private VendingMachine _machine;
    public HasCoinState(VendingMachine machine) ⇒ _machine = machine;

    public void InsertCoin() ⇒ Console.WriteLine("Hai già inserito una moneta.");
    public void EjectCoin()
    {
        Console.WriteLine("Moneta restituita.");
        _machine.SetState(_machine.NoCoinState);
    }

    public void Dispense()
    {
        Console.WriteLine("Erogazione in corso...");
        _machine.SetState(_machine.NoCoinState);
    }
}

// Context
class VendingMachine
{
    public IState NoCoinState { get; }
    public IState HasCoinState { get; }

    private IState _currentState;

    public VendingMachine()
    {
        NoCoinState = new NoCoinState(this);
        HasCoinState = new HasCoinState(this);
        _currentState = NoCoinState;
    }

    public void SetState(IState newState) ⇒ _currentState = newState;
}

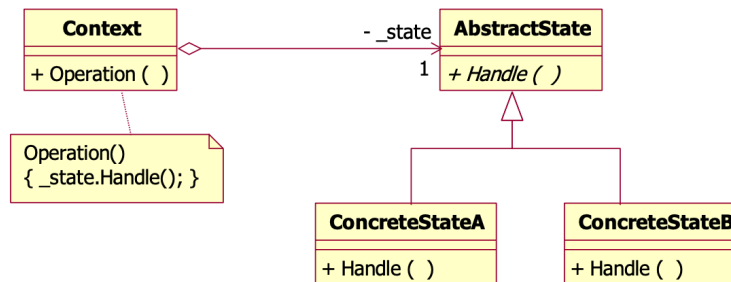
```



```

public void InsertCoin() ⇒ _currentState.InsertCoin();
public void EjectCoin() ⇒ _currentState.EjectCoin();
public void Dispense() ⇒ _currentState.Dispense();
}

```

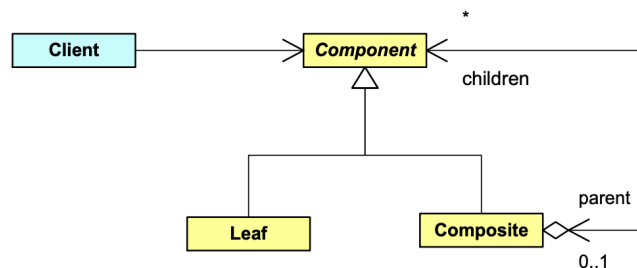


Pattern Composite

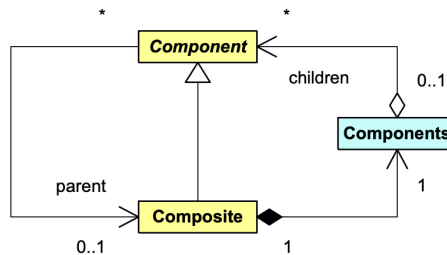
Si utilizza in sistemi che hanno una struttura ad albero in cui si vogliono trattare singoli oggetti e gruppi di oggetti allo stesso modo.

È composto da:

- Component (classe astratta): dichiara l'interfaccia e realizza il comportamento di default.
- Client: accede e manipola gli oggetti della composizione attraverso l'interfaccia di Component.
- Leaf: descrive oggetti che non possono avere figli.
- Composite: descrive oggetti che possono avere figli.



Composite presenta un attributo che rappresenta il contenitore dei figli e può essere di qualsiasi tipo (array, lista, albero, tabella hash, ...).



Siccome il client vede solo l'interfaccia di component, all'interno di questa devono essere inserite tutte le operazioni che devono essere utilizzate dai clienti. Alcune di queste operazioni possono essere prive di significato per gli oggetti foglia (Add, Remove, ...). Possiamo dunque scegliere tra due approcci:

- Trasparenza: dichiaro tutto al livello più alto, in modo che il cliente possa trattare gli oggetti in modo uniforme, ma il cliente potrebbe cercare di fare cose senza senso, come aggiungere figli alle foglie.
- Sicurezza: tutte le operazioni sui figli vengono messe in Composite, a questo punto, qualsiasi invocazione sulle foglie genera un errore in fase di compilazione, quindi il cliente deve effettuare gli opportuni controlli e conoscere due interfacce differenti.

Pattern Visitor

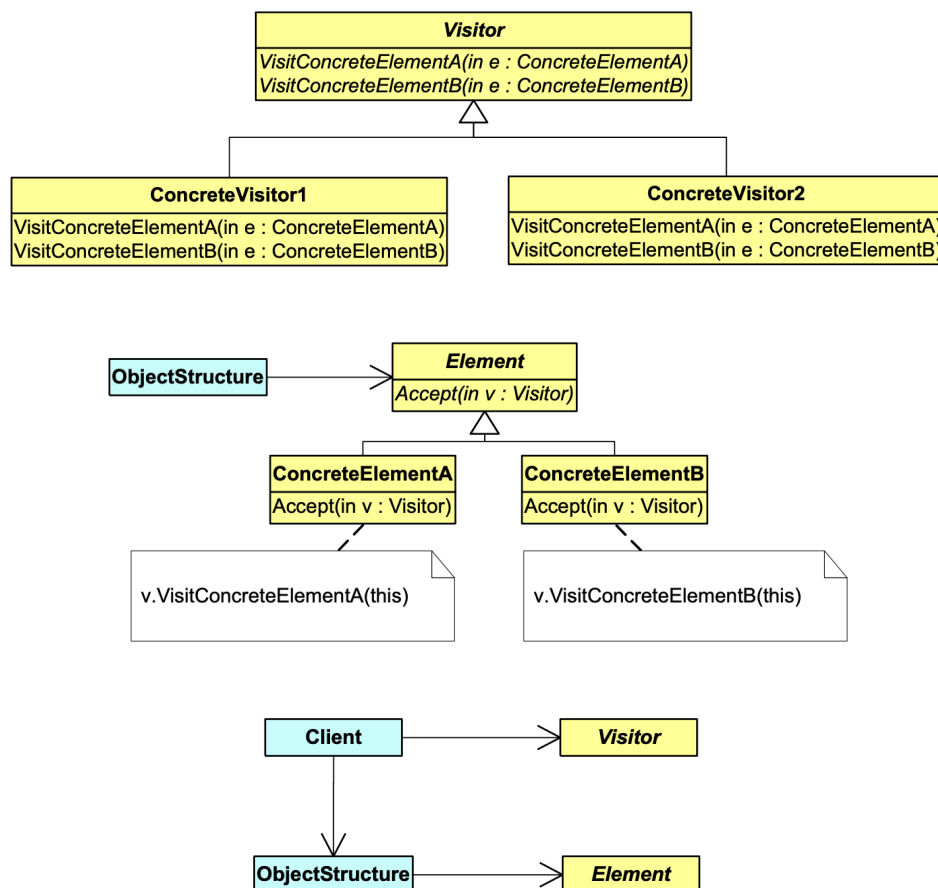
Permette di definire una nuova operazione da effettuare su gli elementi di una struttura, senza dover modificare le classi degli elementi coinvolti.

Vengono quindi separate le operazioni dagli oggetti su cui vengono eseguite
→ ogni operazione viene raccolta in una singola classe apposita, chiamata visitor, la quale presenta un'operazione per ogni tipo di oggetto. Gli oggetti devono invece accettare la visita delle istanze di queste nuove classi. In questo modo, per aggiungere una nuova operazione, è sufficiente progettare una nuova classe visitor.

Abbiamo dunque questi elementi:

- Visitor (classe astratta o interfaccia): dichiara un metodo Visit per ogni classe di elementi concreti.

- ConcreteVisitor: definisce tutti i metodi Visit.
- Element (classe astratta o interfaccia): dichiara un metodo Accept che accetta un Visitor come argomento.
- ConcreteElement: definisce il metodo Accept.
- ObjectStructure: può essere realizzata come Composite o come normale collezione (array, lista, ...), deve poter enumerare i suoi elementi. Deve dichiarare un'interfaccia che permetta a un cliente di far visitare la struttura a un Visitor.



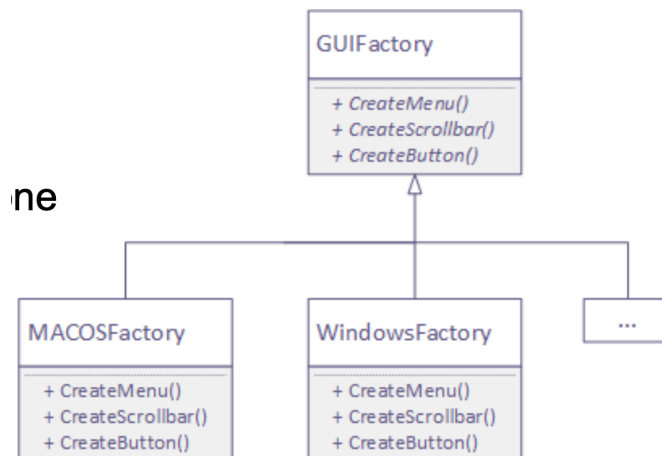
Occorre considerare i seguenti aspetti del pattern visitor:

- Incapsulamento: ogni Visitor deve essere in grado di accedere allo stato degli elementi su cui deve operare.

- La gerarchia composta da Element deve essere stabile, poiché non è semplice aggiungere nuovi ConcreteElement: ciò implicherebbe definire in tutti i Visitor esistenti un metodo apposito visit per quel ConcreteElement.
- Stato: durante l'operazione ogni Visitor può modificare il proprio stato – ad esempio, per accumulare dei valori o altro.
- L'operazione Accept è di tipo double dispatch, poiché dipende sia da Visitor che da Element.

Pattern Abstract Factory

Il pattern Abstract Factory fornisce un'interfaccia per creare famiglie di oggetti correlati senza conoscere le loro classi concrete. In questo modo si ha un sistema indipendente da come gli oggetti vengono creati, composti e rappresentati.



Conseguenze di questo pattern:

- Cambiare in modo semplice la famiglia di prodotti utilizzata. Questo è garantito dalla presenza dell'interfaccia.
- Promuove la coerenza nell'utilizzo dei prodotti.
- Difficile aggiungere supporto per nuove tipologie di prodotti.

▼ 12.0 - Dalla progettazione all'implementazione

Implementazione delle associazioni

Associazioni con molteplicità 0..1 e 1..1

Si aggiunge alla classe cliente un attributo membro che rappresenta:

- il riferimento all'oggetto della classe fornitore
- e/o l'identificatore univoco dell'oggetto della classe fornitore (solo se persistente)
- o il valore dell'oggetto della classe fornitore (solo nel caso di composizione e molteplicità 1..1)

Associazioni con molteplicità 0..* o 1..*

Aggiungere alla classe cliente un attributo membro che referencia un'istanza di una classe contenitore. Una classe contenitore è una classe le cui istanze sono collezioni di (riferimenti a) oggetti della classe fornitore. Esempi di classi contenitore sono vettori, stack, liste, alberi, ecc.

I contenitori possono essere classificati in funzione

- del modo in cui contengono gli oggetti
 - contenimento per riferimento

Qui ogni oggetto esiste per conto proprio, quindi può essere in più contenitori contemporaneamente. Nel contenitore viene memorizzato solo il riferimento, dunque la distruzione del contenitore non comporta la distruzione dell'oggetto.
 - contenimento per valore

Ogni oggetto esiste solo perché contenuto in un altro oggetto. Nel contenitore viene duplicato l'oggetto, quindi la distruzione del contenitore comporta anche la distruzione dell'oggetto.
- dell'omogeneità o eterogeneità di tali oggetti
 - oggetti omogenei: tutti gli oggetti contenuti sono dello stesso tipo
 - oggetti eterogenei: gli oggetti contenuti possono essere di tipo diverso

Per implementare un contenitore di oggetti eterogenei è necessaria l'ereditarietà. Il tipo contenuto nella classe contenitore deve dunque essere la superclasse radice della gerarchia (nel peggiore dei casi, object).

Identificazione degli oggetti

Per poter mantenere un riferimento ad un oggetto, occorre poterlo identificare univocamente.

Nel caso di strutture dati interamente contenute nello spazio di indirizzamento dell'applicazione, un oggetto può essere identificato univocamente mediante il suo indirizzo (logico) di memoria.

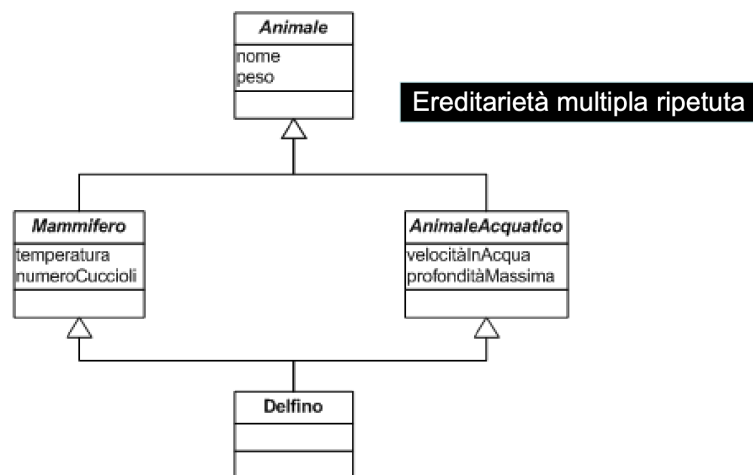
Nel caso di database o di sistemi distribuiti, a ogni oggetto deve essere associato un identificatore univoco persistente tramite il quale deve essere possibile risalire all'oggetto stesso, sia che risieda in memoria, su disco o in rete.

L'identificatore univoco è un attributo che al momento della creazione dell'oggetto viene inizializzato con:

- un valore generato automaticamente dal sistema
- il valore della chiave primaria di una tabella relazionale

Implementazione dell'ereditarietà multipla

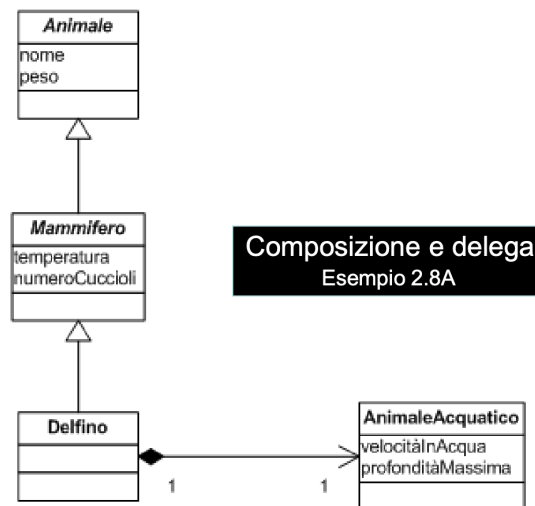
Se in progettazione esistono strutture con ereditarietà multipla ma il linguaggio di programmazione non ammette l'ereditarietà multipla, è necessario convertire le strutture con ereditarietà multipla in strutture con solo ereditarietà semplice.



Ci sono due possibilità:

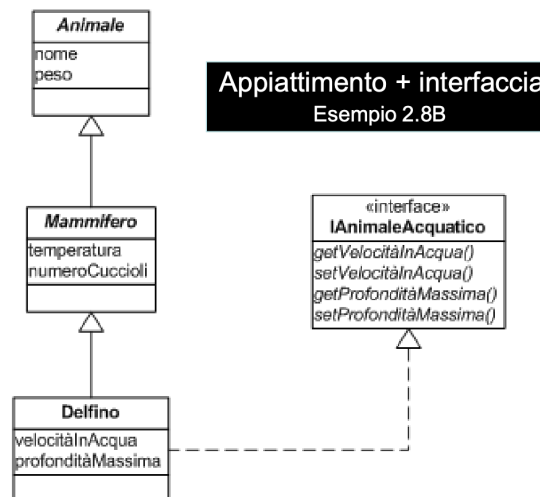
- Composizione e delega

Scegliere la più significativa tra le superclassi ed ereditare esclusivamente da questa. Le caratteristiche delle superclassi escluse vengono incorporate nella classe specializzata tramite composizione e delega e non tramite ereditarietà.



- Interfaccia

Appiattire tutto in una gerarchia semplice e implementare un'interfaccia.



Miglioramento delle prestazioni

A volte, per migliorare le prestazioni, occorre un oggetto possa accedere direttamente ai valori di un altro oggetto, aggirando così l'incapsulamento. Questo tipo di modifica deve essere presa in considerazione solo dopo che tutti gli altri aspetti del progetto sono stati soggetti a misure e modifiche.

▼ 13.0 - Version Control Systems

VCS

Tutti i software hanno diverse versioni (versioni di test, versioni di rilascio, versioni per diverse piattaforme). Tramite il Version Control è possibile tenere traccia di tutte queste versioni. In particolare, permette di controllare vecchie versioni e di far esistere più versioni contemporaneamente.

La soluzione home made per gestire parte di questi problemi è quella di mantenere copie multiple delle differenti versioni del programma. Questo però è inefficiente e bisogna gestire tutta la parte di autorizzazioni in maniera manuale.

Un VCS consente invece di:

- memorizzare codice sorgente
- fornire una cronologia delle modifiche
- lavorare in parallelo a diversi aspetti del software senza interferenze
- fornisce un modello di sviluppo
- incrementa la produttività

Progetto

Un progetto è un insieme di file di qualsiasi tipo in version control.

Repository

Posto in cui sono memorizzati i file correnti e storici dei file. Tipicamente risiede su un server remoto sicuro e affidabile.

Working folder

Copia locale dei file di una repository. Tutto il lavoro svolto sui file in una repository viene inizialmente svolto in una working folder.

L'azione di copia del contenuto di una repository su una working folder è detta check out.

Quando l'utente finisce di modificare i file e invia le modifiche alla repository, viene detto check in.

Workflow

Insieme delle seguenti azioni:

- copia dei file dalla repository alla working folder (check out)
- modifica del codice all'interno della working folder
- aggiornamento della repository dalla working folder (check in)
- ripeti

Revisions

Nuova versione di un file in version control, data da:

1. check out
2. modifiche
3. check in

Per efficienza, in una revision non viene memorizzato l'intero file, ma solo:

- le differenze con la versione precedente
- il nuovo numero di versione
- altri metadati (autore, log ecc.)

Questo fa diventare il check in e il check out un pò più lenti, visto che devono essere applicate le differenze con tutte le versioni precedenti per calcolare il file corrente.

Branches

Un branch è la divergenza di due revisions che partono da una stessa base.

Lock-Modify-Unlock model (LMU)

In questo modello la repository consente a una sola persona di modificare un file nello stesso momento. Ogni volta che un utente vuole modificare un file,

dovrebbe prima effettuare il lock di questo. Se un file è locked, nessun altro utente può modificarlo. Quando l'utente termina la modifica, deve effettuare l'unlock.

Questo modello di lavoro può però portare ad alcuni problemi:

- Problemi amministrativi, ad esempio se un utente si dimentica di effettuare l'unlock di un file.
- A volte non è necessario effettuare un lock per modificarlo, ad esempio se un utente vuole modificare la parte iniziale di un file e un altro utente la parte finale.
- Non è possibile effettuare il lock di un file se si lavora offline su una working folder.

Copy-Modify-Merge model (CMM)

In questo modello non esistono lock. Ogni volta che qualcuno effettua il check-in di una copia di lavoro, le sue modifiche vengono unite a quelle degli altri utenti.

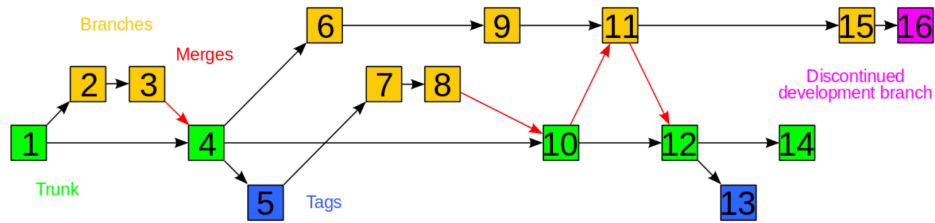
Il check-in può quindi avere due risultati:

- Successo: la versione finale non dipende dall'ordine con cui vengono applicate le modifiche.
- Conflitto: accade quando due utenti modificano lo stesso pezzo di codice, quindi la versione finale dipende dall'ordine con cui vengono applicate le modifiche. I conflitti devono essere risolti a mano.

L'assenza di conflitti non assicura che le modifiche realizzate da utenti differenti lavorano insieme, ma solo che non hanno modificato le stesse linee di codice in maniera differente.

The revision tree

CMM porta alla formazione di una linea di sviluppo (trunk) con branch che vanno a creare un albero chiamato revision tree. In realtà è un grafo aciclico, ma viene comunque chiamato albero per approssimazione.



Branch

È possibile effettuare un branch di un insieme di file in CMM. In questo modo due copie di tali file possono essere sviluppate in maniera indipendente. Quando gli update sono completati, si può effettuare il merge di questi branch nel main trunk.

Tag

Un tag è un segnalibro che punta a un commit specifico e serve per marcare una versione importante del progetto, come una release.

Push/Pull

Push e pull consentono di copiare revisions da una repository ad un'altra. La sorgente fa push, mentre la ricevente fa pull.

Push e pull scambiano solamente le modifiche effettuate, non i file effettivi.

LMU vs CMM

CMM è migliore nella maggior parte dei casi perché la quantità di tempo che viene spesa per risolvere conflitti è generalmente minore di quella persa utilizzando un sistema di lock.

LMU è migliore nei casi in cui si utilizzano file che non si possono unire (es. immagini).

Distributed VCS

Con l'utilizzo di CMM, la repository centrale inizia a perdere la sua importanza. Si iniziano così a diffondere distributed CVS, nelle quali la working folder di ogni peer agisce da repository, avendo al suo interno la cronologia completa, possibilità di effettuare commit, branch e merge e possibilità di lavorare anche offline.

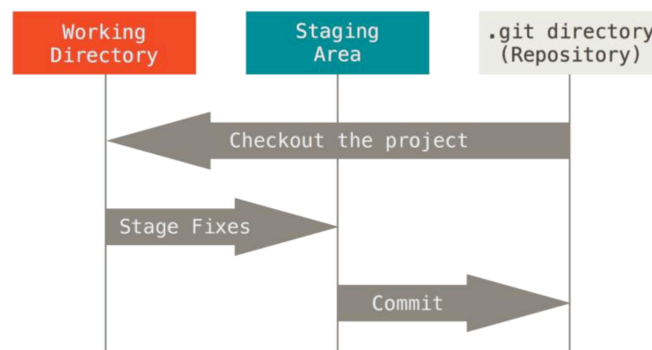
Nelle distributed VCS la sincronizzazione tra peer avviene scambiando patch (set di modifiche). La visualizzazione della cronologia e i commit avvengono molto più velocemente rispetto a un CVCS in quanto sono effettuati in locale.

Contro delle DCVS:

- C'è bisogno di mantenere un backup, in quanto gli altri utenti potrebbero non aver ancora accettato le tue modifiche.
- Non esiste una vera e propria versione più recente.
- Non ci sono numeri delle revisioni. Ci sono solo GUID che identificano le singole modifiche, ma non sono numeri di release (per questo c'è bisogno dei tag).

Git

Presenta una directory, ovvero una main repository centralizzata. Esistono working directory, ovvero copie locali della repository. Esiste una staging area, ovvero un file in cui vengono specificati i file modificati che devono essere salvati nella repository.



Un file può trovarsi dunque in tre stati:

- Committed: file salvato nella repository locale.
- Modified: file modificato, ma di cui non è ancora stato fatto il commit.
- Staged: file contrassegnato per essere parte del prossimo commit.

Git non memorizza solo le differenze tra le versioni, ma salva ogni commit come una istantanea completa dell'intero progetto.

Modulo 2

▼ 1.0 - Modelli, Linguaggi e Modelli di Processo di sviluppo

Modello

Per l'ingegnere del software un modello costituisce una visione semplificata di un sistema che rende il sistema stesso:

- Più accessibile alla comprensione e alla valutazione.
- Facilita il trasferimento di informazione e la collaborazione tra persone.

I processi model-based utilizzano diagrammi espressi con notazioni formali, cioè attraverso l'uso di un linguaggio con sintassi e semantica ben precise.

L'insieme dei modelli che descrivono un sistema dovrebbe formare una descrizione completa, consistente e non troppo ridondante. La transizione tra un modello e l'altro deve essere continua. Inoltre, i modelli sono connessi tra loro in modo sistematico: un elemento in un modello deve avere il suo o i suoi corrispettivi in un altro.

La proprietà della tracciabilità afferma che in qualsiasi direzione si percorra la sequenza di modelli generati, deve essere possibile mappare uno o più elementi in un modello in uno o più elementi in un altro.

Linguaggi di modellazione

Un linguaggio di modellazione è un linguaggio semiformale che può essere utilizzato per descrivere un sistema di qualche natura. Abbiamo bisogno di un tale linguaggio poichè il linguaggio naturale è troppo impreciso, mentre il codice è preciso, ma troppo dettagliato.

Con le attuali tecnologie a disposizione abbiamo la necessità di creare sia modelli ad alto livello di astrazione, sia il codice. Questo però crea un problema di fondo, in quanto i modelli ed il codice sono tipicamente disallineati. L'approccio migliore per mantenere l'allineamento e la tracciabilità è apportare le modifiche direttamente al modello di progettazione e, ove possibile, generare codice da questo attraverso opportuni generatori di codice, mentre negli altri casi modificare direttamente a mano il codice coerentemente con il modello.

Tipi di linguaggi di modellazione sono i seguenti:

- Modelli semantici dei dati
 - Entità-relazioni (E-R)
- Modelli orientati all'elaborazione dati
 - Diagrammi di flusso dei dati
- Modelli orientati alla classificazione
 - Modelli orientati agli oggetti
- Modelli operazionali
 - Automi a stati finiti
 - Reti di Petri
- Modelli descrittivi
 - Logica del primo ordine
 - Logica temporale

Esiste ormai una convergenza su un unico linguaggio di modellazione di tipo grafico: UML (Unified Modeling Language).

Modelli di processo

Un processo di sviluppo è un insieme ordinato di passi che coinvolge tutte quelle attività, vincoli e risorse necessari per produrre il desiderato output a partire dall'insieme dei requisiti in ingresso. Tipicamente un processo è composto da differenti fasi messe in relazione una con l'altra.

Generiche fasi:

- Specifica: cosa il sistema dovrebbe fare e vincoli di sviluppo.
- Sviluppo: produzione del sistema software.
- Validazione: testare che il sistema sviluppato sia quello che il committente voleva.
- Evoluzione: cambiamenti nel prodotto in accordo a modifiche dei requisiti o incremento delle funzionalità del sistema.

In altre parole, un modello di processo definisce un template attorno al quale organizzare e dettagliare un vero processo di sviluppo. Prescrive le fasi attorno alle quale il processo dovrebbe essere organizzato e l'ordine di tali fasi, l'interazione e la coordinazione del lavoro tra le diverse fasi.

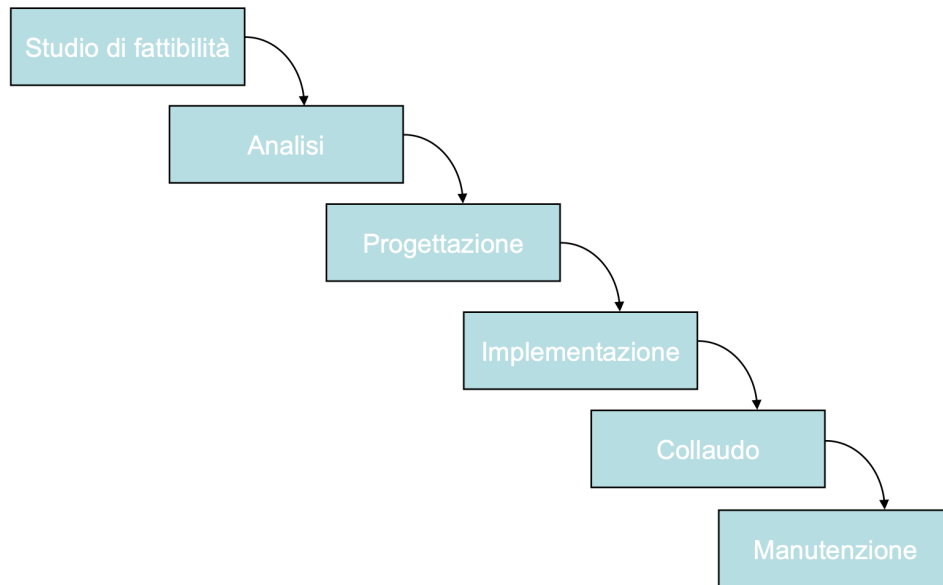
Modelli di processo possono essere di diverso tipo:

- Modello a cascata
- Modelli evolutivi
- Sviluppo incrementale–iterativo
- Modello a spirale
- Modelli specializzati
 - Sviluppo a componenti
 - Modello dei metodi formali
 - Sviluppo aspect-oriented
 - Sviluppo model driven
 - Unified Process (UP – RUP)

È possibile anche utilizzare per lo stesso progetto modelli ibridi, ossia sistemi composti di sotto-sistemi, per ognuno dei quali è possibile adottare un diverso modello di sviluppo.

Modello a cascata

Fasi distinte, in cascata tra loro.



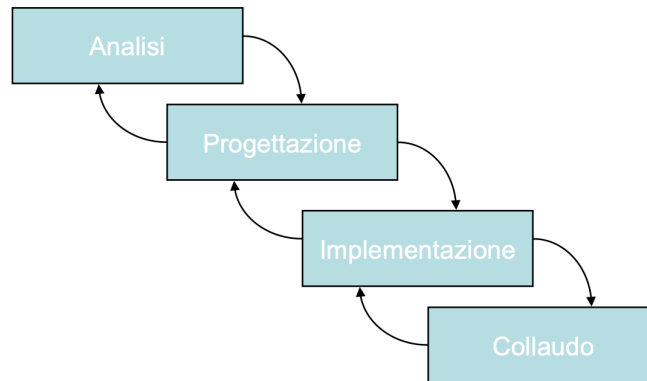
Il modello si fonda sul presupposto che introdurre cambiamenti sostanziali nel software in fasi avanzate dello sviluppo ha costi troppo elevati pertanto, ogni fase deve essere svolta in maniera esaustiva prima di passare alla successiva, in modo da non generare retroazioni.

Le uscite che una fase produce come ingresso per la fase successiva sono i cosiddetti semilavorati del processo di sviluppo:

- Documentazione di tipo cartaceo.
- Codice dei singoli moduli.
- Sistema complessivo.

I limiti di questo modello sono dati dal fatto che nella realtà la visione che i clienti hanno del sistema evolve man mano che il sistema prende forma e quindi le specifiche cambiano in continuazione. Nel campo della progettazione "le idee migliori vengono in mente per ultime", quando si comincia a vedere qualcosa di concreto. Inoltre, problemi di prestazioni costringono spesso a rivedere le scelte di progetto.

Evoluzioni successive al modello originale ammettono forme limitate di retroazione a un livello.



Prototipo

Per evitare problemi, prima di iniziare a lavorare sul sistema vero e proprio è meglio realizzare un prototipo in modo da fornire agli utenti una base concreta per meglio definire le specifiche. Una volta esaurito il compito, il prototipo viene abbandonato (throw-away prototyping) e si procede a costruire il sistema reale secondo i canoni del modello a cascata. Questo approccio risulta però quasi sempre così dispendioso da annullare i vantaggi economici che il modello a cascata dovrebbe garantire.

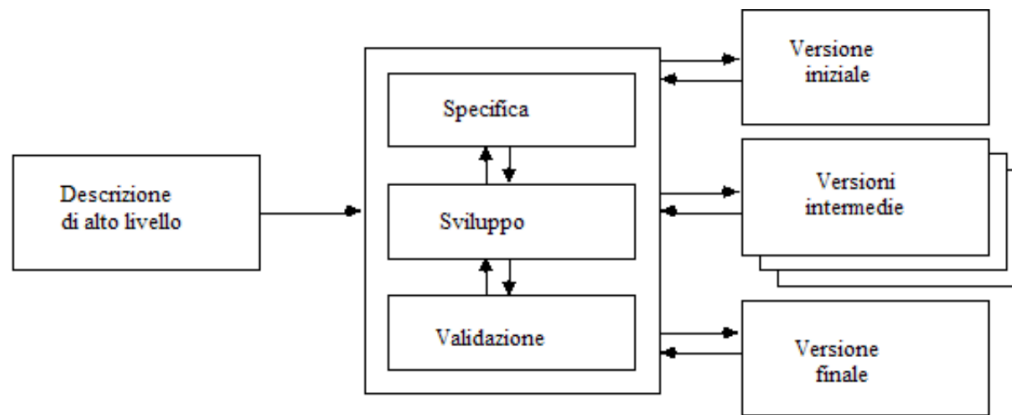
Un prototipo è un modello approssimato dell'applicazione che come obiettivo quello di essere mostrato al cliente al fine di ottenere un'indicazione su quanto il prototipo colga i reali fabbisogni. È una strategia interessante nel caso in cui lo sviluppo dell'applicazione parta inizialmente con requisiti non perfettamente noti o instabili, ma deve essere sviluppato in tempi brevi e con costi minimi.

Esistono diverse tipologie di prototipazioni:

- Prototipazione "throw-away": il prototipo è del tipo usa e getta con l'obiettivo di comprendere meglio le richieste del cliente e quindi migliorare la definizione dei requisiti del sistema. Il prototipo si concentra sulle parti che sono mal comprese con l'obiettivo di contribuire a chiarire i requisiti.
- Programmazione esplorativa: il prototipo si trasforma progressivamente nel prodotto finale. L'obiettivo è quello di lavorare a stretto contatto con il cliente per chiarire completamente i requisiti e giungere a un prodotto finale. Prima si sviluppano le parti del sistema che sono ben chiare, poi si aggiungono nuove parti/funzionalità come proposto dal cliente.

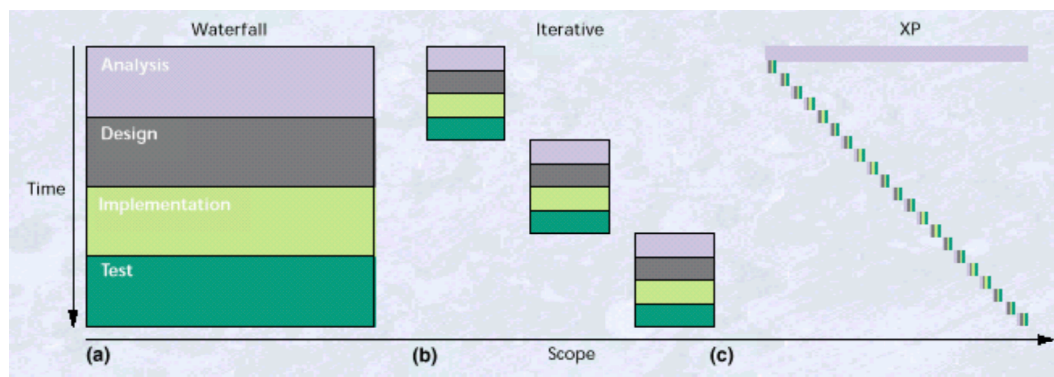
Modelli evolutivi

Partendo da specifiche molto astratte si sviluppa un primo prototipo da sottoporre al cliente e da raffinare successivamente.



Il vantaggio fondamentale è che ad ogni iterazione è possibile confrontarsi con gli utenti per quanto riguarda le specifiche e le funzionalità (raffinamento dell'analisi), e rivedere le scelte di progetto (raffinamento del design).

I modelli evolutivi si sono orientati verso cicli sempre più brevi e iterazioni sempre più veloci, fino ad arrivare al modello più radicale che prende il nome di Extreme Programming (XP). L'illustrazione mostra l'evoluzione dal modello a cascata all'extreme programming.



I problemi di questi tipi di modelli sono che il processo di sviluppo non è visibile (ad es., documentazione non disponibile) e che il sistema sviluppato è poco strutturato (modifiche frequenti). L'applicabilità è infatti rivolta a sistemi

di piccole dimensioni, sistemi che avranno breve durata o parti di sistemi più grandi.

Sviluppo incrementale-iterativo

Sviluppo incrementale: si costruisce il sistema sviluppandone sistematicamente e in sequenza parti ben definite. Una volta costruita una parte, essa non viene più modificata. È di fondamentale importanza essere in grado di specificare perfettamente i requisiti della parte da costruire prima della sua implementazione.

Sviluppo iterativo: si effettuano molti passi dell'intero ciclo di sviluppo del software, per costruire iterativamente tutto il sistema, aumentandone ogni volta il livello di dettaglio.

Sviluppo incrementale-iterativo: si individuano sottoparti relativamente autonome, si realizza il prototipo di una di esse e si continua con altre parti. Si aumenta progressivamente l'estensione e il dettaglio dei prototipi, tenendo conto delle altre parti interagenti.

Rational Unified Process

Rational Unified Process (RUP) è un modello di processo software iterativo sviluppato da Rational Software.

RUP è un modello ibrido, e contiene elementi di tutti i modelli di processo generici. Inoltre non definisce un singolo, specifico processo, bensì un framework adattabile che può dar luogo a diversi processi in diversi contesti.

RUP individua tre diverse visioni del processo di sviluppo:

- Prospettiva dinamica: mostra le fasi del modello nel tempo.
- Prospettiva statica: mostra le attività del processo coinvolte.
- Prospettiva pratica: suggerisce le buone prassi da seguire durante il processo.

Prospettiva dinamica

- Inception (Avvio): generalizzazione dell'analisi di fattibilità.

Lo scopo principale è quello di delineare nel modo più accurato possibile il business case, ovvero comprendere il tipo di mercato al quale il progetto

afferisce e identificare gli elementi importanti affinché esso conduca a un successo commerciale, identificare tutte le entità esterne (persone e sistemi) che interagiranno con il sistema e definire tali interazioni.

Fra gli strumenti utilizzati ci sono un modello dei casi d'uso, la pianificazione iniziale del progetto, la valutazione dei rischi, una definizione grossolana dei requisiti e così via.

- **Elaboration (Elaborazione):** definisce la struttura complessiva del sistema.

Comprende l'analisi di dominio e una prima fase di progettazione dell'architettura. Al termine di questa fase bisogna avere un modello dei casi d'uso completo all'80%, una descrizione dell'architettura del sistema, lo sviluppo di un'architettura eseguibile che dimostri il completamento degli use case significativi, la revisione del business case e dei rischi e la pianificazione del progetto complessivo.

Al termine si transita in una situazione di rischio più elevato, in cui le modifiche del progetto saranno più difficili e dannose.

- **Construction (Costruzione):** progettare, programmare e testare il sistema.

Le diverse parti del sistema vengono sviluppate parallelamente e poi integrate. Al termine della fase si dovrebbe avere un sistema software funzionante e la relativa documentazione pronta.

- **Transition (Transizione):** il sistema passa dall'ambiente dello sviluppo a quello del cliente finale.

Vengono condotte le attività di training degli utenti e il beta testing del sistema a scopo di verifica e validazione. Si deve in particolare verificare che il prodotto sia conforme alle aspettative descritte nella fase di Inception, se questo non è vero si procede a ripetere l'intero ciclo.

Ogni fase può essere eseguita in modo ciclico, e l'intero insieme delle fasi può essere eseguito in modo incrementale.

Prospettiva statica

La prospettiva statica di RUP si concentra sulle attività di produzione del software chiamate workflow. RUP identifica sei workflow principali e tre di supporto. Il vantaggio di presentare sia la visione statica che quella dinamica è

che le fasi dello sviluppo non sono associati a specifici workflow, ma tutti i workflow possono essere attivi in ogni stadio del processo.

I workflow sono:

- Modellazione delle attività aziendali: i processi aziendali sono modellati utilizzando il business case.
- Requisiti: vengono identificati gli attori che interagiscono con il sistema e sviluppati i casi d'uso per modellare i requisiti.
- Analisi e Progetto: viene creato e documentato un modello di progetto utilizzando modelli architetturali, dei componenti, degli oggetti e sequenziali.
- Implementazione: i componenti del sistema sono implementati e strutturati nell'implementazione dei sottosistemi. La generazione automatica del codice a partire dai modelli velocizza questa fase.
- Test: il test dei sottosistemi è un processo iterativo eseguito in parallelo all'implementazione.
- Rilascio: viene creata una release del prodotto, viene distribuita agli utenti e installata nelle loro postazioni di lavoro.
- Gestione della configurazione e delle modifiche: workflow di supporto che gestisce i cambiamenti nel sistema.
- Gestione del progetto: workflow di supporto che gestisce lo sviluppo del sistema.
- Ambiente: workflow di supporto che rende disponibili gli strumenti adeguati al team di sviluppatori.

Prospettiva pratica

La prospettiva pratica di RUP descrive la buona prassi di ingegneria del software che si raccomanda di usare nello sviluppo dei sistemi. Le pratiche fondamentali sono sei:

1. Sviluppare il software ciclicamente

Pianificare gli incrementi del sistema basati sulle proprietà del cliente.
Sviluppare e consegnare le funzioni con la priorità più alta all'inizio del

processo di sviluppo.

2. Gestire i requisiti

Documentare esplicitamente i requisiti del cliente e i cambiamenti effettuati. Analizzare l'impatto dei cambiamenti sul sistema prima di accettarli.

3. Usare architetture basate sui componenti

Strutturare l'architettura del sistema con un approccio a componenti.

4. Creare modelli visivi del software

Usare modelli grafici UML per rappresentare le visioni statiche e dinamiche del software.

5. Verificare la qualità del software

Assicurarsi che il software raggiunga gli standard di qualità dell'organizzazione.

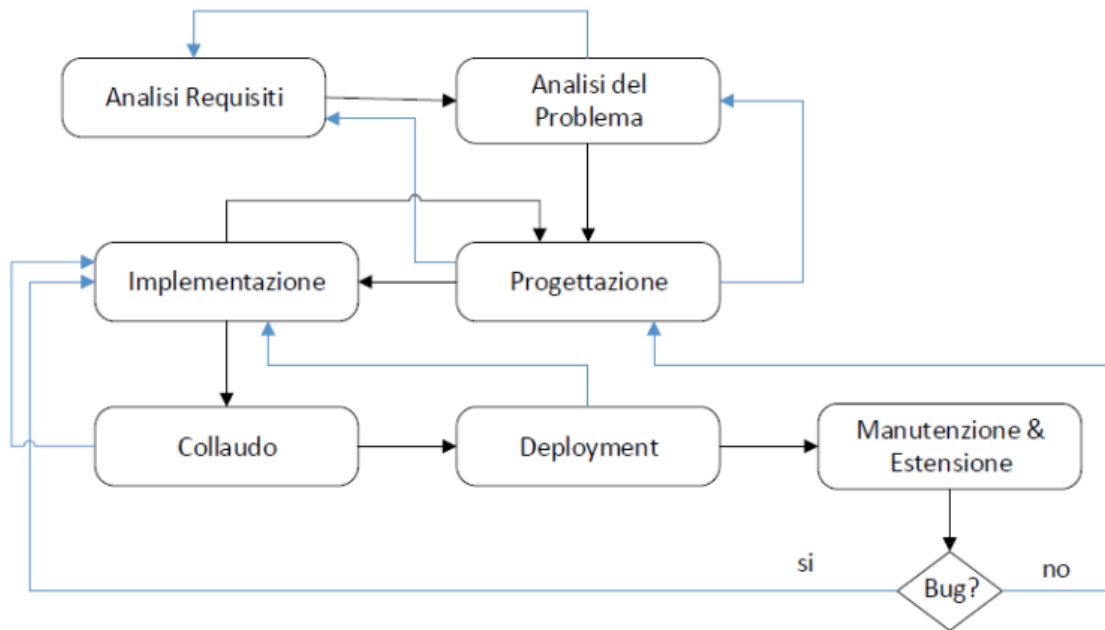
6. Controllare le modifiche al software

Gestire i cambiamenti del software usando un sistema per la gestione delle modifiche, procedure e strumenti di gestione della configurazione.

Il nostro processo di sviluppo

La definizione di un processo non può prescindere dalle caratteristiche dell'organizzazione in cui esso avviene e dalla competenza ed esperienza degli attori coinvolti. Differenti tipi di sistemi inoltre richiedono differenti tipi di processi di sviluppo. L'uso di un processo inappropriato riduce la qualità e l'utilità del prodotto software che deve essere sviluppato o migliorato.

Noi seguiremo un processo di sviluppo simil-RUP adottando UML come linguaggio di modellazione del software.



▼ 2.0 - Analisi dei requisiti

I requisiti di un sistema rappresentano la descrizione dei servizi forniti e dei vincoli operativi.

Tipologie di requisiti

I requisiti di solito vengono forniti a livelli diversi di descrizione e questo porta a una prima classificazione:

- Requisiti utente.

Dichiarano quali servizi il sistema dovrebbe fornire, i vincoli sotto cui deve operare e sono requisiti molto astratti e di alto livello che vengono specificati nella prima fase interlocutoria con il committente. Tipicamente sono espressi in linguaggio naturale e corredati da qualche diagramma.

- Requisiti di sistema.

Definiscono le funzioni, i servizi e i vincoli operativi del sistema in modo dettagliato. È una descrizione dettagliata di quello che il sistema dovrebbe fare e viene inserita all'interno del documento dei requisiti del sistema.

I requisiti di sistema vengono tipicamente divisi in tre diverse tipologie:

- Requisiti funzionali

Elenco di tutti i servizi che il sistema dovrebbe fornire. Per ogni servizio dovrebbe essere indicato come reagire a particolari input, come comportarsi in particolari situazioni e in alcuni casi specificare cosa il sistema non dovrebbe fare.

- Requisiti non funzionali

Non riguardano direttamente le specifiche funzionali del prodotto. Possono essere di due tipologie:

- Requisiti del prodotto: specificano o limitano le proprietà complessive del sistema.

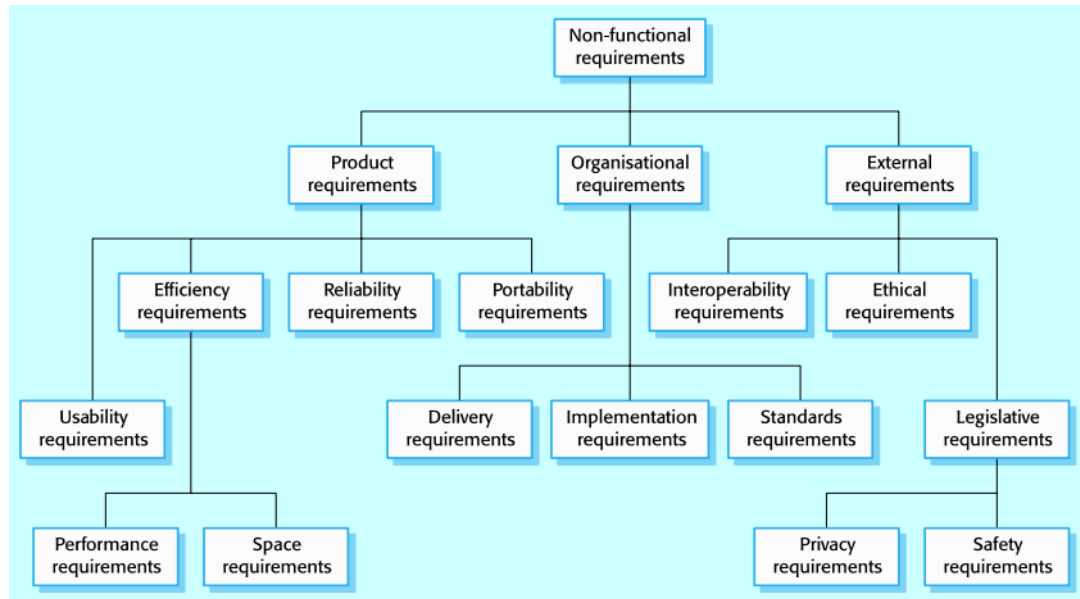
Esempi: affidabilità, prestazioni, protezione dei dati, disponibilità dei servizi, tempi di risposta, occupazione di spazio, capacità dei dispositivi di I/O, rappresentazione dei dati nelle interfacce, etc.

- Requisiti organizzativi: vincolano il processo di sviluppo adottato.

Esempi: politiche e procedure dell'organizzazione cliente e sviluppatrice, specifiche degli standard di qualità da adottare, uso di un particolare CASE tool e linguaggi di implementazione, limiti di budget, requisiti di consegna e milestone, etc.

- Requisiti esterni: tutti i vincoli che derivano da fattori non provenienti dal sistema e dal suo processo di sviluppo.

Esempi: necessità di interoperabilità con altri sistemi software o hardware prodotti da altre organizzazioni, requisiti legislativi sulla privacy dei dati, requisiti etici, etc.



- Requisiti di dominio

Derivano dal dominio di applicazione del sistema. Solitamente includono una terminologia propria del dominio del sistema o si riferiscono ai suoi concetti.

Analisi dei requisiti

L'obiettivo dell'analisi dei requisiti è quello di specificare le proprietà che il sistema dovrà avere senza descrivere una loro possibile realizzazione.

L'analisi dei requisiti si suddivide in 4 fasi:

1. Raccolta dei requisiti

Obiettivo: raccogliere tutte le informazioni su cosa il sistema deve fare secondo le intenzioni del cliente.

2. Analisi dei requisiti

Obiettivo: definire il comportamento del sistema da realizzare.

3. Analisi del dominio

Obiettivo: definire la porzione del mondo reale rilevante per il sistema.

4. Analisi e gestione dei rischi

Obiettivo: identificare e gestire i possibili rischi che possono fare fallire o intralciare la realizzazione del sistema.

Raccolta dei requisiti

Obiettivo: raccogliere tutte le informazioni su cosa il sistema deve fare secondo le intenzioni del cliente.

Non prevede passi formali, né ha una notazione specifica, perché dipende moltissimo dal particolare tipo di problema.

I risultati di questa fase sono:

- Un documento testuale scritto dall'analista, discusso e approvato dal cliente.
- Una versione iniziale del vocabolario o glossario contenente la definizione precisa e non ambigua di tutti i termini e i concetti utilizzati.

I metodi utilizzati per raggiungere l'obiettivo possono essere interviste, questionari, studio di documenti che esprimono i requisiti in forma testuale, osservazione passiva o attiva del processo da modellare, studio di sistemi software esistenti, prototipi.

Il Documento dei Requisiti risultante da questa fase deve specificare in modo chiaro e univoco cosa il sistema dovrà fare. Un buon modo per organizzare i requisiti e facilitare la tracciabilità è quello di elencare i requisiti in una tabella.

Tabella dei requisiti

Id Requisito	Requisito	Tipo
Numero identificativo univoco	Descrizione del requisito	Funzionale/non funzionale/dominio

In questo modo sarà possibile poi riferirsi ad uno specifico requisito durante le fasi successive.

Analisi dei requisiti

Obiettivo: definire il comportamento del sistema da realizzare.

Il risultato di questa fase è un modello comportamentale che descrive in modo chiaro e conciso le funzionalità del sistema.

La strategia da adottare è quella, partendo dalla tabella dei requisiti, della scomposizione funzionale tramite analisi top-down, la quale consente di identificare le singole funzionalità previste dal sistema.

Analisi del dominio

Obiettivo: definire la porzione del mondo reale rilevante per il sistema.

Permette di gestire la complessità intrinseca del mondo reale, concentrandosi maggiormente sugli aspetti che sono importanti per il progetto.

Il risultato di questa fase è la prima versione del vocabolario partendo dai sostantivi che si trovano nei requisiti.

Analisi e gestione dei rischi

Analisi sistematica e completa di tutti i possibili rischi che possono fare fallire o intralciare la realizzazione del sistema in una qualsiasi fase del processo di sviluppo. Ogni rischio presenta due caratteristiche: probabilità che avvenga e costo.

Esempi di rischi sono:

- Rischi relativi ai requisiti (I requisiti sono perfettamente noti e possono essere rispettati?).
- Rischi relativi alle risorse umane (È possibile contare sulle persone e sull'esperienza necessarie per lo sviluppo del progetto?).
- Rischi relativi alla protezione e privacy dei dati (Di che tipo sono i dati che dobbiamo trattare? Quali sono i possibili attacchi informatici a cui il sistema può essere soggetto?).
- Rischi tecnologici (Si sta scegliendo la tecnologia corretta? Si è in grado di aggregare correttamente i vari componenti del progetto (ad es., GUI, DB, ...)? Quali saranno i possibili futuri sviluppi della tecnologia?).
- Rischi politici (Ci sono delle forze politiche in grado di intralciare lo sviluppo del progetto?).

A seguito dell'analisi dei rischi si predispone un piano che permetta di reagire in modo controllato ed efficace sui rischi che hanno una maggiore probabilità di avvenire e un costo elevato.

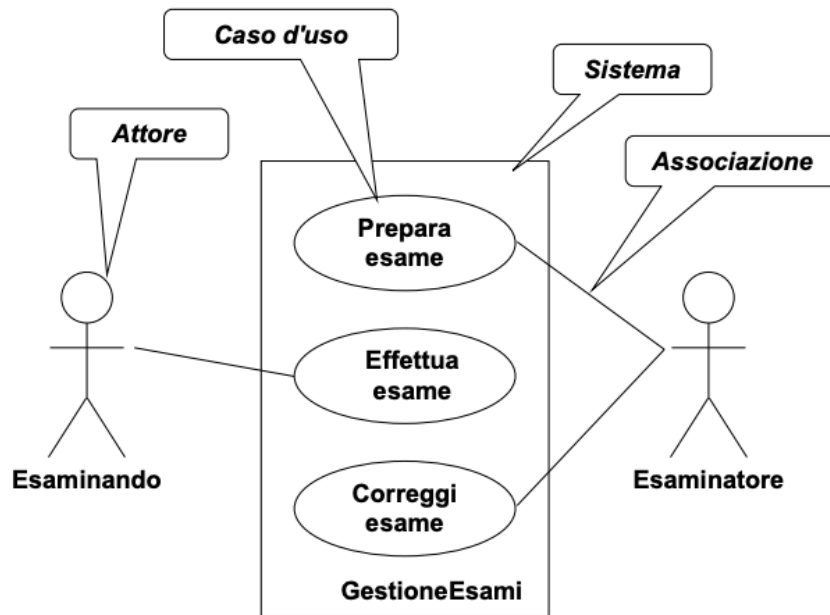
Casi d'uso e scenari

I casi d'uso e i relativi scenari permettono di formalizzare i requisiti funzionali, di comprendere meglio il funzionamento del sistema e di comunicare meglio con il cliente.

Costruzione di un caso d'uso

I passaggi per costruirli sono:

1. Individuare il confine del sistema.
2. Individuare gli attori.
L'attore è il ruolo interpretato da un utente (persona o sistema esterno) nei confronti del sistema.
3. Individuare i casi d'uso.
Ogni caso d'uso modella un servizio richiesto al sistema da un attore o da un altro caso d'uso.
4. Disegnare i diagrammi dei casi d'uso.
Ogni diagramma modella le associazioni tra attori e casi d'uso e tra i vari casi d'uso.
5. Descrivere i dettagli di ogni singolo caso d'uso mediante scenari.
Ogni scenario descrive sia l'interazione tra l'attore e il sistema che le elaborazioni necessarie per soddisfare la richiesta dell'attore.
6. Ricontrollare e validare i casi d'uso insieme al cliente.



Lo scenario principale del caso d'uso Effettua esame in figura è il seguente:

1. L'esaminando entra nel sistema (login).
2. L'esaminando inizia l'esame.
3. L'esaminando naviga tra le domande e risponde.
4. L'esaminando termina l'esame.
5. L'esaminando esce dal sistema (logout).

Struttura di un caso d'uso

Un caso d'uso viene sempre descritto dal punto di vista di un attore e comprende:

- 0+ precondizioni, ovvero condizioni che devono essere tutte verificate prima che il caso d'uso possa essere eseguito.
- 1+ scenari, ovvero sequenze di passi che descrivono le interazioni tra l'attore e il sistema necessarie a raggiungere l'obiettivo richiesto. Potrebbero comprendere ramificazioni (if) e iterazioni (for, foreach e while). Ogni sequenza di passi deve essere scritta in una forma narrativa strutturata che utilizza il vocabolario di dominio.
 - 1 scenario principale.

- 0+ scenari alternativi, i quali rappresentano possibili varianti anomale del flusso scatenate da opzioni, condizioni d'errore, violazione della sicurezza, etc. (es. come deve comportarsi il sistema se l'esaminando non è riuscito ad autenticarsi)
- 0+ postcondizioni, ovvero condizioni che devono essere tutte vere quando il caso d'uso termina l'esecuzione con successo.

Relazioni tra casi d'uso

- Generalizzazione/Specializzazione

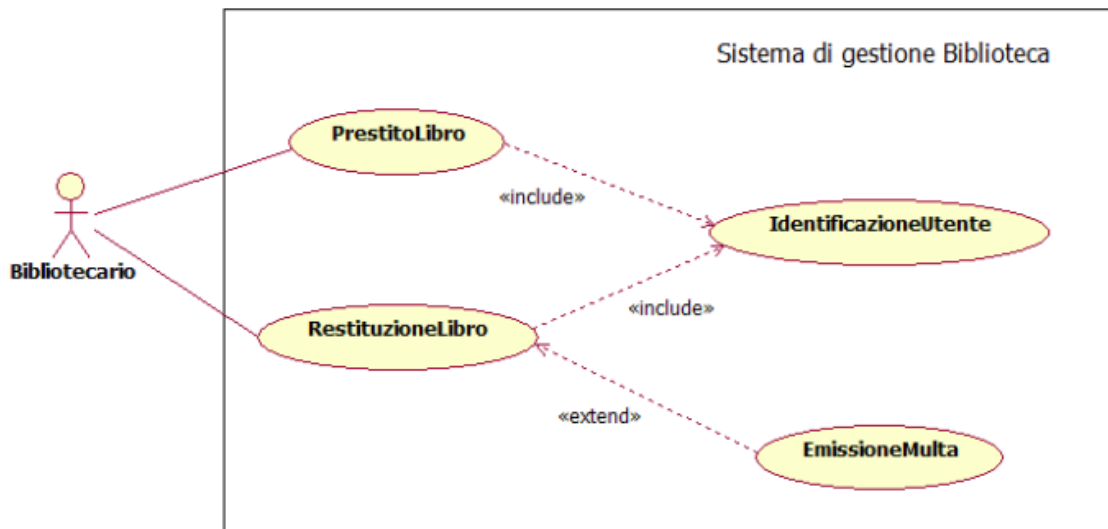
Si utilizza quando un caso d'uso è simile ad un altro, ma fa qualcosa di più. È applicabile anche agli attori: un attore può essere la specializzazione di un altro attore.

- Inclusione

Si utilizza quando un caso d'uso usa almeno una volta un altro caso d'uso.

- Estensione

Si utilizza quando è necessario aggiungere un comportamento opzionale a un caso d'uso esistente.



▼ 3.0 - Analisi dei requisiti: Sicurezza e Privacy

Nuova normativa

General Data Protection Regulation

Obbligo di aderenza di un prodotto software che tratti dati personali ai principi della GDPR. Tratta i seguenti argomenti:

- privacy by design & by default
- minimalità, proporzionalità
- anonimizzazione, pseudonimizzazione

Pseudonimizzazione: processo di trattamento dei dati personali in modo tale che i dati non possano più essere attribuiti a un interessato specifico senza l'utilizzo di informazioni aggiuntive. Occorre adottare misure tecniche e organizzative che separino in dati in modo da garantire la non attribuzione a una persona identificata o identificabile.

- trasferimento dati fuori dalla EU
- adeguatezza delle misure di sicurezza

Principi

I dati personali devono essere:

- trattati in modo lecito, equo e trasparente nei confronti dell'interessato.
- raccolti per finalità determinate, esplicite e legittime, e successivamente trattati in modo non incompatibile con tali finalità.
- adeguati, pertinenti e limitati a quanto necessario rispetto alle finalità per le quali sono trattati ("minimizzazione dei dati").
- esatti e, se necessario, aggiornati.
- conservati in una forma che consenta l'identificazione degli interessati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati.
- trattati in modo da garantire un'adeguata sicurezza dei dati personali, compresa la protezione, mediante misure tecniche e organizzative adeguate, da trattamenti non autorizzati o illeciti e dalla perdita, dalla distruzione o dal danno accidentali ("integrità e riservatezza").

Articolo 25

Protezione dei dati fin dalla progettazione e protezione di default.

1. Sia al momento di determinare i mezzi del trattamento sia all'atto del trattamento stesso il responsabile del trattamento mette in atto misure tecniche e organizzative adeguate, quali la pseudonimizzazione, volte ad attuare i principi di protezione dei dati, quali la minimizzazione, in modo efficace e a integrare nel trattamento le necessarie garanzie al fine di soddisfare i requisiti del presente regolamento e tutelare i diritti degli interessati.
2. Il responsabile del trattamento mette in atto misure tecniche e organizzative adeguate per garantire che siano trattati, di default (senza l'intervento della persona fisica), solo i dati personali necessari per ogni specifica finalità del trattamento; ciò vale per la quantità dei dati raccolti, l'estensione del trattamento, il periodo di conservazione e l'accessibilità.

Articolo 32

Sicurezza del trattamento.

1. Il responsabile del trattamento e l'incaricato del trattamento mettono in atto misure tecniche e organizzative adeguate per garantire un livello di sicurezza adeguato al rischio, che comprendono tra l'altro:
 - a. la pseudonimizzazione e la cifratura dei dati personali.
 - b. la capacità di assicurare la continua riservatezza, integrità, disponibilità e resilienza dei sistemi e dei servizi che trattano i dati personali.
 - c. la capacità di ripristinare tempestivamente la disponibilità e l'accesso dei dati in caso di incidente fisico o tecnico.
 - d. una procedura per provare, verificare e valutare regolarmente l'efficacia delle misure tecniche e organizzative al fine di garantire la sicurezza del trattamento.
2. Nel valutare l'adeguato livello di sicurezza, si tiene conto in special modo dei rischi presentati da trattamenti di dati derivanti in particolare dalla distruzione, dalla perdita, dalla modifica, dalla divulgazione non autorizzata o dall'accesso, in modo accidentale o illegale, a dati personali trasmessi, memorizzati o comunque trattati.

3. L'adesione a un codice di condotta approvato di cui all'articolo 40 o a un meccanismo di certificazione approvato di cui all'articolo 42 può essere utilizzata come elemento per dimostrare la conformità ai requisiti di cui al paragrafo 1 del presente articolo.

Articolo 45

Il trasferimento di dati personali verso un paese terzo o un'organizzazione internazionale è ammesso se la Commissione ha deciso che il paese terzo, o un territorio o uno o più settori specifici all'interno del paese terzo, o l'organizzazione internazionale in questione garantiscano un livello di protezione adeguato. In tal caso il trasferimento non necessita di autorizzazioni specifiche.

Articolo 46

In mancanza di una decisione ai sensi dell'articolo 45, il responsabile del trattamento o l'incaricato del trattamento può trasferire dati personali verso un paese terzo o un'organizzazione internazionale solo se ha offerto garanzie adeguate e a condizione che siano disponibili diritti azionabili degli interessati e mezzi di ricorso effettivi per gli interessati.

Concetti base

Sicurezza informatica

Salvaguardia dei sistemi informatici da potenziali rischi e/o violazioni dei dati che hanno come obiettivo quello di ricavare del contenuto informativo.

La sicurezza informatica si preoccupa di impedire l'accesso ad utenti non autorizzati, di regolamentare l'accesso ai diversi soggetti con autorizzazioni diverse ed evitare che i dati appartenenti al sistema informatico vengano copiati, modificati o cancellati.

Metodi di protezione

Si possono utilizzare metodi di protezione di diverso tipo:

- Legali
- Organizzativi
- Tecnologici

- Fisici
- Crittografici
- Biometrici

Questi metodi possono essere utilizzati per uno o più dei seguenti scopi:

- Prevenzione
- Rilevazione
- Contenimento
- Ripristino

Autenticazione forte

Per autenticazione forte si intende un'autenticazione che richiede una somma di tre fattori:

- Qualcosa che sei (es. impronta digitale)
- Qualcosa che hai (es. carta magnetica)
- Qualcosa che sai (es. password)

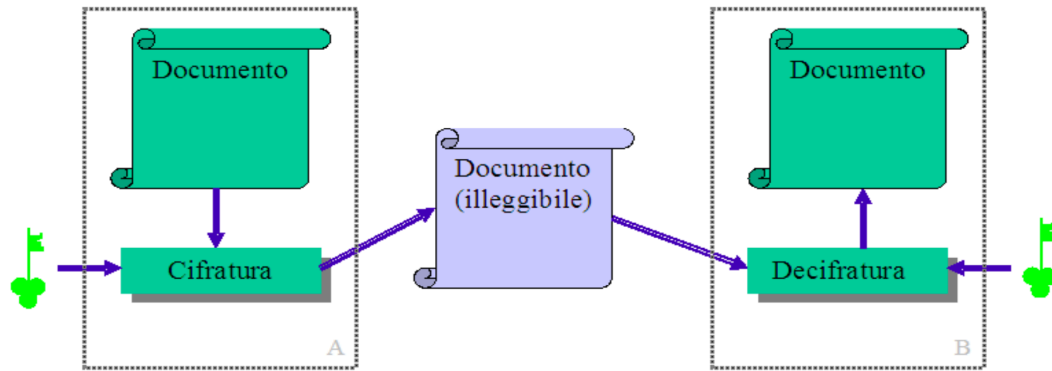
L'autenticazione forte presenta però diversi problemi come quello di utilizzare troppe operazioni di confronto che influenzano la velocità di accesso, e anche il grande numero di falsi negativi per via del fatto che c'è una maggiore possibilità di errore in 3 confronti.

Crittografia

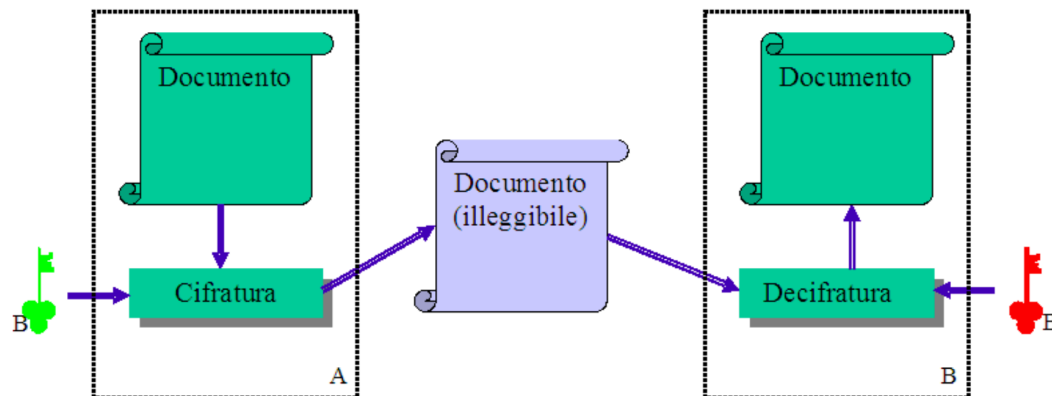
Diversi tipi di crittografia:

- Crittografia sincrona: non identifica nè autentica.

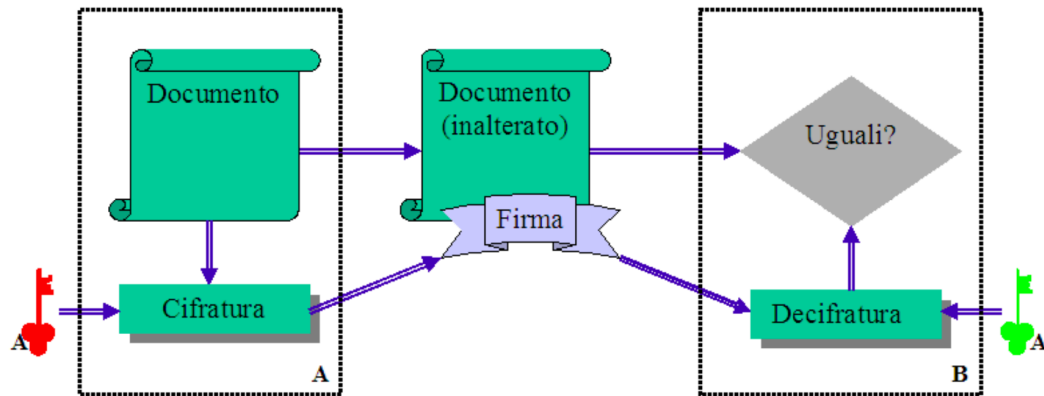
Viene utilizzata una singola chiave cifra e decifra, mentre è implementata con dispositivi segreti, algoritmi segreti o chiavi segrete.



- Crittografia asincrona: identificazione e quindi autenticazione e paternità.
È basata sulla teoria della complessità computazionale. Vengono utilizzate due chiavi (pubblica e privata) correlate ma non facilmente calcolabili l'una con l'altra. Chiunque può usare la chiave pubblica del destinatario per crittografare un messaggio, ma solo chi possiede la chiave privata può decifrare il messaggio.



La firma digitale è una crittografia asincrona che funziona al contrario: la chiave privata viene utilizzata per firmare il documento, mentre quella pubblica per verificarne la firma.



Entrambe hanno lo scopo di garantire riservatezza.

Esistono infrastrutture per la certificazione della chiave pubblica, ossia terze parti fidate che possano certificare l'autenticità di una chiave pubblica.

Sistemi critici

I sistemi critici sono sistemi tecnici o socio-tecnici da cui dipendono persone o aziende. Se questi sistemi non forniscono i loro servizi come ci si aspetta possono verificarsi seri problemi e importanti perdite. Ci sono tre tipi principali di sistemi critici:

- Sistemi safety-critical: i fallimenti possono provocare incidenti, perdita di vite umane o seri danni ambientali.
- Sistemi mission-critical: i malfunzionamenti possono causare il fallimento di alcune attività e obiettivi diretti.
- Sistemi business-critical: i fallimenti possono portare a costi molto alti per le aziende che li usano.

La proprietà più importante di un sistema critico è la sua fidatezza. Sistema fidato = disponibilità + affidabilità + sicurezza e protezione.

Attacchi informatici

La sicurezza e la protezione dei sistemi critici sono diventate sempre più importanti con l'aumentare delle connessioni di rete. Da una parte le connessioni di rete espongono il sistema ad attacchi da parte di malintenzionati, dall'altra parte la rete fa in modo che i dettagli delle vulnerabilità siano facilmente divulgati e facilita la distribuzione di patch.

Esempi di attacchi sono:

- **Exploit:** metodo che sfrutta un bug o una vulnerabilità, per l'acquisizione di privilegi.
- **Buffer Overflow:** fornire al programma più dati di quanto esso si aspetti di ricevere, in modo che una parte di questi vadano scritti in zone di memoria dove sono, o dovrebbero essere, altri dati o lo stack del programma stesso.
- **Shell code:** sequenza di caratteri che rappresenta un codice binario in grado di lanciare una shell, può essere utilizzato per acquisire un accesso alla linea di comando.
- **Sniffing:** attività di intercettazione passiva dei dati che transitano in una rete.
- **Cracking:** modifica di un software per rimuovere la protezione dalla copia, oppure per ottenere accesso a un'area riservata.
- **Spoofing:** tecnica con la quale si simula un indirizzo IP privato da una rete pubblica facendo credere agli host che l'IP della macchina server da contattare sia il suo.
- **Trojan:** programma che contiene funzionalità maliziose, la vittima è indotta a eseguire il programma poiché questo viene spesso inserito nei videogiochi pirati.
- **Denial of Service:** il sistema viene forzatamente messo in uno stato in cui i suoi servizi non sono disponibili, influenzando così la disponibilità del sistema.

Introduzione alla Security Engineering

Le vigenti leggi, le politiche e i vincoli aziendali sono la base di partenza per la definizione di un piano per la sicurezza, che deve essere progettato e attuato sin dalle prime fasi.

La gestione della sicurezza comprende:

- Gestione degli utenti e dei permessi:
 - inserimento e rimozione di utenti dal sistema

- autenticazione degli utenti
- creazione di appropriati permessi per gli utenti
- Deployment e mantenimento del sistema:
 - installazione e configurazione dei software e del middleware
 - aggiornamento periodico del software con tutte le patch disponibili
- Controllo degli attacchi, rilevazione e ripristino
 - controllo del sistema per accessi non autorizzati
 - identificazione e messa in opera di strategie contro gli attacchi
 - backup per ripristinare il normale utilizzo dopo un attacco

Glossario di sicurezza

- Bene (Asset): una risorsa del sistema che deve essere protetta.
- Esposizione (Exposure): possibile perdita o danneggiamento come risultato di un attacco andato a buon fine.
- Vulnerabilità (Vulnerability): una debolezza nel sistema software che potrebbe essere sfruttata per causare una perdita o un danno.
- Attacco (Attack): sfruttamento di una vulnerabilità del sistema.
- Minaccia (Threat): circostanza che ha le potenzialità per causare perdite e danni.
- Controllo (Control): una misura protettiva che riduce una vulnerabilità del sistema.

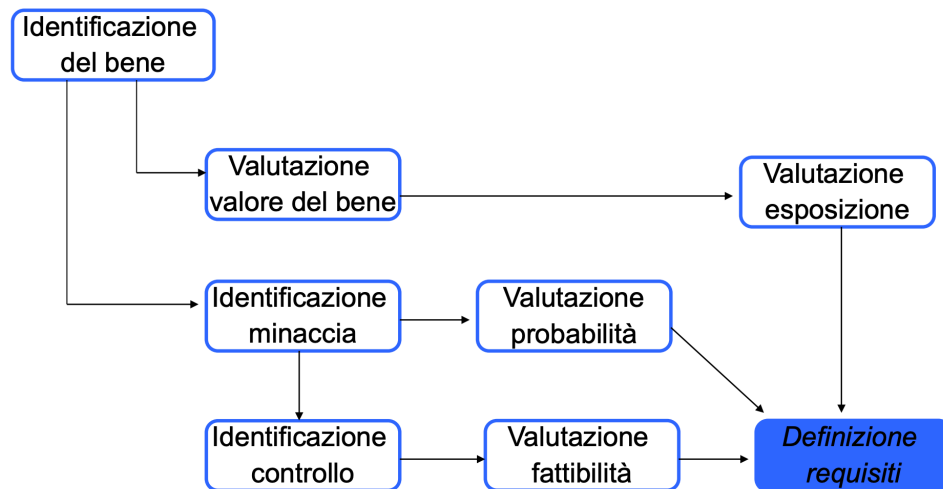
Analisi del rischio

L'analisi del rischio si occupa di valutare le possibili perdite che un attacco può causare ai beni di un sistema. Bilanciare queste perdite con i costi richiesti per la protezione dei beni stessi. Il costo della protezione deve essere molto minore del costo della perdita.

L'analisi del rischio inizia dalla valutazione delle politiche di sicurezza organizzazionali che spiegano cosa dovrebbe e cosa non dovrebbe essere consentito fare. Le politiche di sicurezza propongono le condizioni che

dovrebbero sempre essere mantenute dal sistema di sicurezza, quindi aiutano ad identificare le minacce che potrebbero sorgere.

La valutazione del rischio si struttura nel seguente modo:



Identificazione del bene

Fasi dell'identificazione del bene:

- Analisi del sistema informativo

Durante questa fase si può stabilire la seguente agenda delle attività:

- Analisi delle risorse fisiche
- Analisi delle risorse logiche
- Analisi delle dipendenze fra risorse

- Analisi delle risorse fisiche

In questa attività, il sistema informatico viene visto come insiemi di dispositivi che, per funzionare, hanno bisogno di spazio, alimentazione, condizioni ambientali adeguate, protezioni da furti e danni materiali.

In particolare occorre individuare sistematicamente tutte le risorse fisiche, ispezionare e valutare tutti i locali che ospiteranno le risorse fisiche e verificare la cablatura dei locali.

- Analisi delle risorse logiche

Il sistema viene visto come insieme di informazioni, flussi e processi. In particolare occorre:

- Classificare le informazioni in base al valore che hanno per l'organizzazione, il grado di riservatezza e il contesto di appartenenza.
- Classificare i servizi offerti dal sistema informatico affinché non presentino effetti collaterali pericolosi per la sicurezza del sistema.
- Analisi delle dipendenze tra risorse
Per ciascuna risorsa (fisica o logica) occorre individuare di quali altre risorse essa ha bisogno per funzionare correttamente. Questa analisi tende ad evidenziare le risorse potenzialmente critiche, ovvero quelle da cui dipende il funzionamento di un elevato numero di altre risorse.

Identificazione delle minacce

In questa fase si cerca di definire quello che non deve poter accadere nel sistema. Si parte dal considerare come evento indesiderato qualsiasi accesso che non sia esplicitamente permesso. A tal fine è possibile in generale distinguere tra:

- Attacchi intenzionali

Gli attacchi vengono caratterizzati in funzione della risorsa che viene attaccata e delle possibili tecniche usate per l'attacco. Si distingue tra tecniche a livello fisico e a livello logico.

Gli attacchi a livello fisico sono principalmente tesi a sottrarre o danneggiare le risorse critiche (furto o danneggiamento).

Gli attacchi a livello logico sono principalmente tesi a sottrarre informazione o degradare l'operatività del sistema. Dal punto di vista dei risultati che è indirizzato a conseguire un attacco logico può essere classificato come:

- Intercettazione e deduzione (attacco alla riservatezza).
- Intrusione (attacco all'integrità e alla riservatezza).
- Disturbo (attacco alla disponibilità).
- Eventi accidentali

Una possibile casistica degli eventi accidentali che accadono più di frequente:

- Livello fisico:
 - guasti ai dispositivi che compongono il sistema
 - guasti di dispositivi di supporto (es. condizionatori)
- Livello logico:
 - perdita di password o chiave hardware
 - cancellazione di file
 - corruzione del software di sistema

Valutazione dell'esposizione

A ogni minaccia occorre associare un rischio così da indirizzare l'attività di individuazione delle contromisure verso le aree più critiche. Per rischio s'intende una combinazione della probabilità che un evento accada con il danno che l'evento può arrecare al sistema.

La probabilità di occorrenza di attacchi intenzionali dipende principalmente dalla facilità di attuazione e dai vantaggi che potrebbe trarne l'intruso.

Il danno si misura come grado di perdita dei tre requisiti fondamentali (riservatezza, integrità, disponibilità). Nel valutare il danno si tiene conto delle dipendenze tra le risorse e dell'eventuale propagazione del malfunzionamento.

Individuazione del controllo

Occorre scegliere il controllo da adottare per neutralizzare gli attacchi individuati. Per farlo occorre valutare il rapporto costo/efficacia, il quale mira ad evitare che i controlli presentino un costo ingiustificato rispetto all'efficacia che forniscono.

Le principali voci di costo sono le seguenti:

- Costo di messa in opera del controllo
- Peggioramento dell'ergonomia dell'interfaccia utente
- Decadimento delle prestazioni del sistema nell'erogazione dei servizi
- Aumento della burocrazia

Controlli di carattere organizzativo

Condizione essenziale affinché la tecnologia a protezione del sistema informatico risulti efficace e venga utilizzata nel modo corretto da personale pienamente consapevole.

Devono quindi essere definiti con precisione ruoli e responsabilità nella gestione sicura di tale sistema. Per ciascun ruolo, dall'amministratore al semplice utente, devono essere definite norme comportamentali e procedure precise da rispettare.

Controlli di carattere tecnico

- Etichettatura delle informazioni, allo scopo di avere un controllo più fine dei diritti di accesso.
- Moduli software di cifratura integrati con le applicazioni.
- Apparecchiature di telecomunicazione in grado di cifrare il traffico dati in modo trasparente alle applicazioni.
- Firewall e server proxy in corrispondenza di eventuali collegamenti con reti TCP/IP.
- Chiavi hardware e/o dispositivi di riconoscimento degli utenti basati su rilevamenti biofisici.

Integrazione dei controlli

Un insieme di controlli non deve presentarsi come una collezione di espedienti non correlati tra loro. È importante integrare i vari controlli in una politica di sicurezza organica. Occorre operare una selezione dei controlli adottando un sottoinsieme di costo minimo che rispetti alcuni vincoli:

- completezza delle contromisure: il sottoinsieme deve fare fronte a tutti gli eventi indesiderati.
- omogeneità delle contromisure: le contromisure devono essere compatibili e integrabili tra loro.
- ridondanza controllata delle contromisure: la ridondanza delle contromisure ha un costo e deve essere rilevata e vagliata accuratamente.

- effettiva attuabilità delle contromisure: l'insieme delle contromisure deve rispettare tutti i vincoli imposti dall'organizzazione nella quale andrà ad operare.

Ciclo di vita della valutazione del rischio

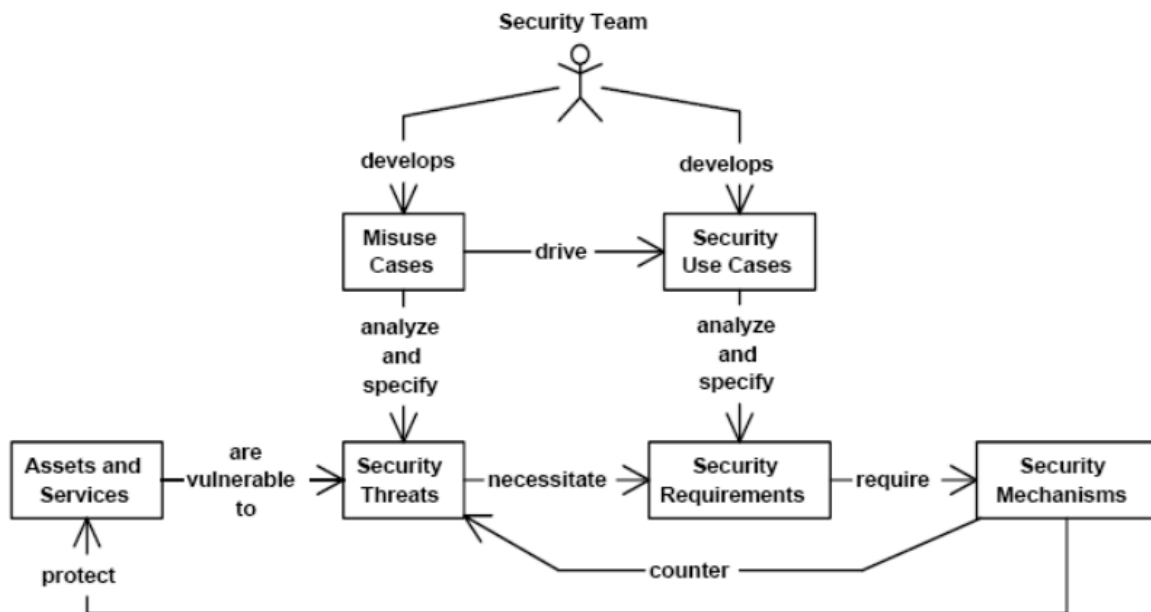
La valutazione del rischio dovrebbe essere parte di tutto il ciclo di vita del software: dall'ingegnerizzazione dei requisiti al deployment del sistema.

Il risultato della valutazione del rischio è un insieme di decisioni ingegneristiche che influenzano la progettazione o l'implementazione del sistema o limitano il modo in cui esso è usato. Questo porta all'introduzione di nuovi requisiti di sicurezza.

Security use case e misuse case

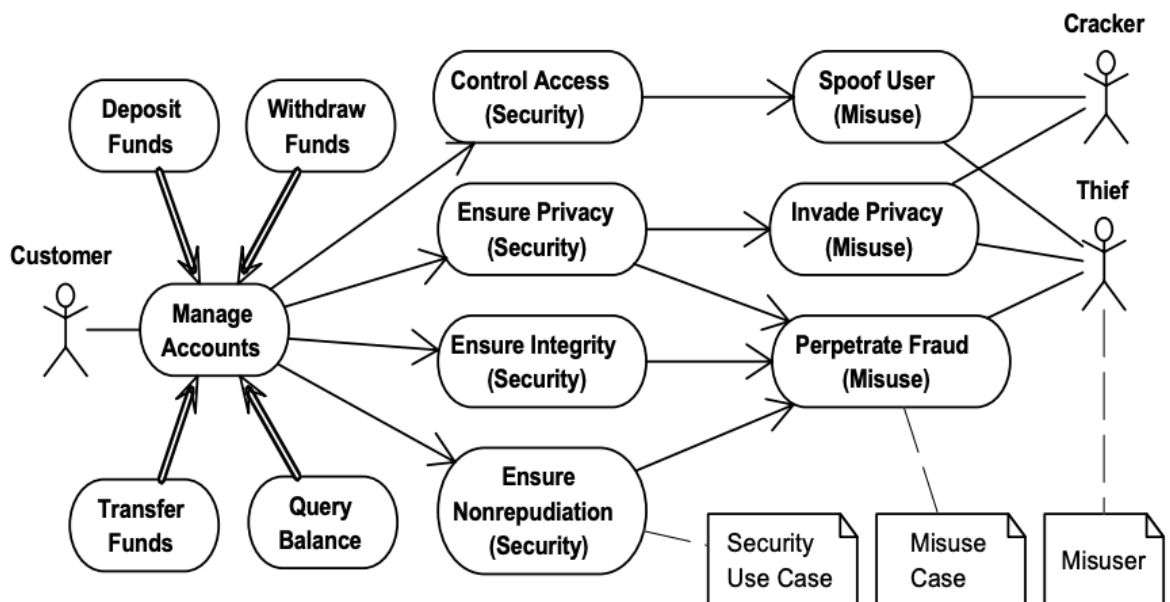
I misuse case si concentrano sulle interazioni tra l'applicazione e gli attaccanti che cercano di violarla. La condizione di successo di un misuse case è l'attacco andato a buon fine.

È invece compito dei security use case specificare i requisiti tramite i quali l'applicazione dovrebbe essere in grado di proteggersi dalle minacce.



	Misuse Case	Security Use Case
Uso	Analizzare e specificare le minacce	Analizzare e specificare i requisiti di sicurezza
Criteri di successo	Successo attaccanti	Successo applicazione
Prodotto da	Security Team	Security Team
Usato da	Security Team	Requirements Team
Attori Esterni	Attaccanti, Utenti	Utenti
Guidato da	Analisi vulnerabilità dei beni e analisi minacce	Misuse case

Un esempio di utilizzo di Security e Misuse case è il seguente:



I casi d'uso non dovrebbero mai specificare meccanismi di sicurezza e vincoli progettuali, le quali decisioni devono essere lasciate alla progettazione.

Specifica dei requisiti di sicurezza

Quasi sempre questa tipologia dei requisiti è espressa nella forma "non deve", non è sempre possibile specificare i requisiti associati alla sicurezza in modo quantitativo. Definisce comportamenti inaccettabili per il sistema, non definisce funzionalità richieste al sistema.

Categorie di requisiti di sicurezza

- Requisiti di identificazione: specificano se un sistema deve identificare gli utenti prima di interagire con loro.
- Requisiti di autenticazione: specificano come identificare gli utenti.
- Requisiti di autorizzazione: specificano i privilegi e i permessi di accesso degli utenti identificati.
- Requisiti di immunità: specificano come il sistema deve proteggersi da virus, worm e minacce simili.
- Requisiti di integrità: specificano come evitare la corruzione dei dati.
- Requisiti di scoperta delle intrusioni: specificano quali meccanismi utilizzare per scoprire gli attacchi al Sistema.
- Requisiti di non-ripudiazione: specificano che una parte interessata in una transazione non può negare il proprio coinvolgimento.
- Requisiti di riservatezza: specificano come deve essere mantenuta la riservatezza delle informazioni.
- Requisiti di controllo della protezione: specificano come può essere controllato e verificato l'uso del sistema.
- Requisiti di protezione della manutenzione del sistema: specificano come un'applicazione può evitare modifiche autorizzate da un accidentale annullamento dei meccanismi di protezione.

▼ 4.0 - Analisi del problema

Introduzione

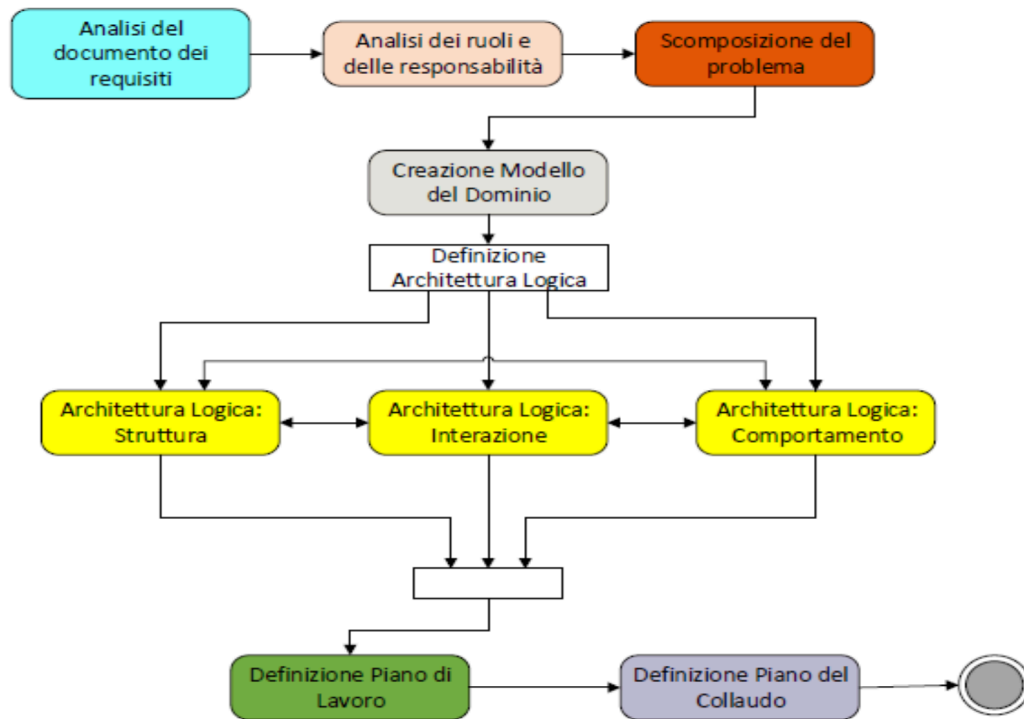
Obiettivo: esprimere fatti il più possibile oggettivi sul problema focalizzando l'attenzione su sottosistemi, ruoli e responsabilità insiti negli scenari prospettati durante l'analisi dei requisiti senza descrivere la sua possibile soluzione.

Capire come strutturare a grandi linee l'applicazione senza dire quali tecnologie utilizzare.

Risultato:

- Architettura logica
- Piano di lavoro

- Piano del collaudo



1. Analisi del Documento dei Requisiti
Obiettivo: concentrarsi sull'analisi delle funzionalità e dei rischi evidenziati nel documento dei requisiti.
2. Analisi dei Ruoli e delle Responsabilità
Obiettivo: analizzare bene i ruoli emersi nei casi d'uso e porre particolare attenzione nell'attribuzione delle responsabilità a tali ruoli tenendo conto dell'analisi del rischio.
3. Scomposizione del problema
Obiettivo: se possibile suddividere il problema in sotto-problemi più piccoli.
4. Creazione del Modello del Dominio
Obiettivo: partendo dal vocabolario si costruisce il diagramma delle classi del dominio.
5. Architettura Logica: Struttura
Obiettivo: creazione dei diagrammi strutturali (package e classi) dell'Architettura Logica.

6. Architettura Logica: Interazione

Obiettivo: creazione dei diagrammi di interazione (diagrammi di sequenza) dell'Architettura Logica.

7. Architettura Logica: Comportamento

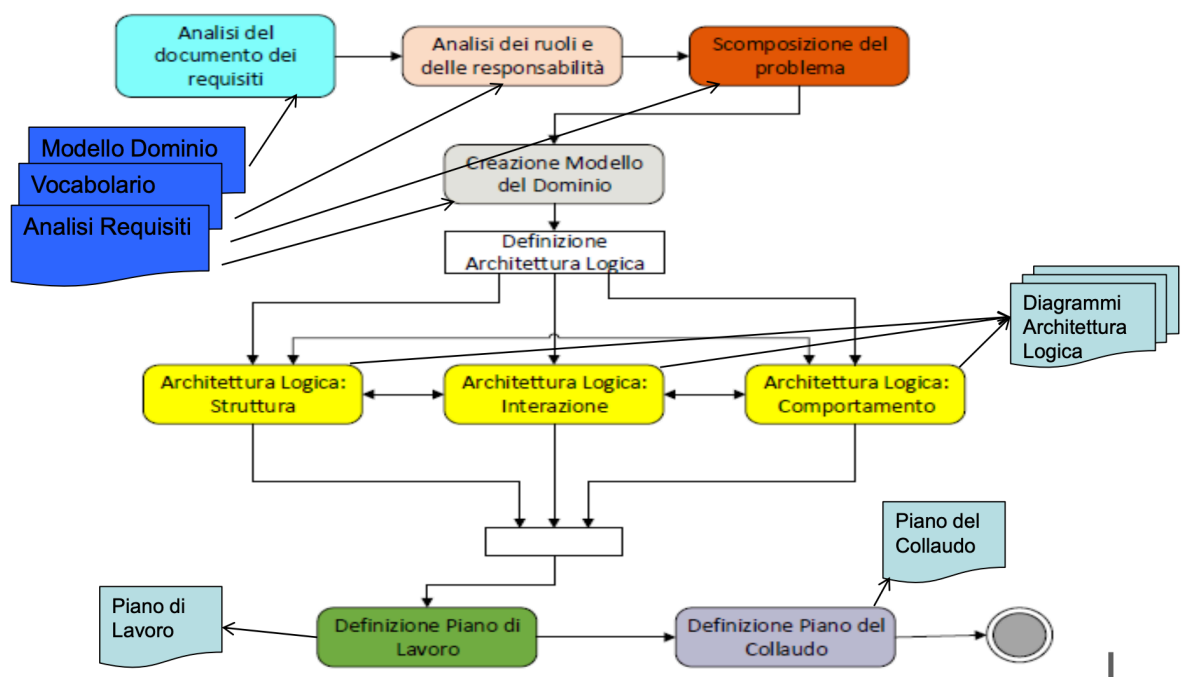
Obiettivo: creazione dei diagrammi di comportamento (stato e attività) dell'Architettura Logica.

8. Definizione del Piano di Lavoro

Obiettivo: assegnare le responsabilità a ciascun membro del team di progetto, stabilire i vincoli temporali, impostare eventuali milestone, ecc.

9. Definizione del Piano del Collaudo

Obiettivo: definire i risultati attesi da ogni entità (classe, sottosistema) che compare nell'Architettura Logica.



Architettura logica

L'Architettura Logica è un insieme di modelli UML costruiti per definire una struttura concettuale del problema robusta e modificabile.

Attraverso l'Architettura Logica l'analista descrive:

- la struttura (insieme delle parti)

- il comportamento atteso (da tutto e da ciascuna parte)
- le interazioni

così come possono essere dedotte dai requisiti, dalle caratteristiche del problema e del dominio applicativo senza alcun riferimento alle tecnologie realizzative.

Piano di lavoro

Il Piano di Lavoro esprime l'articolazione delle attività in termini di risorse umane, temporali, di elaborazione, ecc. necessarie allo sviluppo del prodotto in base ai risultati dell'Analisi del Problema.

Piano del collaudo

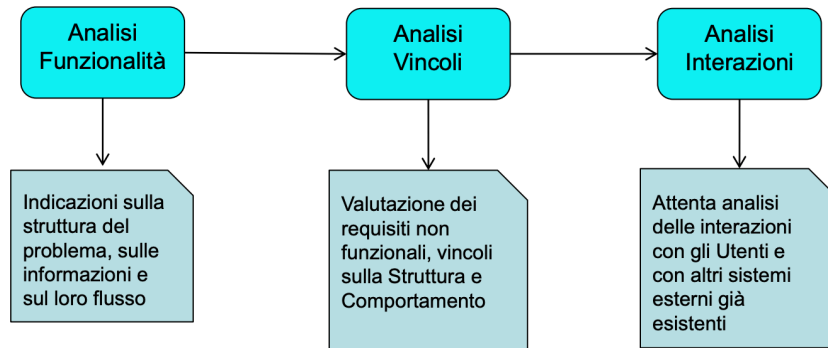
Il Piano del Collaudo definisce l'insieme dei risultati attesi da ogni entità definita nell'Architettura Logica in relazione a specifiche sollecitazioni stimolo-risposta prevista. Un modo per impostare il Piano del Collaudo è quello di fornire i test delle classi definite nell'Architettura Logica, così da pianificare già in questa fase i test di integrazione dei vari sottosistemi.

Analisi documento dei requisiti

Il Documento dei Requisiti evidenzia:

- le funzionalità / i servizi che dovranno essere sviluppati (requisiti funzionali).
- i vincoli che dobbiamo tenere in considerazione (requisiti non funzionali).
- il "mondo esterno" con cui si dovrà interagire.
- i "rischi" legati a possibili attacchi alla protezione, integrità e privacy dei dati (requisiti di sicurezza).

Partendo da tale documento occorre applicare un'attenta analisi delle singole funzionalità, vincoli, interazioni e rischi.



Analisi delle funzionalità

Per ogni funzionalità espressa nei Casi d'Uso vanno analizzati:

- il tipo, ovvero l'obiettivo della funzionalità: memorizzazione dei dati, interazione con l'esterno, gestione/manipolazione dei dati.

Etichettiamo la funzionalità in modo da rendere evidente durante la creazione dell'Architettura Logica dove posizionarla. Se la funzionalità è molto complessa potrebbe appartenere a più tipi differenti: marcare la funzionalità come "complessa".

- le informazioni coinvolte: analisi sistematica di tutte le informazioni sulle quali deve "operare" la funzionalità. In particolare individuare il tipo di dato (composto/singolo) e il grado di riservatezza richiesto.
- il "flusso delle informazioni": quali sono le informazioni che:
 - si ricevono in input: permette di dare indicazioni sulla validazione dell'input e se sono presenti vincoli sui valori ammessi.
 - vanno prodotte in output: permette di valutare dall'esterno se la funzionalità si comporta come ci si aspetta.

Si possono predisporre diverse tabelle di analisi:

- Tabella funzionalità (una sola tabella per tutte le funzionalità)
- Tabella informazioni/flusso (una tabella per ogni funzionalità)

Tabella Funzionalità

Funzionalità	Tipo	Grado Complessità	Requisiti Collegati
nome funzionalità come compare nei casi d'uso	tipo della funzionalità	indicare se complessa o semplice	Requisiti a cui è collegata la funzionalità

Tabella Informazioni / Flusso

Informazione	Tipo	Livello di riservatezza/ privacy	Input/Output	Vincoli
nome (o id)	composto/ semplice	grado di riservatezza richiesto	specificare se input o output	eventuali vincoli sui valori attesi

Analisi dei vincoli

Per ogni requisito non funzionale vanno analizzati:

- il tipo, ovvero a quale categoria appartiene: vincoli sulle performance, sui tempi di risposta, sulla scalabilità, sull'usabilità, sulla protezione dei dati, ecc.

Etichettiamo il requisito in modo da rendere evidente la categoria. Alcuni requisiti potrebbero influenzare diversi aspetti del sistema, nel caso indicarli tutti, insieme anche al tipo di impatto (es: portano ad un peggioramento).

- le funzionalità coinvolte: indicare le funzionalità che vengono influenzate dal requisito.

Si può utilizzare una tabella dei vincoli (una sola tabella per tutti i vincoli):\

Requisito	Categorie	Impatto	Funzionalità
Requisito non funzionale	tipo	indicare il tipo di impatto	funzionalità coinvolte

Analisi delle interazioni

Vanno distinte le interazioni con gli umani da quelle con sistemi esterni.

Nel caso delle interazioni con gli umani:

- analizzare le eventuali interfacce identificate con il cliente nella fase di Analisi dei Requisiti, oppure delineare le possibili interfacce.
- individuare le maschere di inserimento/output dati.
- individuare le sole informazioni necessarie da mostrare in ogni maschera.
- creare un legame tra maschere – informazioni – funzionalità.

Si può predisporre una tabella delle maschere (una sola tabella per tutte le maschere):

Maschera	Informazioni	Funzionalità
Nome della maschera	indicare le informazioni gestite nella maschera	funzionalità coinvolte

Nel caso delle interazioni con sistemi esterni occorre analizzare i sistemi esterni con cui si dovrà interagire e individuare i protocolli di interazione con tali sistemi.

Si utilizza una tabella dei sistemi esterni (una sola tabella per tutti i sistemi esterni):

Sistema	Descrizione	Protocollo di Interazione	Livello di Protezione
Nome del sistema	Descrizione del sistema e delle sue principali funzionalità	Specificare il protocollo di interazione richiesto	Specificare il livello di protezione garantito dal sistema

Analisi ruoli e responsabilità

Per ogni Attore individuato nei Casi d'Uso specificare:

- le responsabilità – cosa deve fare in ogni funzionalità.
- specificare le informazioni a cui può accedere relativamente a ciascuna funzionalità in cui è coinvolto con relativa indicazione del tipo di accesso.
- le maschere che può visualizzare.
- il suo livello di riservatezza – quale livello di riservatezza è necessario avere per poter ricoprire quel ruolo.

- la numerosità attesa – il numero di persone che possono giocare quel ruolo.

Si possono predisporre due tabelle:

- Tabella ruoli (una sola tabella per tutti i ruoli)
- Tabella ruolo-informazioni (una tabella per ogni ruolo)

Tabella Ruoli

Ruolo	Responsabilità	Maschere	Riservatezza	Numerosità
Nome del ruolo	Indicare le responsabilità assegnate	Indicare le maschere che devono essere visualizzate	Livello di riservatezza necessaria	Indicare la numerosità massima

Tabella Ruolo-Informazioni

Informazione	Tipo Accesso
Informazione	Specificare il tipo di accesso

Scomposizione del problema

Il punto di partenza è l'Analisi delle Funzionalità. Per ogni funzionalità marcata come "complessa" nella Tabella delle Funzionalità occorre valutare se sia possibile operare una scomposizione. Per ogni sotto-funzionalità occorre poi analizzare quale legame sussiste con le altre e quali informazioni devono fluire tra queste.

Si possono predisporre due tabelle:

- Tabella scomposizione funzionalità
- Tabella sotto-funzionalità

Tabella Scomposizione Funzionalità

Funzionalità	Scomposizione
nome funzionalità	elenco delle sotto-funzionalità

Tabella Sotto-Funzionalità

Sotto-Funzionalità	Sotto-Funzionalità	Legame	Informazioni
nome sotto-funzionalità	nome sotto-funzionalità	specificare il tipo di dipendenza/legame logico	specificare le informazioni scambiate

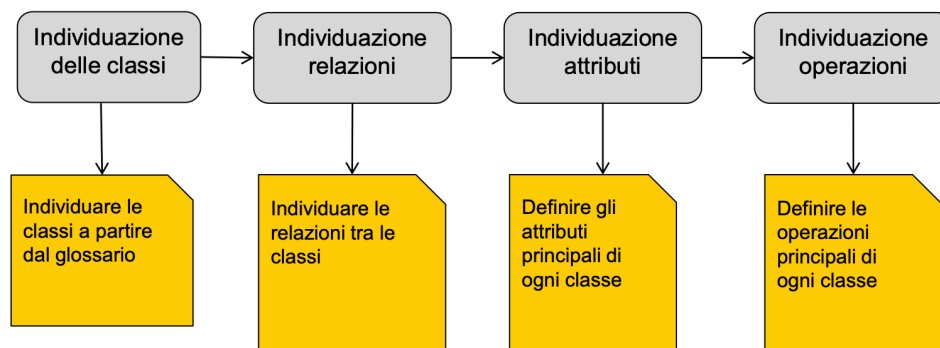
Creazione modello del dominio

Il punto di partenza è costituito dall'insieme di:

- glossario definito nell'Analisi dei Requisiti
- tabelle informazioni/flusso

Sulla base di questi si costruisce il diagramma delle classi che rappresenta il Modello del Dominio (non tutti i vocaboli elencati nel glossario diventeranno classi). Tale modello poi sarà riusato nell'Architettura Logica come modello dei dati.

Il modello del dominio sarà costruito individuando le classi rilevanti per il problema, le relazioni tra le classi e, per ogni classe, gli attributi e le operazioni fornite all'esterno.



Individuazione delle classi

Dal glossario eliminare i nomi che sicuramente non si riferiscono a classi, indicano attributi (dati di tipo primitivo) o indicano operazioni. Scegliere un solo termine significativo se più parole indicano lo stesso concetto (sinonimi).

Per determinare se includere una classe nel modello occorre chiedersi se il sistema deve interagire in qualche modo con gli oggetti della classe, utilizzando informazioni (attributi) contenute in essi o utilizzare i servizi (operazioni) offerti.

Individuazione delle relazioni

In ogni tipo di relazione, esiste un cliente C che dipende da un fornitore di servizi F. C ha bisogno di F per lo svolgimento di alcune funzionalità che C non è in grado di effettuare autonomamente.

Tipo di relazione	Cliente	Fornitore
Ereditarietà	sottoclasse	superclasse
Associazione (composizione / aggregazione)	contenitore	contenuto
Dipendenza	classe dipendente (source)	classe da cui si dipende (target)

Un'associazione può

- Rappresentare un contenimento logico (aggregazione)
Un oggetto x di classe X è associato a un oggetto y di classe Y in modo non esclusivo, ovvero x può condividere y con altri oggetti anche di tipo diverso.
Es. una lista d'esame contiene degli studenti
- Rappresentare un contenimento fisico (composizione)
Un oggetto x di classe X è associato a un oggetto y di classe Y in modo esclusivo, ovvero y esiste solo in quanto contenuto in x.
Es. un triangolo contiene tre vertici
- Non rappresentare un reale contenimento (collaborazioni)
Una classe A è in relazione USA con una classe B (A USA B) quando A ha

bisogno della collaborazione di B per lo svolgimento di alcune funzionalità che A non è in grado di effettuare autonomamente.

Es. una fattura si riferisce a un cliente

Le associazioni possono nascondere una classe (classe di associazione). Es. la connessione "matrimonio" tra Persona e Persona può nascondere una classe Matrimonio.

Individuazione degli attributi

La base di partenza per la ricerca degli attributi sono le Tabelle di Informazione/Flusso. Ogni attributo modella una proprietà atomica di una classe. Proprietà non atomiche devono essere modellate come associazioni.

Nel caso di attributi calcolabili (ad esempio, età), specificare sempre l'operazione di calcolo mai l'attributo. Se memorizzare oppure no un attributo calcolabile è una decisione progettuale.

Individuazione delle operazioni

- Operazioni standard
Operazioni che tutti gli oggetti hanno per il semplice fatto di esistere e di avere degli attributi e delle relazioni con altri oggetti (costruttore, aggiungi, rimuovi, conta, ecc.). Sono implicite e, di norma, non compaiono nel diagramma delle classi di analisi.
- Altre operazioni
Rappresentano servizi offerti agli altri oggetti, compaiono nel diagramma delle classi di analisi.

Mettere ogni operazione "vicino" ai dati a essa necessari. Inoltre occorre posizionare le operazioni più generali più in alto possibile nella gerarchia, mentre quelle specializzate più in basso.

Architettura Logica: Struttura

La parte strutturale dell'Architettura Logica dovrebbe essere composta di due tipi differenti di diagrammi UML:

- Diagramma dei Package: fornisce una visione di alto livello dell'architettura.

- Diagramma delle classi (uno o più diagrammi in base alla complessità): fornisce una visione più dettagliata del contenuto dei singoli package.

Sarebbe opportuno organizzare sin da subito l'Architettura Logica usando un pattern architetturale chiamato Boundary-Control-Entity (BCE). BCE è un pattern architetturale che suggerisce di basare l'architettura di un sistema sulla partizione sistematica degli use case in oggetti di tre categorie:

- Entity: è la dimensione relativa alle entità cui corrisponde l'insieme delle classi che includono funzionalità relative alle informazioni che caratterizzano il problema.
- Boundary: è la dimensione relativa alle funzionalità che dipendono dall'ambiente esterno cui corrisponde l'insieme delle classi che incapsulano l'interfaccia del sistema verso il mondo esterno.
- Control: è la dimensione relativa agli enti che incapsulano il controllo.

L'architettura di un sistema software risulta quasi fisiologicamente articolata in una sequenza di livelli (layer) verticali che viene tipicamente mantenuta anche in fase di progetto e implementazione. Nello specifico vi sono 3 livelli:

- Il livello di applicazione
Comprende le parti che provvedono a elaborare l'informazione di ingresso, a produrre i risultati attesi e a presentare le informazioni in uscita.
- Il livello di presentazione
Comprende le parti che realizzano l'interfaccia utente. Per aumentare la riusabilità delle parti, questo livello è progettato e costruito astraendo quanto più possibile dai dettagli degli specifici dispositivi di I/O.
- Il livello delle entità
Forma il (modello del) dominio applicativo.

Diagramma dei package

1. Il primo package che si può identificare è il package costituito dal Modello del Dominio creato nella fase precedente tale Modello costituisce la parte entity dell'architettura.
2. Poi è possibile creare un package per ognuna delle diverse funzionalità identificate nella Tabella delle Funzionalità control.

3. Vengono creati uno o più package per la parte di boundary.
4. Infine si identificano le dipendenze logiche tra i package.

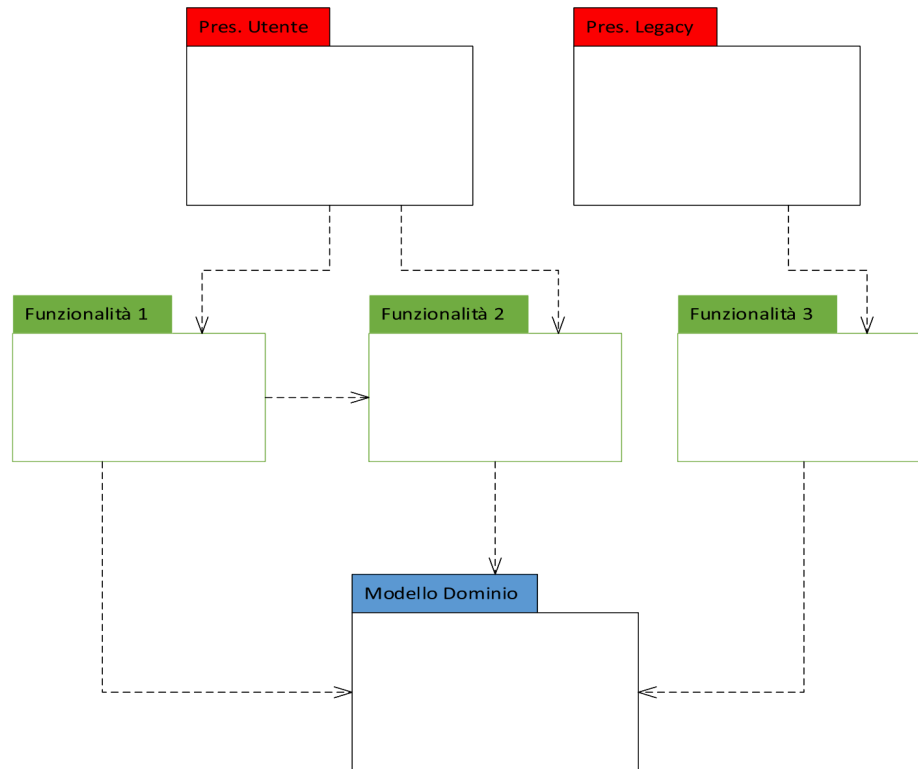


Diagramma delle classi

Dopo aver stabilito la struttura di alto livello dell'Architettura Logica si dettagliano le classi che compongono ogni package. Se le classi da specificare per ogni package sono poche si può realizzare un unico diagramma delle classi. Altrimenti è possibile creare un diagramma delle classi separato per ogni package.

Attenzione a non introdurre scelte di progettazione in questa fase. Indicare solo quelle classi che sono deducibili dal problema.

Architettura logica: interazione

Descrivere le interazioni tra le entità identificate nella parte strutturale attraverso opportuni diagrammi di sequenza, i quali evidenziano:

- lo scambio di messaggi tra gli oggetti

- l'ordine in cui i messaggi vengono scambiati tra gli oggetti

Architettura logica: comportamento

Descrivere il comportamento delle entità identificate nella parte strutturale attraverso opportuni Diagrammi di Stato o delle Attività, senza scendere troppo nei dettagli.

Diagramma di Stato per mostrare come si comportano alcune entità complesse a seguito delle interazioni e degli eventi che avvengono nel sistema. Diagramma delle Attività per dettagliare funzionamenti complessi delle entità.

Definizione del piano di lavoro

Dopo la creazione dell'Architettura Logica è possibile iniziare a suddividere il lavoro. In particolare è necessario:

- suddividere le responsabilità ai diversi membri del team di progetto e sviluppo.
- stabilire le tempistiche per la progettazione di ciascuna parte
- stabilire i tempi di sviluppo di ciascun sotto-sistema
- programmare i test di integrazione tra le parti
- identificare i tempi di rilascio delle diverse versioni del prototipo
- identificare un piano per gli sviluppi futuri

Definizione del piano del collaudo

Lo scopo del piano del collaudo è cercare di precisare il comportamento atteso da parte di una entità prima ancora di iniziarne il progetto e la realizzazione.

Un piano del collaudo va concepito e impostato da un punto di vista logico, cercando di individuare categorie di comportamenti e punti critici. In molti casi tuttavia può anche risultare possibile definire in modo precoce piani di collaudo concretamente eseguibili, avvalendosi di strumenti del tipo JUnit/NUnit che sono ormai diffusi in tutti gli ambienti di programmazione.

JUnit è framework per unit-testing per il linguaggio Java, mentre NUnit per tutti i linguaggi .Net.

▼ 5.0 - Progettazione

Obiettivo: attraverso una serie di raffinamenti successivi dell'Architettura Logica arrivare ad ottenere l'Architettura del Sistema. Questa fase deve mirare non solo a individuare e descrivere una soluzione al problema (what/how), ma soprattutto a descrivere i motivi (why) che l'hanno determinata.

Risultato:

- Architettura del Sistema
- Schema Persistenza
- Piano finale del Collaudo
- Indicazioni per il Deployment

Fasi della progettazione



1. Progettazione Architeturale

Obiettivo: definire l'Architettura del Sistema tenendo conto di tutti i vincoli e delle forze in gioco.

2. Progettazione di Dettaglio

Obiettivo: progettare nel dettaglio ogni aspetto del Sistema.

3. Progettazione della Persistenza

Obiettivo: progettare i meccanismi per la persistenza dei dati.

4. Progettazione del Collaudo

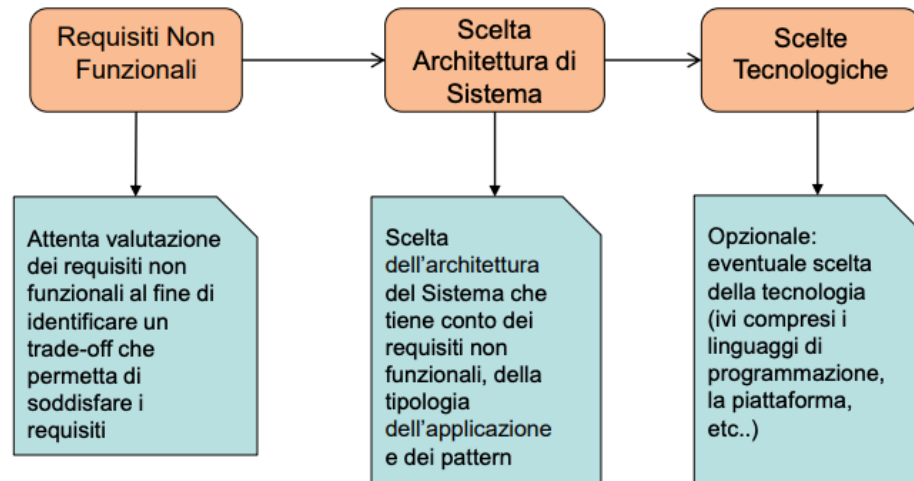
Obiettivo: definire in modo chiaro e preciso come il sistema dovrà essere collaudato una volta terminata l'implementazione.

5. Progettazione per il Deployment

Obiettivo: progettare il sistema in modo da rendere semplice il deployment sulle macchine e per garantire la sicurezza.

Progettazione architetturale

Nella Progettazione Architetturale gli ingegneri devono prendere delle decisioni che influenzano profondamente il sistema.



La struttura dell'architettura tipicamente è condizionata dalla tipologia di applicazione che si vuole realizzare e dai requisiti non funzionali.

Requisiti non funzionali

- prestazioni → localizzare le operazioni critiche all'interno di un piccolo numero di componenti e minimizzare le comunicazioni possibili tra essi.

Questo porta a dover definire componenti grandi per ridurre la comunicazione.

- protezione dei dati → struttura stratificata con alto livello di convalida di protezione a ogni strato, collocando le risorse più critiche nello strato più interno e protetto.
- sicurezza → operazioni relative devono essere tutte collocate in un singolo componente o in un piccolo insieme di componenti. Questo porta a dover definire componenti grandi per localizzare le operazioni.
- disponibilità → componenti ridondanti in modo che sia possibile sostituirli e aggiornarli senza fermare il sistema.
- manutenibilità → componenti piccoli, atomici, autonomi che possono essere modificati velocemente. Produttori di informazione devono essere separati dai consumatori e devono essere evitate strutture dati condivise. Questo porta a dover sviluppare componenti di piccole dimensioni.

Alcuni di questi requisiti, come abbiamo visto, vanno in conflitto, per questo a volte occorre trovare un compromesso.

Scelta Architettura di Sistema

La scelta dell'architettura deve basarsi su:

- Architettura Logica definita in fase di Analisi del Problema
- Trade-off requisiti non funzionali
- Adozione di Pattern Architeturali

- Blackboard

Il pattern Blackboard aiuta a strutturare quelle applicazioni in cui vengono applicate strategie di soluzione non deterministiche (tipici problemi di intelligenza artificiale). I diversi sotto-sistemi condividono la stesse conoscenze attraverso la Blackboard (lavagna che viene popolata dalle conoscenze dei singoli componenti) al fine di costruire una soluzione approssimata o parziale.

- MVC/BCE

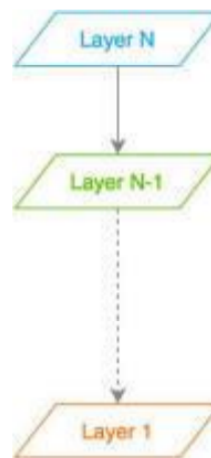
Il pattern MVC divide le applicazioni in tre distinte parti:

- Model: gestisce i dati
- Controller: manipola i dati
- View: mostra i dati



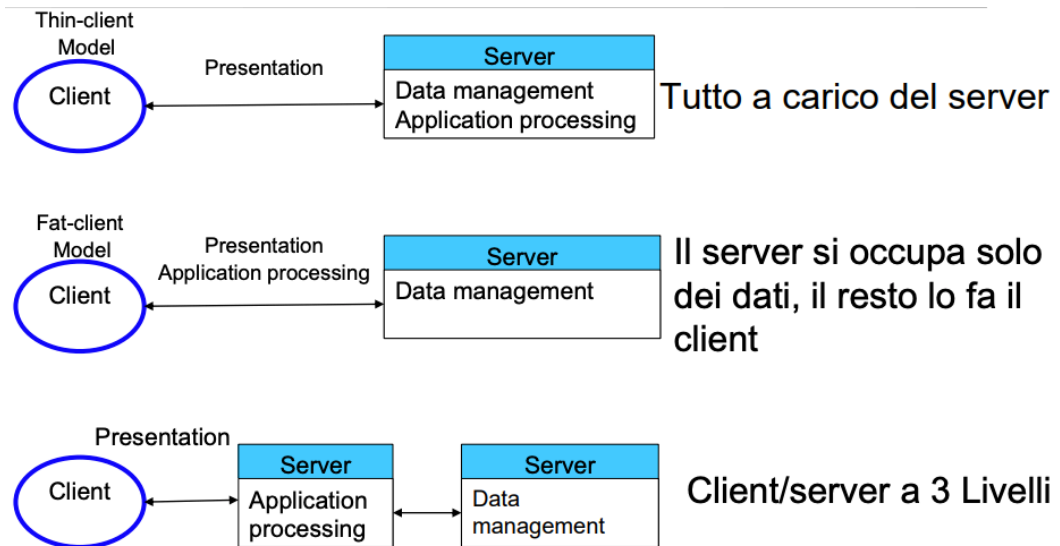
- Layer

Il pattern Layer aiuta a strutturare quelle applicazioni che possono essere scomposte in gruppi di sotto-attività in cui ciascun gruppo si trova a un ben definito livello di astrazione.



- Client/Server

Il pattern client/server aiuta a strutturare un'applicazione come un insieme di servizi forniti da uno o più server e un insieme di client che utilizza tali servizi. Ci sono diversi tipi di client/server:

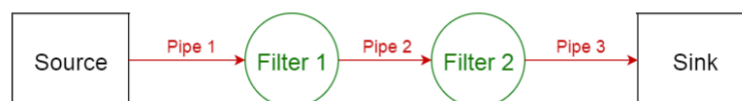


- Broker

Il pattern Broker può essere usato per strutturare sistemi distribuiti con un disaccoppiamento tra i diversi sotto-sistemi che comunicano tra loro attraverso remote server invocation. Il Broker è responsabile della coordinazione delle comunicazioni, come inoltrare richieste, inviare risposte ed eccezioni.

- Pipe & Filters

Il pattern Pipe & Filters aiuta a strutturare quelle applicazioni che processano flussi di dati. Ogni passo del processo è incapsulato in un apposito filtro e i dati attraversano una pipe di filtri.



Talvolta è necessario usare stili architetturali diversi per parti diverse del sistema al fine di soddisfare tutti i vincoli imposti dai requisiti.

Progettazione di Dettaglio

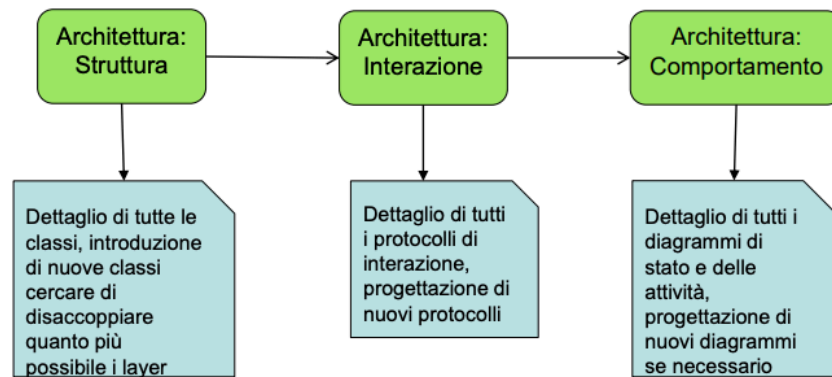
La Progettazione di Dettaglio definisce il dettaglio dell'Architettura del Sistema nelle sue tre viste:

- Struttura

- Interazione
- Comportamento

Durante la Progettazione di Dettaglio, i modelli prodotti nell'Analisi devono essere modificati al fine di definire in dettaglio le classi e delle loro relazioni, supportare caratteristiche specifiche per comunicazioni, diagnostica, protezione dei dati, ..., riuso di classi e/o componenti disponibili, miglioramento delle prestazioni, supporto alla portabilità, ...

È importante comunque mantenere la massima indipendenza possibile da linguaggio di programmazione, DBMS, sistema operativo, hardware, ecc. (tenerle in conto solo se vincolanti).



Architettura: Struttura

Durante la Progettazione di Dettaglio della parte di Struttura è necessario definire

- tipi di dato che non sono stati definiti in precedenza
- strutture dati necessarie per l'effettiva implementazione del sistema
- operazioni che non erano emerse durante la fase di Analisi del Problema
- eventuali nuove classi necessarie per il corretto funzionamento del sistema

Occorre effettuare un'attenta analisi e valutazione del livello di protezione offerto dalle tecnologie scelte, e nel caso in cui tale livello non risulta

essere quello minimo richiesto dall'applicazione occorre progettare specifiche parti del sistema per prevenire i buchi di sicurezza. Ove possibile occorre cercare di applicare il pattern Adapter.

Applicazione dei principi di progettazione con particolare attenzione al "Dependency Inversion Principle" al fine di poter modificare implementazione, aspetto grafico e aggiungere nuove funzionalità avendo un impatto minimo sul sistema.

Architettura: Interazione

Durante la Progettazione di Dettaglio della parte di Interazione è necessario:

- ridefinire i protocolli di interazione emersi in fase di Analisi dettagliandoli tenendo conto delle nuove entità emerse in progettazione.
- progettare accuratamente i protocolli di interazione verso i sistemi esterni.
- definire nuovi protocolli di interazione tra le classi che sono state introdotte nella progettazione.

Architettura: Comportamento

Durante la progettazione di dettaglio della parte di Comportamento è necessario:

- definire gli algoritmi che implementano le operazioni complesse/complicate in modo chiaro e preciso avvalendosi eventualmente di diagrammi delle attività.
- dettagliare i diagrammi di stato/attività già definiti nella fase precedente.
- eventualmente aggiungere diagrammi di stato/attività per le nuove entità emerse in questa fase.

Progettazione della persistenza

Il progettista dopo un'attenta valutazione di vincoli (tempi di risposta, requisiti di protezione e privacy, ...), tipologia di accesso (lettura, scrittura,

ricerche), criticità e consistenza dei dati (quanto spesso cambiano i dati? quali sono i costi di eventuale perdite nelle modifiche dei dati?), dovrà scegliere la tecnica migliore di persistenza.

Non è sempre detto che l'adozione di un (R)DBMS sia la risposta corretta. Per esempio se dobbiamo memorizzare dei log la strategia migliore è quella delle scrittura su file. In generale possiamo affermare che quando si ha a che fare con un numero considerevole di dati anche di natura eterogenea, che cambiano molto spesso, la quale perdita di modifiche può essere un problema, necessità di ripristino di versioni precedenti a seguito di un malfunzionamento, la scelta consigliata è quella di avvalersi di un DBMS.

L'output di questa fase può essere rappresentato da:

- lo schema E-R del DB che dovrà supportare l'applicazione
- il formato del/i file che dovranno essere scritti/letti dall'applicazione

Progettazione Concettuale (E/R)

Il primo passo è quello della raccolta dei requisiti relativi alla base di dati. Un concetto presente in tali requisiti può venir rappresentato come:

- Entità, se ha proprietà significative e descrive oggetti con esistenza autonoma.
- Attributo, se è semplice e non ha proprietà.
- Associazione, se correla due o più concetti.
- Generalizzazione/specializzazione, se è caso più generale/particolare di un altro.

Per affrontare progetti complessi è opportuno adottare uno specifico modo di procedere, ovvero una strategia di progettazione. Le strategie più notevoli sono:

- Strategia top-down: si parte da uno schema iniziale molto astratto ma completo, che viene successivamente raffinato fino ad arrivare allo schema finale.
- Strategia bottom-up: si suddividono le specifiche in modo da sviluppare semplici schemi parziali ma dettagliati, che poi vengono

integrati tra loro.

- Strategia inside-out: lo schema si sviluppa "a macchia d'olio", partendo dai concetti più importanti, che quindi vengono espansi aggiungendo quelli a essi correlati, e così via.

Nella pratica si fa spesso uso di una strategia ibrida. Tale strategia prevede:

- Analisi dei requisiti
 - Analizzare i requisiti ed eliminare le ambiguità
 - Costruire un glossario dei termini, raggruppare i requisiti
- Passo base
 - Definire uno schema scheletro con i concetti più rilevanti
- Passo di decomposizione (se necessario o appropriato)
 - Decomporre i requisiti con riferimento ai concetti nello schema scheletro
- Passo iterativo (da ripetere finché non si è soddisfatti)
 - Raffinare i concetti presenti sulla base delle loro specifiche
 - Aggiungere concetti per descrivere specifiche non descritte
- Passo di integrazione (se si è decomposto)
 - Integrare i vari sottoschemi in uno schema complessivo, facendo riferimento allo schema scheletro
- Analisi di qualità (ripetuta e distribuita)
 - Verificare le qualità dello schema e modificarlo

Lo schema E/R deve essere verificato accuratamente per verificare che risponda a requisiti di:

- Correttezza: non devono essere presenti errori (sintattici o semantici).
- Completezza: tutti i dati di interesse devono essere specificati.
- Leggibilità: riguarda anche aspetti prettamente estetici dello schema.

- Minimalità: è importante capire se esistono elementi ridondanti nello schema; in alcuni casi ciò non è un problema, ma può essere viceversa una scelta di progettazione volta a favorire l'esecuzione di certe operazioni.

Progettazione del collaudo

La base di partenza di questa attività è il Piano del Collaudo sviluppato nell'analisi. Dopo la Progettazione di Dettaglio è possibile scrivere i test unitari di ciascuna classe. Successivamente vanno progettati con cura anche i test di integrazione del sistema.

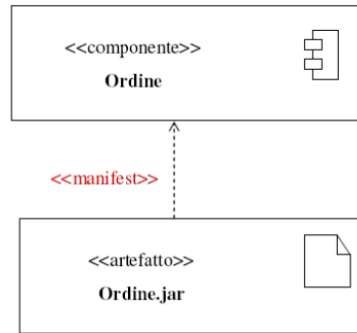
L'output di questa attività è rappresentato dalla Suite completa dei test unitari e di integrazione.

Progettazione per il deployment

I diagrammi di deployment documentano la distribuzione fisica di un sistema, mostrando i vari pezzi di software in esecuzione sulle macchine fisiche. I diagrammi di deployment quindi mostrano i collegamenti che permettono la comunicazione fisica tra i pezzi hardware, le relazioni tra macchine fisiche e processi software, con l'indicazione dei vari punti in cui viene eseguito il codice.

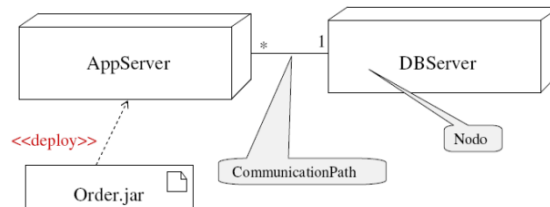
Gli elementi principali del diagramma:

- Artifact
Specifica porzione fisica di informazioni utilizzata o prodotta dal processo di sviluppo del software. Tipicamente viene utilizzata una relazione di dipendenza <<manifest>> che illustra gli elementi di modellazione (in genere, componenti) utilizzati nella costruzione o generazione di un artefatto.



- Node

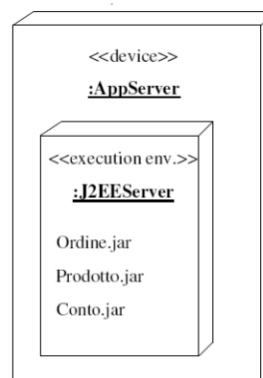
Un'unità sulla quali risiedono e/o sono eseguiti componenti/artefatti. L'allocazione degli artefatti su un nodo viene rappresentata con una relazione di dipendenza «deploy» tra il nodo e l'artefatto.



- Device

È una risorsa fisica computazionale con capacità elaborative sulla quale possono essere allocati artefatti per l'esecuzione.

L'Execution Environment è un nodo che offre l'ambiente per l'esecuzione di specifici tipi di componenti che sono allocati su di esso («OS», «databasesystem», «J2EE container», ...).



Deployment Specification: è un insieme di proprietà che determinano i parametri di esecuzione di un artefatto allocato su un nodo.

<<deployment spec>> Orderdesc.xml
execution: execKind transaction: Boolean

<<deployment spec>> <u>:Orderdesc.xml</u>
execution: thread transaction: true